

UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Software Architecture Design

Documentazione Gruppo D9 Task R2

Migrazione database e aggiunta suggerimenti nel microservizio T1 – AdminService.

Anno Accademico 2025/2026

ID Gruppo:

D9

Id Task:

R2

Componenti del gruppo:

Nome

Saggiomo Luca
Cinque Salvatore
Gargiulo Salvatore
Russo Claudio

E-mail

luc.saggiomo@studenti.unina.it
salva.cinque@studenti.unina.it
salvatore.gargiulo19@studenti.unina.it
claudio.russo10@studenti.unina.it

Matricola

DE9000084
DE9000096
DE9000154
M63001316

Sommario

Documentazione Gruppo D9 Task R2	1
1. Identificazione del Team e definizione degli obiettivi del progetto	6
1.1 Descrizione sintetica del Task Assegnato	6
1.2 Approccio di sviluppo e gestione del progetto	6
2. Analisi dei Requisiti per il Task Assegnato	7
2.1 Storie utente	8
2.2 Elenco dei requisiti	8
2.2.1 Requisiti funzionali	8
2.2.2 Requisiti non funzionali	9
2.3 Modello concettuale dei suggerimenti	9
2.4 Casi d'uso	10
2.4.1 Scenari	10
2.5 Analisi dell'impatto	13
3. Progettazione della soluzione	15
3.1 Scelte di progetto	15
3.1.1 Suggerimenti e Linee Guida	15
Glossario dei termini	15
3.1.2 Modello concettuale di suggerimenti e linee guida	15
3.2 Migrazione del Database	17
3.2.1 Criticità rilevate	18
3.2.2 Modello ER	19
3.2.3 Modello Logico	20
3.3 Refactoring dei Package JAVA del microservizio T1	22
3.3.1 Criticità dell'architettura del sistema	22
3.3.2 DTO	24
3.3.3 Mapper	24
3.3.4 Model	24
3.3.5 Controller	25
3.3.6 Service	25
3.3.7 Repository	25
3.3.8 Security	25

3.3.9	Validation	26
3.3.10	Exception	26
3.4	Class Diagram del package Model	27
3.5	Component Diagrams	30
3.6	REST API	31
3.6.1	Criteri di accettazione	31
3.6.2	Schemi degli input	32
3.6.3	Un esempio	34
3.6.4	Suggestion Controller	36
3.6.5	UploadSuggestions	36
3.6.6	UploadSuggestionImage	38
3.6.7	GetSuggestions	40
3.6.8	DeleteSuggestion	42
3.6.9	DeleteSuggestionImage	44
3.6.10	Guideline Controller	46
3.6.11	UploadGuidelines	46
3.6.12	UploadGuidelineImage	48
3.6.13	GetGuidelines	50
3.6.14	DeleteGuidelines	51
3.6.15	DeleteGuidelineImage	52
3.7	Diagrammi di sequenza	54
3.7.1	UploadSuggestion	54
3.7.2	UploadSuggestionImage	55
3.7.3	GetSuggestions	55
3.7.4	DeleteSuggestion	56
3.7.5	DeleteSuggestionImage	56
4.	Implementazione e struttura del progetto modificato	57
4.1	Refactoring	57
4.1.1	Anti-pattern individuati	57
4.1.2	Gestione delle eccezioni centralizzata	57
4.1.3	Security	57
4.1.4	AuthTokenFilter	57

4.2	Descrizione delle modifiche al microservizio T1	59
4.2.1	File di configurazione	59
4.2.2	Frontend	60
4.2.3	Backend	62
4.2.4	Package api	62
4.2.5	Package config	62
4.2.6	Package validation	63
4.2.7	Package util	63
4.2.8	Package security	63
4.2.9	Package model	63
4.2.10	Package repository	65
4.2.11	Package service	66
4.2.12	Package controller	67
4.2.13	Package controller.view	68
4.2.14	Package dto	68
4.2.15	Package mapper	68
4.2.16	Package exception	68
5.	Deployment diagram	69
6.	Descrizione dei Test effettuati	71
6.1	Unit testing	71
6.1.1	Controller	71
	SuggestionController	71
	GuidelineController	74
6.1.2	Service	76
	SuggestionService	76
	GuidelineService	80
6.1.3	ImageService	82
6.1.4	Repository	84
	SuggestionRepository	84
	GuidelineRepository	85
	AdminRepository	86
	AssignmentRepository	87

ScalataRepository	88
TeamRepository	88
ClassUTRepository	89
InteractionRepository	90
OperationRepository	90
OpponentRepository	91
6.2 Integration testing	91
6.2.1 Postman	91
UploadSuggestions	91
GetSuggestions	96
DeleteSuggestion	97
UploadGuidelines	98
GetGuidelines	100
DeleteGuideline	101
UploadSuggestionImage	102
UploadGuidelineImage	103
DeleteSuggestionImage	103
DeleteGuidelineImage	104
7. Possibili integrazioni	104

1. Identificazione del Team e definizione degli obiettivi del progetto

- ID team: D9
- Componenti del gruppo

Nome	Email	Matricola
Saggiomo Luca @	luc.saggiomo@studenti.unina.it	DE9000084
Cinque Salvatore	salva.cinque@studenti.unina.it	DE9000096
Gargiulo Salvatore	salvatore.gargiulo19@studenti.unina.it	DE9000154
Russo Claudio	claudio.russo10@studenti.unina.it	M63001316

- URL della Versione di Partenza sul repository
https://github.com/Testing-Game-SAD-2023/A13/tree/D9_TaskD9_CaricamentoSuggerimenti_MigrazioneMongoDB
- URL del Repository del progetto consegnato
https://github.com/lucasaggiomo/A13/tree/D9_TaskD9_CaricamentoSuggerimenti_MigrazioneMongoDB

1.1 Descrizione sintetica del Task Assegnato

Il task R2 presenta due principali interventi sul microservizio T1.

- Il primo riguarda la migrazione del database da uno non relazionale, MongoDB, ad uno relazionale. La scelta iniziale di progetto di adottare un database non relazionale influiva negativamente sulla facilità di modifica ed evoluzione del servizio, data la poca leggibilità delle query Mongo, specialmente per chi proviene da SQL, e la mancanza di vincoli relazionali e ottimizzazioni di query e transazioni. La nuova scelta di progetto, ossia quella di migrare ad un database relazionale, è stata proprio guidata da tali requisiti. Un altro vincolo fondamentale è quello di implementare la migrazione in modo tale che essa impatti il meno possibile sulla gestione della persistenza del microservizio.
- Il secondo, invece, riguarda l'aggiunta di nuove funzionalità per l'admin in T1 per permettergli di inserire dei suggerimenti, utili agli studenti durante le partite, tramite un file di testo con formato scelto correttamente.

1.2 Approccio di sviluppo e gestione del progetto

Il progetto è stato sviluppato adottando un approccio Agile di tipo iterativo. Lo sviluppo è stato organizzato in tre iterazioni, ciascuna delle quali aveva come obiettivo la realizzazione di un incremento funzionante del sistema, inizialmente limitato nelle funzionalità ma progressivamente esteso nelle iterazioni successive.

Nella prima iterazione sono stati individuati i requisiti relativi alle nuove funzionalità del sistema da implementare (legati alla gestione dei Suggerimenti), tramite user stories e

diagramma dei casi d'uso. È stato inoltre effettuato uno studio del database MongoDB già presente nel sistema, in modo da individuare tutte le entità coinvolte e le loro relazioni. L'iterazione è terminata con la realizzazione di un modello concettuale del database e con una prima modifica del codice, tramite l'aggiornamento delle classi Model con l'adozione delle annotazioni di Jakarta.

Nella seconda iterazione del sistema sono state apportate modifiche al database, a seguito di considerazioni effettuate nella prima review del progetto, come la scelta di distinguere i suggerimenti validi in generale per tutte le classi (definiti Linee Guida) e i suggerimenti associati ad una classe specifica (definiti Suggerimenti) e l'aggiunta di un'immagine eventuale associata ai suggerimenti. Sono state quindi sviluppate le funzionalità richieste nelle classi Service riguardo le operazioni da effettuare con i suggerimenti e le linee guida, a partire dai diagrammi comportamentali delineati, insieme all'aggiunta degli endpoint intercettati dai Controller corrispondenti e ad una prima modifica dell'interfaccia grafica per poter avere una prima demo del sistema, compresa di una prima versione dei test di unità in JUnit e di integrazione in Postman.

Nell'ultima iterazione sono stati documentati gli endpoint REST API legati alle funzionalità richieste tramite OpenAPI, insieme al refactoring di numerose problematiche rilevate nel sistema attuale riguardanti la gestione dell'autenticazione e delle eccezioni. È stato inoltre introdotto il concetto di ordine sequenziale dei suggerimenti. Sono stati implementati i meccanismi di validazione ritenuti necessari e sono state ultimate le modifiche all'interfaccia grafica, con annessa internazionalizzazione per supporto multilingua. Sono stati successivamente completati e documentati i test in JUnit e in Postman, insieme ad altri diagrammi relativi alla documentazione del sistema.

Per ogni iterazione, sono state individuate varie sotto-task relative alle attività da svolgere in tale iterazione, assegnate di volta in volta ai membri del gruppo. La gestione del progetto è stata supportata da incontri periodici da remoto, utilizzati per monitorare lo stato di avanzamento di tali attività, in modo da coordinare il lavoro del gruppo e affrontare eventuali problemi rilevati durante lo sviluppo.

2. Analisi dei Requisiti per il Task Assegnato

Nel seguito i seguenti termini saranno utilizzati con queste accezioni:

<i>Termine</i>	<i>Descrizione</i>	<i>Sinonimi</i>
Admin	L'amministratore che accede al sistema	Amministratore
Utente	L'utente generico che utilizza il sistema. Può essere un Admin o un altro microservizio del sistema.	

2.1 Storie utente

Sono elencate le storie utente ricavate a partire dalle interviste con gli stakeholder:

- *Come Admin, voglio caricare un file contenente dei suggerimenti associati a una classe, oppure validi in generale per tutte le classi, in modo da poter dare un aiuto pratico ai miei studenti durante una partita.*
- *Come Utente, mi aspetto che il suggerimento presenti un contenuto informativo testuale, accompagnato da al più un'immagine*
- *Come Admin, voglio caricare un'immagine associata ad un suggerimento, in modo da aggiungere una componente visiva.*
- *Come Utente, voglio visualizzare i suggerimenti esistenti, in modo da tenerne traccia.*
- *Come Admin, voglio cancellare un suggerimento, in modo che non sia più visibile ai miei studenti.*
- *Come Admin, voglio cancellare l'immagine associata ad un suggerimento, in modo che non sia più visibile ai miei studenti.*
- *Come Utente, mi aspetto che i Suggerimenti siano mostrati mantenendo un certo ordine prestabilito.*
- *Come Admin mi aspetto che le nuove operazioni siano esposte come REST API e che siano rese disponibili ai microservizi interessati.*
- *Come Admin mi aspetto che il sistema offra un'interfaccia grafica che mi consenta di interagire con l'API REST, in modo tale da usufruire dei servizi del sistema direttamente nel browser.*
- *Come Utente mi aspetto che l'interfaccia grafica supporti sia la lingua italiana che la lingua inglese, in modo da rendere il sistema più accessibile.*

2.2 Elenco dei requisiti

A partire dalle storie utente, sono stati definiti i seguenti requisiti funzionali e non funzionali:

2.2.1 Requisiti funzionali

- Il sistema deve permettere all'Admin di caricare un file contenente uno o più suggerimenti relativi ad una specifica classe già caricata, oppure validi in generale per tutte le classi
- Il sistema deve permettere all'Admin di caricare un'immagine associata ad un suggerimento già esistente
- Il sistema deve permettere all'Admin di cancellare un suggerimento caricato in precedenza
- Il sistema deve permettere all'Admin di cancellare l'immagine associata ad un suggerimento caricata in precedenza
- Il sistema deve permettere all'Admin di visualizzare i suggerimenti precedentemente caricati

2.2.2 Requisiti non funzionali

- Il database del sistema deve essere un **database relazionale**
- Di ogni Suggerimento si memorizza:
 - o Eventuale riferimento alla classe associata se si tratta di un suggerimento specifico per una classe
 - o Il contenuto testuale informativo del suggerimento proposto dall'Admin
 - o La data di upload
 - o Il livello di aiuto fornito
 - o Al più un'immagine
- I suggerimenti devono seguire un criterio di ordinamento progressivo stabilito dall'Admin
- Il sistema deve **esporre REST API** per rendere accessibili le nuove funzionalità relative ai suggerimenti, in modo che i microservizi del sistema possano accedervi.
- Il sistema deve disporre di un'interfaccia grafica che consenta di interagire con l'API REST, in modo tale da usufruire dei servizi del sistema direttamente nel browser.
- Le operazioni di caricamento e cancellazione possono essere effettuate solo da un Admin autenticato nel sistema.

2.3 Modello concettuale dei suggerimenti

Sulla base dei requisiti funzionali e non funzionali raccolti, è stato definito un primo modello concettuale per l'entità **Suggerimento**, presentato nel *class diagram* che segue e raffinato nella successiva fase di progettazione.

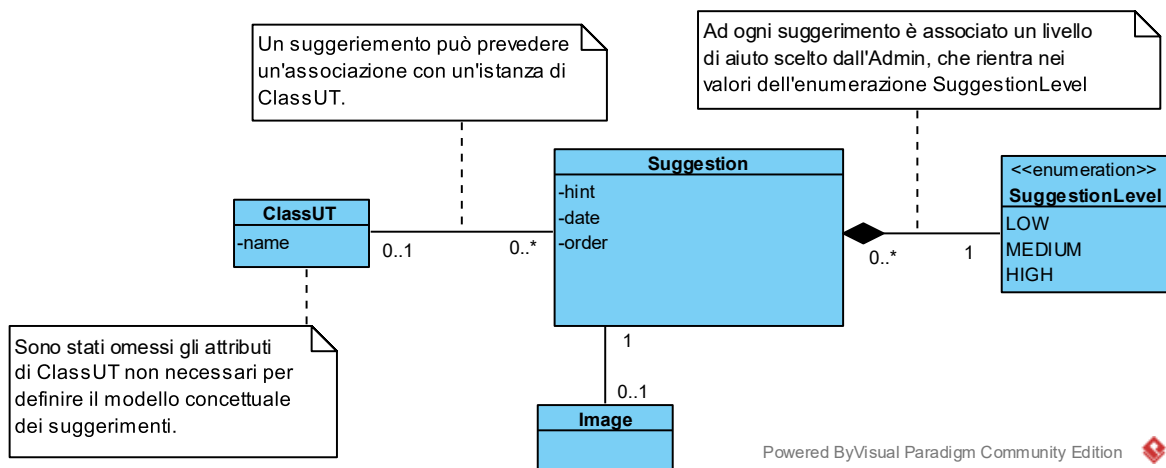


Figura 1 – Diagramma delle classi che mostra il modello concettuale del Suggerimento

Il diagramma mostra che la classe **Suggestion** presenta gli attributi conformi ai requisiti non funzionali, vale a dire:

- Il contenuto informativo testuale (*hint*)
- La data di upload del suggerimento (*date*)

- Il livello di aiuto del suggerimento (*level*). Per soddisfare il requisito di classificazione, è stato introdotto un tipo enumerativo (*SuggestionLevel*) che limita i valori ammissibili alle costanti **LOW**, **MEDIUM** e **HIGH**, in accordo con il formato del suggerimento atteso dal gruppo {AGGIUNGERE NOME}.
- Un eventuale riferimento alla classe associata se si tratta di un suggerimento specifico per una classe
- Un campo *order*, in accordo con il requisito che prevede che l'Admin possa stabilire un criterio di ordinamento progressivo.
- Un'eventuale immagine associata al suggerimento

2.4 Casi d'uso

A partire dai requisiti sono stati definiti i nuovi casi d'uso da implementare, relativi alle operazioni da effettuare sui suggerimenti.

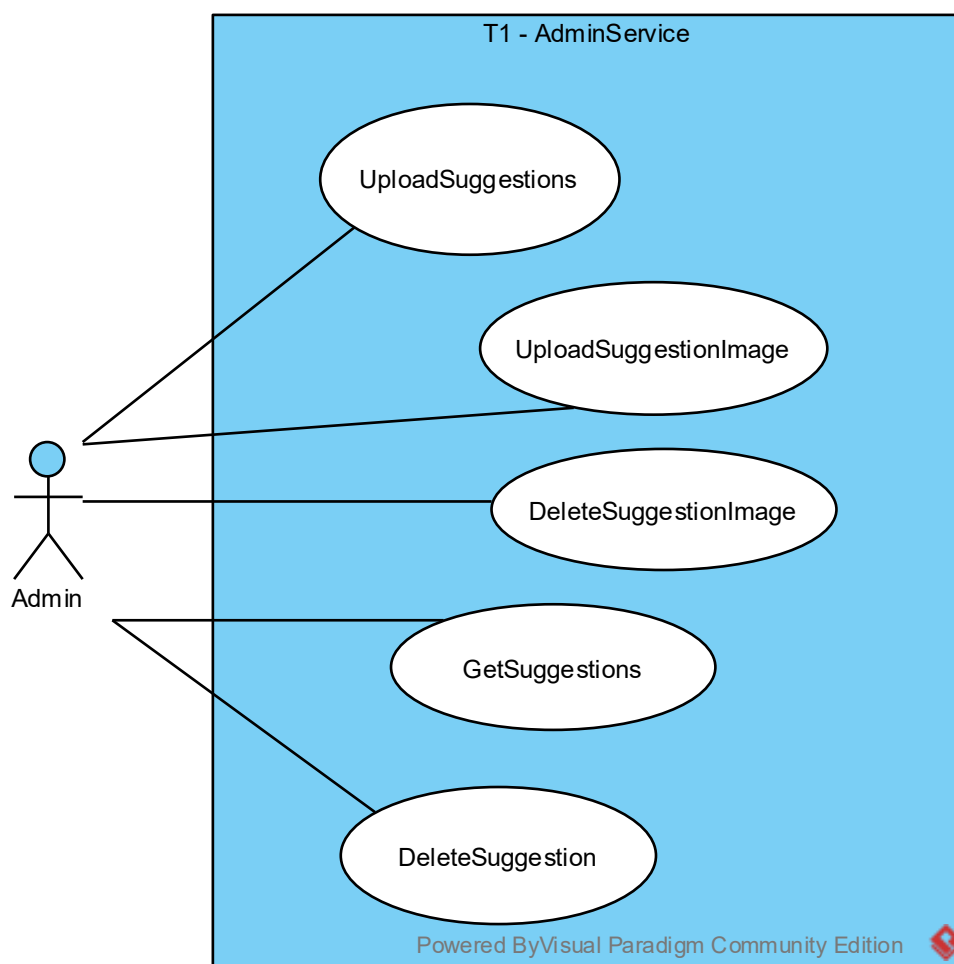


Figura 2 – Diagramma dei casi d'uso. Sono omessi i casi d'uso già implementati

2.4.1 Scenari

Caso d'uso:	UploadSuggestions
Attore primario	Admin

<i>Attore secondario</i>	-
<i>Descrizione</i>	L'Admin carica un file contenente uno o più suggerimenti
<i>Pre-Condizioni</i>	L'Admin è autenticato nel sistema e, nel caso in cui il suggerimento sia specifico per una classe, deve risultare che la classe riferita esista già nel database.
<i>Sequenza di eventi principale</i>	1. Il caso d'uso inizia quando l'Admin carica un file contenente uno o più suggerimenti associati ad una classe o validi in generale per tutte le classi. 2. Il sistema controlla la correttezza del formato dei suggerimenti caricati e che l'eventuale classe esista. 3. Il sistema memorizza i suggerimenti inseriti dall'Admin nel database e il caso d'uso termina.
<i>Post-Condizioni</i>	I suggerimenti sono presenti nel database.
<i>Casi d'uso correlati</i>	-
<i>Sequenza di eventi alternativi</i>	2.1 Il formato dei suggerimenti caricati non è corretto 2.1.1 Il sistema restituisce un ERRORE all'Admin. 2.2 La classe a cui fanno riferimento i suggerimenti non è presente 2.2.1 Il sistema restituisce un ERRORE all'Admin.

<i>Caso d'uso:</i>	GetSuggestions
<i>Attore primario</i>	Admin
<i>Attore secondario</i>	-
<i>Descrizione</i>	L'Admin visualizza i suggerimenti associati ad una classe o validi in generale per tutte le classi.
<i>Pre-Condizioni</i>	L'Admin è autenticato nel sistema. Nel caso in cui si richiedano i suggerimenti associati ad una classe specifica, questa deve esistere nel database.
<i>Sequenza di eventi principale</i>	1. Il caso d'uso inizia quando l'Admin richiede la visualizzazione dei suggerimenti associati alla classe selezionata o validi in generale per tutte le classi. 2. Il sistema mostra all'Admin i suggerimenti associati alla classe
<i>Post-Condizioni</i>	Il sistema mostra all'Admin i suggerimenti richiesti.
<i>Casi d'uso correlati</i>	-
<i>Sequenza di eventi alternativi</i>	2.3 La classe a cui fanno riferimento i suggerimenti non è presente nel database 2.3.1 Il sistema restituisce un ERRORE all'Admin.

<i>Caso d'uso:</i>	UploadSuggestionImage
<i>Attore primario</i>	Admin
<i>Attore secondario</i>	-

<i>Descrizione</i>	L'Admin carica un'immagine associata ad un suggerimento esistente.
<i>Pre-Condizioni</i>	L'Admin è autenticato nel sistema. Il suggerimento deve esistere nel database e, nel caso in cui sia associato ad una classe specifica, anche questa deve esistere nel database.
<i>Sequenza di eventi principale</i>	1. Il caso d'uso inizia quando l'Admin carica un'immagine associata ad un suggerimento esistente. 2. Il sistema controlla le dimensioni del file caricato. 3. Il sistema memorizza l'immagine caricata dall'Admin nella memoria persistente e il caso d'uso termina.
<i>Post-Condizioni</i>	L'immagine è stata caricata in memoria persistente.
<i>Casi d'uso correlati</i>	-
<i>Sequenza di eventi alternativi</i>	1.1 Il suggerimento a cui si fa riferimento non è presente nel database 1.1.1 Il sistema restituisce un ERRORE all'Admin. 2.1 Le dimensioni dell'immagine eccedono il valore limite prefissato 2.1.1 Il sistema restituisce un ERRORE all'Admin.

<i>Caso d'uso:</i>	DeleteSuggestion
<i>Attore primario</i>	Admin
<i>Attore secondario</i>	-
<i>Descrizione</i>	L'Admin cancella un suggerimento associato ad una classe o valido in generale per tutte le classi.
<i>Pre-Condizioni</i>	L'Admin è autenticato nel sistema. Nel caso in cui l'operazione riguardi un suggerimento associato ad una classe specifica, questa deve esistere nel database.
<i>Sequenza di eventi principale</i>	1. Il caso d'uso inizia quando l'Admin richiede la cancellazione di un suggerimento associato ad una classe o valido in generale per tutte le classi. 2. Il sistema cancella il suggerimento dalla memoria persistente.
<i>Post-Condizioni</i>	Il sistema non ha più persistenza del suggerimento cancellato.
<i>Casi d'uso correlati</i>	-
<i>Sequenza di eventi alternativi</i>	1.1 La classe a cui fanno riferimento i suggerimenti non è presente 1.1.1 Il sistema restituisce un ERRORE all'Admin. 1.2 Il suggerimento a cui si fa riferimento non è presente nel database 1.2.1 Il sistema restituisce un ERRORE all'Admin.

<i>Caso d'uso:</i>	DeleteSuggestionImage
<i>Attore primario</i>	Admin
<i>Attore secondario</i>	-
<i>Descrizione</i>	L'Admin cancella l'immagine associata ad un suggerimento esistente.

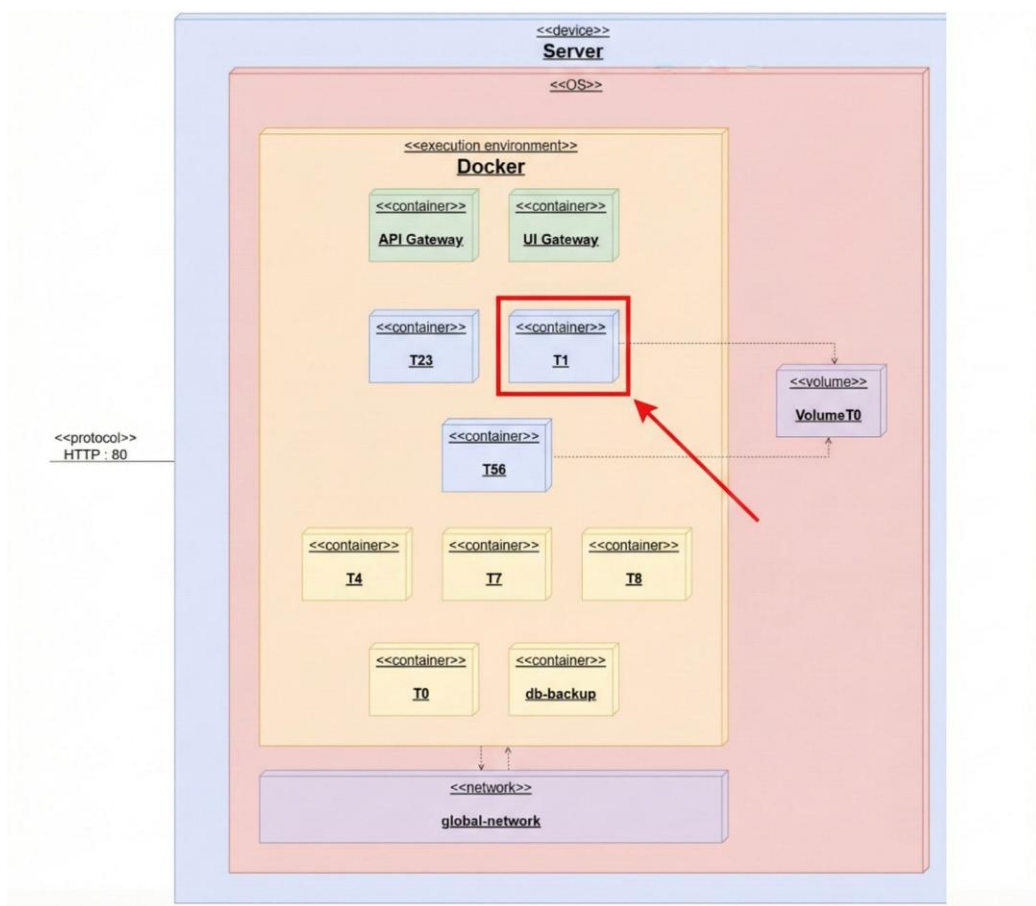
<i>Pre-Condizioni</i>	L'Admin è autenticato nel sistema. Il suggerimento deve esistere nel database e, nel caso in cui sia associato ad una classe specifica, anche questa deve esistere nel database. Inoltre, l'immagine deve essere stata caricata in precedenza.
<i>Sequenza di eventi principale</i>	1. Il caso d'uso inizia quando l'Admin richiede la cancellazione di un'immagine associata ad un suggerimento esistente. 2. Il sistema cancella dalla memoria persistente l'immagine specificata.
<i>Post-Condizioni</i>	Il sistema non ha più persistenza dell'immagine cancellata.
<i>Casi d'uso correlati</i>	-
<i>Sequenza di eventi alternativi</i>	1.1 Il suggerimento a cui si fa riferimento non è presente nel database 1.1.1 Il sistema restituisce un ERRORE all'Admin.

2.5 Analisi dell'impatto

Le modifiche **riguarderanno il microservizio T1**.

- Si prevedono modifiche all'interfaccia grafica del sistema per permettere all'admin di usufruire dei servizi relativi ai suggerimenti.
- Si prevede l'aggiunta delle API REST per poter effettuare le operazioni richieste.
- Si prevedono modifiche alla logica di business del sistema.
- Si prevedono modifiche al modello del database (migrazione relazionale + aggiunta suggerimenti).

Figura 3 – Diagramma di deploy complessivo, evidenziando il microservizio impattato



3. Progettazione della soluzione

3.1 Scelte di progetto

3.1.1 Suggerimenti e Linee Guida

Per differenziare un suggerimento associato ad una classe da uno valido in generale per tutte le classi, sono stati definiti due classi distinte, rispettivamente **Suggerimento** e **Linea Guida**.

In particolare, una **Linea Guida**, che d'ora in poi sarà chiamata anche **Guideline**, è caratterizzata dai seguenti campi:

- Il contenuto testuale informativo del suggerimento proposto dall'Admin
- La data di upload
- Al più un'immagine

Un **Suggerimento**, che d'ora in poi sarà chiamata anche **Suggestion**, aggiunge ai campi precedenti gli ulteriori campi:

- Riferimento della classe associata (obbligatorio)
- Il livello di aiuto fornito

Glossario dei termini

Sono definiti nel seguente glossario, per non avere ambiguità, i termini Linea Guida e Suggerimento:

<i>Termine</i>	<i>Descrizione</i>	<i>Sinonimi</i>
Linea Guida	Un aiuto valido in generale, non associato ad una classe specifica.	Guideline
Suggerimento	Un aiuto associato ad una specifica classe.	Suggestion

3.1.2 Modello concettuale di suggerimenti e linee guida

A partire dalle nuove definizioni, è stato raffinato il modello concettuale dell'entità **Suggestion**, presentato nel *class diagram* che segue insieme alla nuova entità **Guideline**.

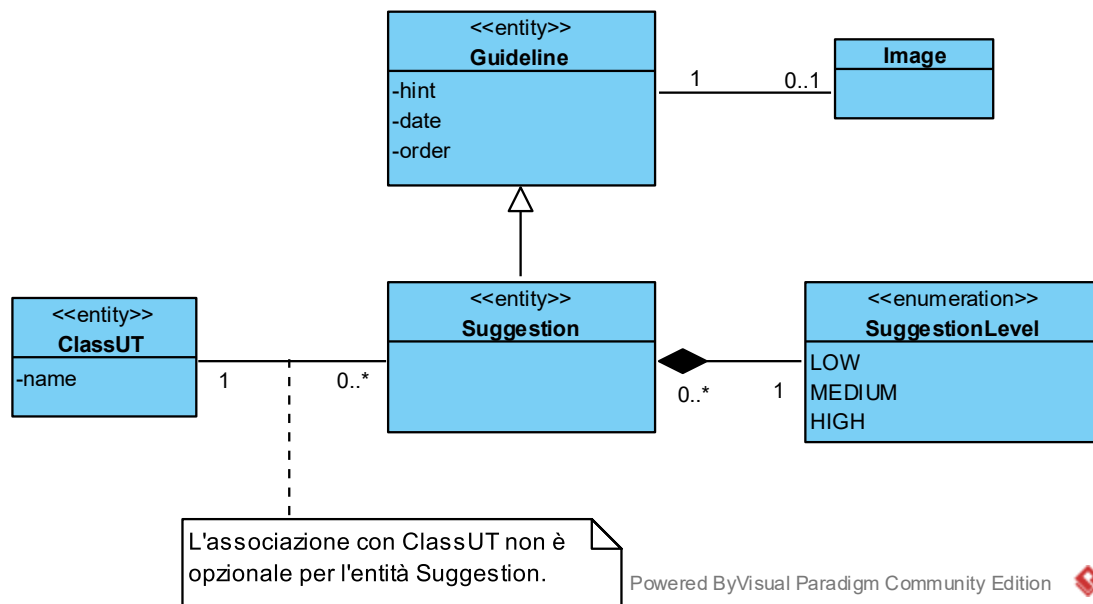


Figura 4 – Diagramma delle classi che mostra il modello concettuale del Suggerimento e della Linea Guida

L'entità **Suggestion** estende l'entità **Guideline** da cui eredita i campi comuni *hint*, *date*, *order*, e il riferimento a *Image*, il cui significato è rimasto invariato rispetto al precedente diagramma. L'entità **Suggestion** aggiunge dunque il riferimento ad un'istanza di **ClassUT** (non più opzionale, in seguito alla distinzione tra Guideline e Suggestion) e al livello.

Dunque, per Linea Guida intendiamo una qualunque istanza di Guideline che non sia anche istanza di Suggestion. Definiamo Suggerimento una qualunque istanza di Suggestion.

È stato scelto di non caratterizzare una Linea Guida con un livello di aiuto, in modo da avere delle Linee Guida il più generiche possibili.

3.2 Migrazione del Database

Prima di procedere con la migrazione, è stata effettuata un'operazione di *reverse engineering* per ricavare un **modello logico del database MongoDB** presente nel microservizio T1 – AdminService. Da notare che in MongoDB le relazioni non sono "rigide" come in SQL, in quanto è un database non relazionale per natura. Tuttavia, analizzando i nomi dei campi dei singoli Document di MongoDB è stato possibile inferire i collegamenti logici tra le collezioni (come, ad esempio, il campo `teamId` presente in `Assignment` che funge da riferimento logico alla collezione `Team`), insieme alle molteplicità opportune.

Il risultato è stato formalizzato nel seguente diagramma, in cui le chiavi primarie sono indicate con [PK] e i riferimenti esterni con [FK], pur non essendo vincoli nativi di MongoDB:

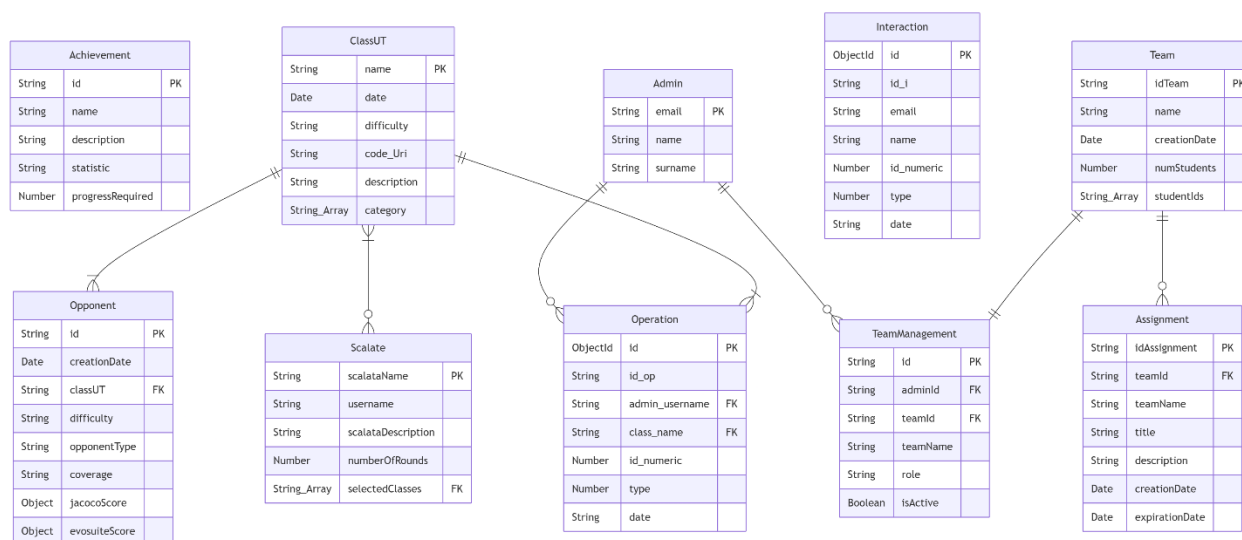
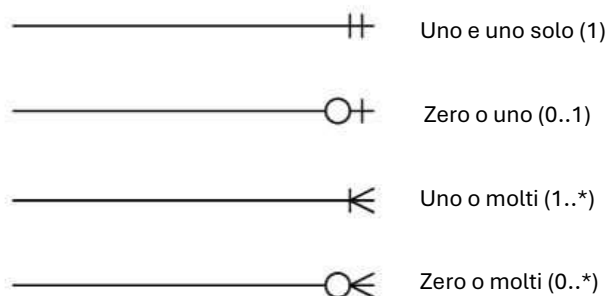


Figura 5 – Modello logico inferito dal database MongoDB

È stata utilizzata la notazione nota come *Crow's foot* per indicare la molteplicità delle relazioni e il relativo significato.



Ad esempio, nel diagramma delle tabelle di MongoDB mostrato, è stata individuata una relazione uno a molti tra `ClassUT` e `Opponent`: ogni istanza di `ClassUT` è associata a una

o molte istanze di Opponent (1..*), mentre ogni istanza di Opponent è associata a una e una sola istanza di ClassUT (1).

3.2.1 Criticità rilevate

L'adozione iniziale di un database NoSQL come MongoDB è stata effettuata al fine di avere un avvio rapido dello sviluppo e per semplificare la realizzazione del sistema, introducendo tuttavia un **debito tecnico** significativo. Questa scelta di progetto iniziale incide infatti negativamente sulla manutenibilità e l'evolubilità del microservizio. L'analisi ha evidenziato le seguenti criticità strutturali:

- **Incoerenza nei tipi di dato utilizzati:** i tipi dei campi non sono sempre coerenti con la loro semantica. Ad esempio, alcuni campi che indicano delle date sono stati realizzati con il tipo String (come in *Interaction* e *Operation*) anziché con il tipo Date di MongoDB (utilizzato invece correttamente in *Assignment*, *Team*, *ClassUT*, *Opponent*).
- **Ambiguità e duplicazione degli identificativi:** alcune collections presentano molteplici campi che assumono il ruolo di id, il che rende ambiguo lo schema dei dati. È questo il caso della collection *Operation*, in cui sono presenti i campi “_id” (ObjectId di default di MongoDB), “id_op” (String) e “id” (Number), e in maniera analoga nel caso della collection *Interaction* (con “_id”, “id_i” e “id”).
- **Dati isolati:** nessuna collection presenta legami logici espliciti verso *Achievement*, né verso *Interaction*, rendendo questa collezione isolata dal resto del database
- **Assenza di vincoli di integrità:** la natura di MongoDB comporta l'assenza di schemi predefiniti e di vincoli di integrità sui dati, permettendo potenzialmente l'esistenza di riferimenti verso documenti inesistenti.
- **De-normalizzazione eccessiva:** Il database è fortemente non normalizzato, con sistematica duplicazione delle informazioni su diversi documenti. Ad esempio, nella collection *TeamManagement* sono memorizzati sia “teamId” che *teamName* per riferirsi ad un particolare *Team*. Sebbene la de-normalizzazione sia tipica nei sistemi NoSQL per migliorare le performance in lettura (evitando di effettuare operazioni di “join” dei sistemi relazionali), nel contesto attuale si rivela un problema perché i dati vengono aggiornati spesso. Tale struttura rende infatti più complesse le operazioni di aggiornamento e mantenimento della consistenza dei dati (ad esempio, a seguito di un'operazione di update di un Document in *Team* è necessario controllare eventuali modifiche da effettuare in *TeamManagement* per preservare la consistenza dei dati).

Tali criticità, insieme alla necessità di strutturare le relazioni in modo rigido, ci hanno spinto a migrare il database verso il **modello relazionale**. In particolare, la scelta del database è ricaduta su **MySQL** data la sua semplicità di utilizzo e ampia diffusione nello standard industriale.

3.2.2 Modello ER

Di seguito è mostrato il modello ER del nuovo database relazionale, ottenuto a partire da un intenso refactoring.

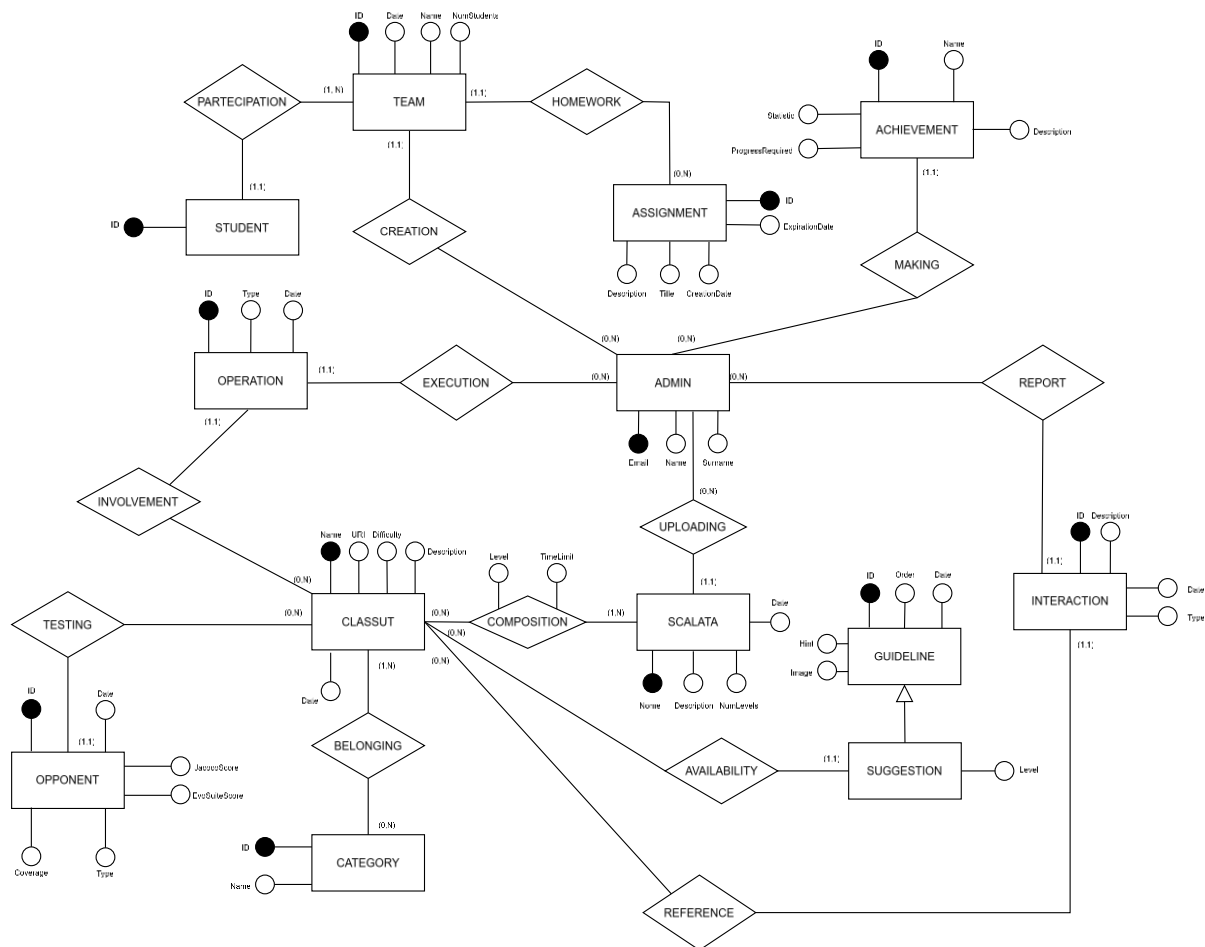


Figura 6 – Modello ER del nuovo database relazionale

Ogni collection del database MongoDB precedente è stato mappato in **un’entità** corrispondente nel modello relazionale, con le dovute modifiche per poter definire correttamente i vincoli di integrità necessari e le relazioni tra le varie entità. Inoltre, c’è stata l’eliminazione della collection “TeamManagement”, che non è mappata a nessuna entità in quanto corrisponde all’associazione tra le entità TEAM e ADMIN.

Come aggiunte vi sono le entità **SUGGESTION** e **GUIDELINE**, che sono legate da una relazione di ereditarietà con vincolo di **copertura parziale** (ovvero possono esistere istanze di GUIDELINE che non sono istanze di SUGGESTION), al fine di raggruppare i campi comuni in GUIDELINE, conforme al modello concettuale mostrato nel diagramma delle classi relativo ai suggerimenti raffinato in precedenza.

Il campo Order nell’entità GUIDELINE è utilizzato per indicare l’ordine progressivo delle Linee Guida e dei Suggerimenti. In particolare:

- per le Linee Guida, il campo Order indica l'ordine progressivo di tale istanza all'interno dell'elenco di tutte le altre Linee Guida
- per i Suggerimenti, indica l'ordine progressivo del suggerimento tra tutti i suggerimenti associati alla stessa istanza di CLASSUT (chiave esterna presente in SUGGESTIONS). Quindi l'ordine è **per classe**, in quanto ogni classe ha una propria sequenza di suggerimenti associati, ordinata in maniera indipendente da quella delle altre.

Sono stati inoltre aggiunti i campi *TimeLimit* e *Level* come attributi della relazione COMPOSITION che lega le entità CLASSUT e SCALATA, in modo da uniformare il database alle modifiche future riguardanti il supporto della modalità di gioco scalata.

L'entità STUDENT è stata aggiunta rispetto al modello non relazionale del database precedente, in quanto si è rilevata necessaria per realizzare la relazione con l'entità TEAM. È caratterizzata dalla sola chiave primaria ID, in quanto la gestione della persistenza degli studenti non è responsabilità del microservizio AdminService. Analogamente sono stati rimossi campi non utilizzati di ADMIN, quali *invitationToken*, *resetToken*, *username*, *password*.

3.2.3 Modello Logico

In seguito, è mostrato il codice SQL relativo alla creazione delle tabelle derivanti dalla progettazione logica effettuata a valle del modello ER sviluppato.

In particolare:

- ogni entità è stata mappata in una tabella
- ogni relazione molti a molti è stata tradotta in una tabella associativa, contenente le chiavi esterne delle tabelle riferite e gli eventuali attributi aggiuntivi (nel caso della relazione tra CLASSUT e SCALATA), con chiave primaria pari alla coppia delle chiavi esterne.
- ogni relazione uno a molti è stata tradotta con l'aggiunta di una chiave esterna nella tabella dal lato "molti"
- alcuni campi sono stati tipizzati con delle enumerazioni apposite, in base ai valori individuati nel codice java:
 - *OperationType* per indicare il tipo di operazione, che può assumere i valori UPLOAD, DELETE, UPDATE.
 - *InteractionType* per indicare il tipo di interazione, che può assumere i valori LIKE, REPORT.
 - *OpponentDifficulty* per indicare la difficoltà di un opponent, che può assumere i valori EASY, MEDIUM, HARD.

3.3 Refactoring dei Package JAVA del microservizio T1

Il diagramma dei package di un'applicazione java descrive tutti i package e le loro interdipendenze di utilizzo e c'è una corrispondenza uno ad uno con le cartelle dei package java.

Tra le modifiche effettuate nel microservizio T1, si è reso necessario un refactoring dei package java per risolvere numerosi anti-pattern e bad smells individuati nell'architettura.

Come confronto di seguito è mostrato il diagramma dei package da cui si è partiti:

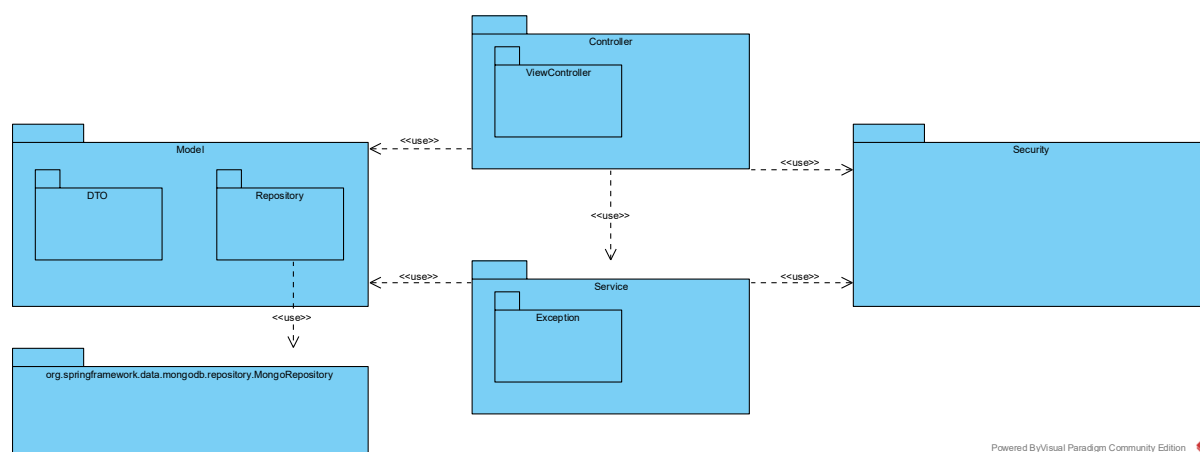


Figura 8 – Diagramma dei package java del sistema precedente

I package rispecchiano i componenti del pattern architetturale Spring MVC adottato nel microservizio T1. Tuttavia, sono state notate molteplici criticità.

3.3.1 Criticità dell'architettura del sistema

Le **responsabilità** dei singoli elementi non erano rispettate a pieno:

- Il package Service **presentava una dipendenza verso il package Security**, in quanto le classi in Service, che dovrebbero occuparsi unicamente della logica di business del sistema, si preoccupavano di effettuare **controlli di sicurezza**, come la verifica dell'autenticazione dell'admin, che invece dovrebbero essere responsabilità delle classi nel package Controller.
- Le classi del package Service si preoccupavano inoltre di **costruire le risposte HTTP** che il controller avrebbe poi restituito all'utente, mentre l'interfacciamento con l'esterno dovrebbe essere responsabilità del package Controller
- Il package Controller presentava una **dipendenza verso il package Model**, utilizzando le classi Model per mappare gli input ricevuti dalle richieste HTTP. In questo modo il Controller **era a conoscenza della struttura logica del database**, la quale dovrebbe essere confinata tra il package Service e il package Repository. Il Controller **dovrebbe fare uso solo dei DTO** per mappare gli input delle richieste

La struttura dei package non rispecchia la loro semantica, in quanto:

- Il package **DTO**, che contiene classi utilizzate per **trasferire i dati** dal frontend al Controller e tra Controller e Service, era stato inserito come sotto-package del package **Model**, che contiene invece le classi java che **modellano il database**. Dunque, non è stato ritenuto corretto semanticamente inserire il package DTO come sotto-package di Model.
- Il package **Repository** contiene le interfacce che ereditano da **MongoRepository** per **interfacciarsi con il database** ed effettuare le operazioni **CRUD** relative alle classi Model. Dunque, non è stato ritenuto corretto semanticamente inserire il package Repository come sotto-package di Model.

A partire da queste criticità è stato effettuato un refactoring complessivo del sistema. Il nuovo diagramma dei package java del microservizio T1 è il seguente:

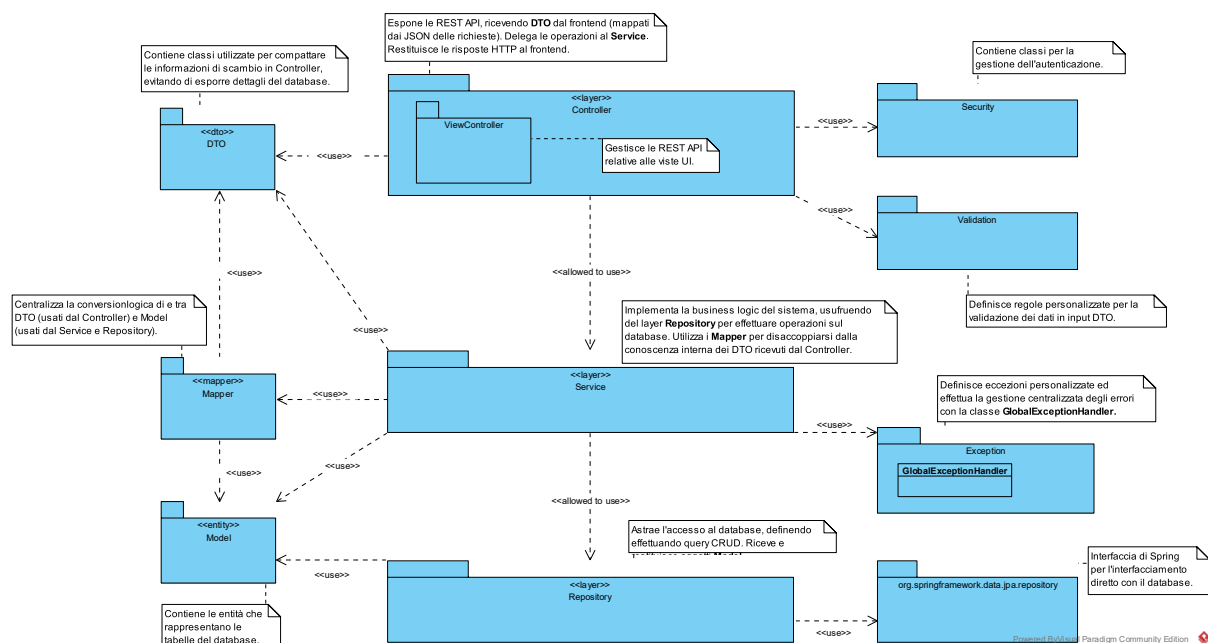


Figura 9 – Diagramma dei package java del sistema attuale

I package **Controller**, **Service** e **Repository** rappresentano adesso dei `<<layer>>`, in quanto raggruppano le classi seguendo il pattern architetturale Spring MVC adottato.

Sono state risolte le varie criticità rilevate nel package diagram precedente. In particolare:

- Ora il layer Service non **presenta nessuna dipendenza verso Security** e si occupa **esclusivamente della logica di business**
- Si è scelto di esportare il package **Exception** al di fuori di **Service**, in quanto si è deciso di aggiungere un gestore globale delle eccezioni **GlobalExceptionHandler** che genera le opportune risposte HTTP da restituire sul frontend in base all'eccezione da gestire.

- Il layer Controller adesso gestisce **solo il caso in cui l'operazione va a buon fine**, restituendo la risposta HTTP apposita (con status 2xx).
- Ora il **layer Controller accede solo ai DTO** per mappare gli input ricevuti dalle richieste HTTP del frontend, che trasferisce al Service
- Si è scelto di aggiungere il nuovo package Mapper per centralizzare la logica di conversione tra **DTO e Model e viceversa, disaccoppiando** in questo modo il layer Service dai DTO e i Controller dai Model.

In seguito, sono esplorati più nel dettaglio i singoli package.

3.3.2 DTO

Il package *DTO* contiene classi utilizzate per compattare le **informazioni di scambio** tra i livelli superiori del sistema, evitando di esporre dettagli del database all'esterno. In particolare, il Controller riceve i DTO come mapping del contenuto informativo delle richieste HTTP. I DTO sono poi mandati al Service come input delle operazioni da effettuare, il quale restituisce all'occorrenza altri DTO.

3.3.3 Mapper

Il package *Mapper* **centralizza la logica di conversione** da oggetti DTO a Model corrispondenti e viceversa. Permette di evitare di esporre i Model all'esterno e che il Service presenti una dipendenza esplicita da *DTO*, consentendo una trasparenza alla conversione.

In generale, ogni classe mapper presente nel package Mapper fa uso della libreria **MapStruct**, che consente il **mapping automatico tra oggetti**. Nel nostro caso specifico, legge le interfacce annotate come *@Mapper* e genera automaticamente classi concrete, effettuando il mapping tra classi Model e DTO, e viceversa. In particolare, fa uso dei seguenti metodi:

- **toEntity** riceve in ingresso un oggetto della classe DTO e restituisce in uscita un oggetto della classe Model
- **toDTO** riceve in ingresso un oggetto della classe Model e restituisce in uscita un oggetto della classe DTO
- **toEntityList** riceve in ingresso una lista di oggetti della classe DTO e restituisce in uscita una lista di oggetti della classe Model
- **toDTOList** riceve in ingresso una lista di oggetti della classe Model e restituisce in uscita una lista di oggetti della classe DTO

Inoltre, vi è la possibilità di **personalizzare la logica di mapping**, escludendo dalla conversione eventuali campi oppure definendo dei metodi di mapping personalizzati.

3.3.4 Model

Il package *Model* **astrae le tabelle del database** ed è utilizzato come input e output delle operazioni CRUD. Riguardo la persistenza delle classi **Suggestion** e **Guideline**, sono

state utilizzate due diverse tabelle, la prima delle quali contiene i campi comuni alle due classi, e la seconda contiene, invece, i campi esclusivi della classe *Suggestion*, in aggiunta ad una chiave esterna, rivolta verso la prima tabella, in modo da discriminare tra le due entità.

3.3.5 Controller

Il package *Controller* espone le **REST API** per accedere alle risorse del microservizio, utilizzando il package *Service* per espletare le funzionalità richieste. Riceve le informazioni in input dal JSON sottoforma di *DTO* e le trasferisce al *Service*. Non può accedere al database, né ai *Model*. Il sotto-package **ViewController** contiene i controller per selezionare e popolare le viste nel frontend.

Come scelta di progetto, sono state implementate **due classi Controller differenti** al fine di gestire le REST API relative alle classi ***Suggestion*** e ***Guideline*** in maniera separata, personalizzando il flusso di controllo e migliorando la **modularità** del codice.

3.3.6 Service

Il package *Service* astrae la **business logic del sistema**, accedendo alle risorse tramite il package *Repository*. Utilizza i *Mapper* per comunicare con il *Controller*: converte gli oggetti *DTO* ricevuti nei corrispondenti *Model* (in ingresso) e viceversa (in uscita). Non può accedere direttamente al database, quindi utilizza i servizi esposti da *Repository*. Così come nel caso dei controller, sono stati implementati due *Service* differenti per le classi ***Suggestion*** e ***Guideline***, che però utilizzano un servizio comune, denominato *ImageService*, per la gestione delle immagini sul filesystem.

3.3.7 Repository

Il package *Repository* **astrae l'accesso al database**, effettuando le query per implementare le operazioni CRUD. Lavora con oggetti del package *Model* per modellare il database. Le interfacce del package *Repository* estendono le interfacce di **Spring Data JPA** da cui dipendono per l'implementazione automatica dei metodi CRUD. In particolare ogni interfaccia contenuta nel package *Repository* estende l'interfaccia generica ***JpaRepository<TModel, TId>***, dove *TModel* è il tipo della classe model per la quale tale repository fornisce le operazioni CRUD e *TId* è il tipo del campo *Id* (chiave primaria) del *TModel* suddetto (tipicamente *Long*, ma sono possibili anche altri tipi come *String*).

3.3.8 Security

Il package *Security* centralizza la gestione dell'autenticazione dell'utente. In particolare, contiene la classe *AuthTokenFilter* che si occupa di filtrare tutte le richieste HTTP prima che queste raggiungano il Controller opportuno, verificando che l'utente sia autenticato. Sono presenti maggiori dettagli sul flusso di validazione di questa classe nella sezione di refactoring in seguito.

3.3.9 Validation

Il package *Validation* contiene la logica di validazione dei DTO in input al sistema. In particolare, definisce le annotazioni personalizzate che possono essere utilizzate sui vari campi / parametri di funzioni da validare, congiuntamente alle classi che implementano la logica di validazione vera e propria. Qui è presente, ad esempio, la validazione del campo order di Guideline e Suggestion, utilizzata dai Controller nelle richieste di upload. La procedura di validazione segue i vincoli stabiliti dai requisiti non funzionali.

3.3.10 Exception

Il package *Exception* raggruppa le varie eccezioni definite nel sistema, insieme al **GlobalExceptionHandler**, che centralizza la gestione delle eccezioni, restituendo le opportune risposte HTTP (con status **4xx** o **500**) in base all'eccezione catturata. Questa classe utilizza l'annotazione di Spring **@RestControllerAdvice** per poter intercettare tutte le eccezioni del sistema. Ogni eccezione è gestita da un metodo dedicato, indicato con l'annotazione **@ExceptionHandler(TipoEccezione.class)**, che si occupa di restituire la risposta HTTP opportuna con il messaggio corretto. Le eccezioni gestite dal GlobalExceptionHandler sono catturate nell'ordine in cui sono definite. Pertanto, le eccezioni più specifiche sono posizionate prima di quelle generiche.

In particolare, queste sono le eccezioni gestite:

- *HttpMessageNotReadableException*
- *MethodArgumentNotValidException*
- *NotFoundException*
- *ForbiddenException*
- *ConstraintViolationException*
- *UnauthorizedException*
- *InvalidImageException*
- *MaxUploadSizeExceededException*
- *ServletException*
- *Exception*: per gestire tutte le eccezioni non catturate in precedenza da nessun altro *handler*

3.4 Class Diagram del package Model

In precedenza, il package Model conteneva le classi che rispecchiavano fedelmente il database MongoDB, il cui schema logico è stato rappresentato in forma tabellare in precedenza, mostrato in Figura 7 – Modello logico del nuovo database relazionale (Vista delle tabelle)Figura 5. Per poter effettuare la migrazione, le classi presenti nel package Model hanno subito numerose variazioni. Il seguente diagramma delle classi del package Model descrive tutte le classi presenti nell’attuale package java Model, che rispecchiano le entità definite nel modello concettuale del database relazionale, il cui schema concettuale e logico sono mostrati rispettivamente in Figura 6 e in Figura 7.

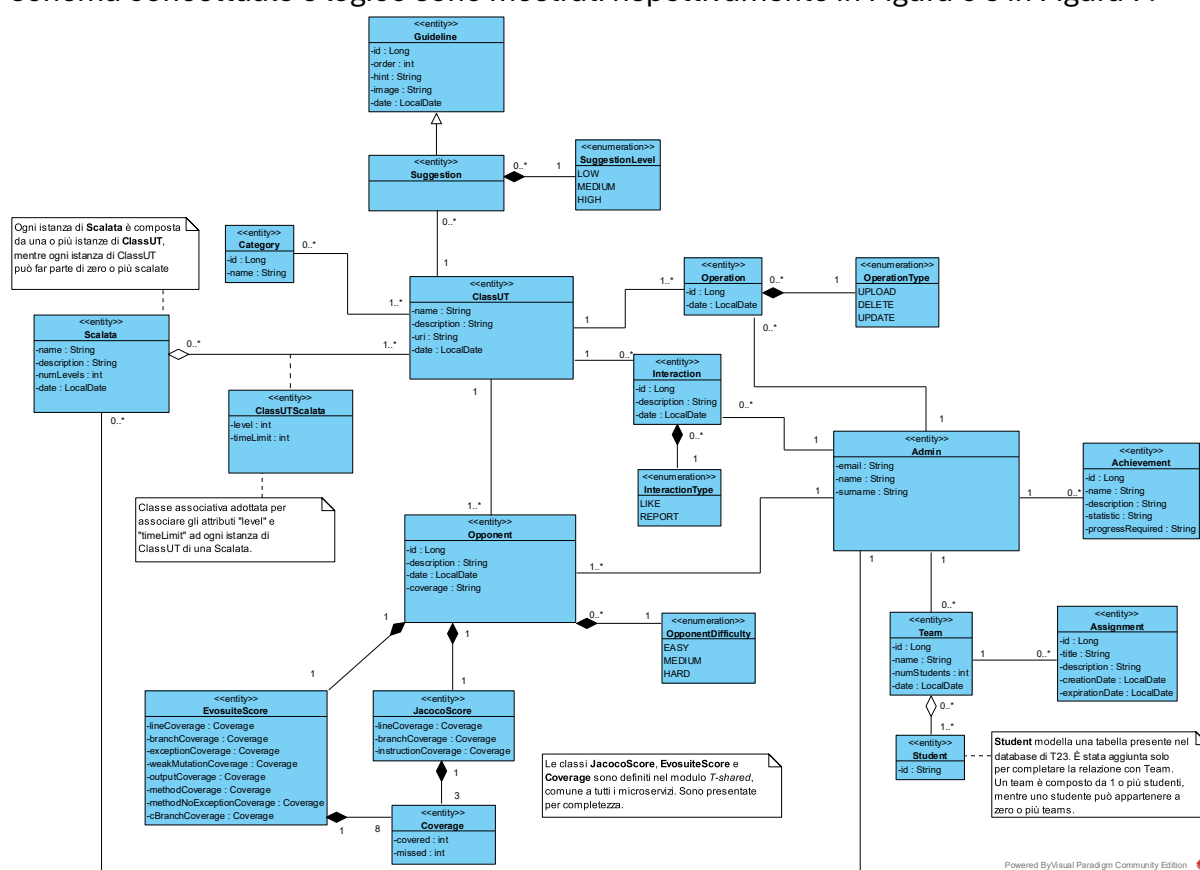


Figura 10 – Diagramma delle classi del package Model del microservizio T1

Per rispecchiare il database relazionale, sono stati creati i tipi enumerativi **InteractionType**, **OperationType** e **OpponentDifficulty**, utilizzati per eliminare eventuali “numeri magici” adottati in precedenza nel sistema per riferirsi a tali informazioni.

Da notare la classe associativa **ClassUTScalata**, necessaria per aggiungere gli attributi specifici dell’associazione molti a molti tra **ClassUT** e **Scalata**.

Come nel modello concettuale del database, la classe **Suggestion** specializza la classe **Guideline**, ereditando in questo modo i campi in comune.

In particolare, per ogni classe model:

- sono stati sostituiti gli import relativi a mongodb con jakarta JPA, in particolare sostituendo l'annotazione di mongodb **@Document** (utilizzata in precedenza per collegare una classe Model con la collection MongoDB associata) con quelle di jakarta **@Entity** e **@Table** (quest'ultima per specificare il nome della tabella corrispondente nel database)
- è stata utilizzata l'annotazione **@Id** per specificare il campo legato alla chiave primaria, eventualmente insieme **@GeneratedValue** per specificare la generazione automatica dell'id nei casi in cui la chiave primaria non sia una stringa.
- Sono state utilizzate le annotazioni necessarie per indicare le relazioni SQL nelle classi model:
 - **@ManyToOne** per indicare che un campo è legato ad una relazione molti ad uno (ovvero è un riferimento ad un oggetto della classe Model con cui è presente la relazione). Utilizzata in congiunzione con **@JoinColumn(name = "nomeFK", referencedColumnName = "nomePKriferita")** per specificare che il campo annotato è mappato nella tabella del suddetto model come chiave esterna ad un'altra tabella, permettendo di indicare il nome della colonna che costituisce la chiave esterna (*nomeFK*) e il nome della chiave primaria nella tabella riferita (*nomePKriferita*).
 - **@OneToMany(mappedBy = "nomeCampo")** per indicare che un campo è legato ad una relazione uno a molti (ovvero è una lista di riferimenti a oggetti della classe Model con cui è presente la relazione), indicata con un'associazione corrispondente **@ManyToOne** sul campo "*nomeCampo*" della classe Model riferita. Eventualmente definisce il **cascade type** come REMOVE per indicare il trigger **ON DELETE CASCADE** sul campo del database (permettendo in questo modo la rimozione automatica dal database di tutte le tuple delle tabelle che presentano una chiave esterna verso il campo in questione, qualora sia eliminato).
 - **@ManyToMany** per indicare che un campo è legato ad una relazione molti a molti (ovvero è una lista di riferimenti a oggetti della classe Model con cui è presente la relazione). Utilizzata in congiunzione con **@JoinTable()** per indicare la tabella associativa creata nel database per rappresentare la relazione molti a molti. È utilizzata per modellare l'associazione molti a molti tra **ClassUT** e **Category**.
 - **@EmbeddedId** utilizzato per indicare una chiave primaria composta. Utilizzata in congiunzione con **@Embeddable** per la chiave primaria di ClassUTScalata (ovvero un'istanza di ClassUTScalataId) e con **@MapsId** per mappare le chiavi esterne nella chiave primaria (in quanto la chiave primaria

di ClassUTScalata è data dalla composizione delle due chiavi esterne verso ClassUT e Scalata)

- Utilizzo di `@Column` per indicare eventualmente il nome della colonna corrispondente al campo nella tabella (qualora sia diverso dal nome del campo) e anche per indicare ulteriori constraint sul campo come `UNIQUE` e `NOT NULL`.
- Utilizzo **delle annotazioni di lombok** per generare automaticamente metodi getter, setter, costruttori senza parametri e con parametri, rimuovendo di conseguenza tutto il codice relativo ai metodi generati automaticamente da tali annotazioni.

Da notare che il campo “image” presente in **Guideline** (e dunque ereditato da **Suggestion**) è di tipo String e rappresenta il nome dell’immagine. Questa scelta è stata presa una volta stabilito che le immagini **andassero salvate all’interno del Volume T0** del sistema (utilizzato anche per salvare le classi sotto-test ClassUT e i relativi test dei vari Opponent).

In particolare, le immagini sono salvate nel percorso seguente del Volume T0:

`/VolumeT0/FolderTree/Images/`

Mentre il nome dell’immagine viene calcolato all’atto dell’upload nel seguente modo:

`{className}_{order}.{estensione}`

Viene preservata l’estensione dell’immagine caricata dall’esterno.

L’immagine viene caricata in un secondo momento rispetto al suggerimento, tramite l’apposito bottone sull’interfaccia grafica. Inizialmente si era pensato di caricare l’immagine insieme al suggerimento, inserendola come stringa in formato Base64 all’interno del file JSON di upload. Tuttavia, tale scelta richiedeva all’Admin di effettuare la conversione in Base64 delle immagini da caricare, in modo tale da ottenere la stringa da inserire all’interno del file JSON di upload, rendendo dunque più complicata la procedura di upload. Inoltre, la presenza di queste stringhe lunghe Base64 rendeva il file JSON decisamente poco leggibile.

Il nome dell’immagine salvata nel database viene utilizzato dal frontend per costruire il path necessario a mostrare l’immagine.

Nel caso delle Linee Guida, che non sono associate ad una classe, il path viene costruito nel modo seguente:

`{order}.{estensione}`

In questo modo si eliminano ambiguità eventuali tra immagini di un suggerimento e immagini di una linea guida.

Per poter gestire il caricamento e la cancellazione delle immagini sul filesystem nel Volume T0, è stata aggiunta la classe **ImageService** nel package **Service**, che mette a disposizione i metodi **storeImage** e **deleteImage**, che presenta anche un controllo dell'estensione del file caricato tramite **Apache Tika**.

Le estensioni supportate sono: `"image/png"`, `"image/jpeg"`, `"image/jpg"`, `"image/xbm"`, `"image/tif"`, `"image/jfif"`, `"image/ico"`, `"image/gif"`, `"image/svg"`, `"image/svgz"`, `"image/webp"`, `"image/bmp"`, `"image/pjp"`, `"image/apng"`, `"image/pjpeg"`, `"image/avif"`.

3.5 Component Diagrams

Di seguito è mostrato il diagramma dei componenti che descrive il nuovo sottosistema **GuidelineAndSuggestionSubsystem** che si occupa della gestione dei suggerimenti e delle linee guida. Sia per Suggestion che Guideline sono presenti un componente di Controller, il quale espone un'API per accedere ai suggerimenti / linee guida e richiede i servizi offerti dal service corrispondente (**GuidelineService** e **SuggestionService**). Questi ultimi usufruiscono a loro volta dei servizi offerti da **ImageService** per caricare, scaricare ed eliminare le immagini associate ai suggerimenti e alle linee guida, nonché del repository corrispondente (**GuidelineRepository** e **SuggestionRepository**), il quale accede al database **AdminDB** tramite interfaccia **JPA**.

Inoltre, **SuggestionController** richiede i servizi di **OpponentSubsystem** per associare i suggerimenti alle classi **ClassUT** corrispondenti.

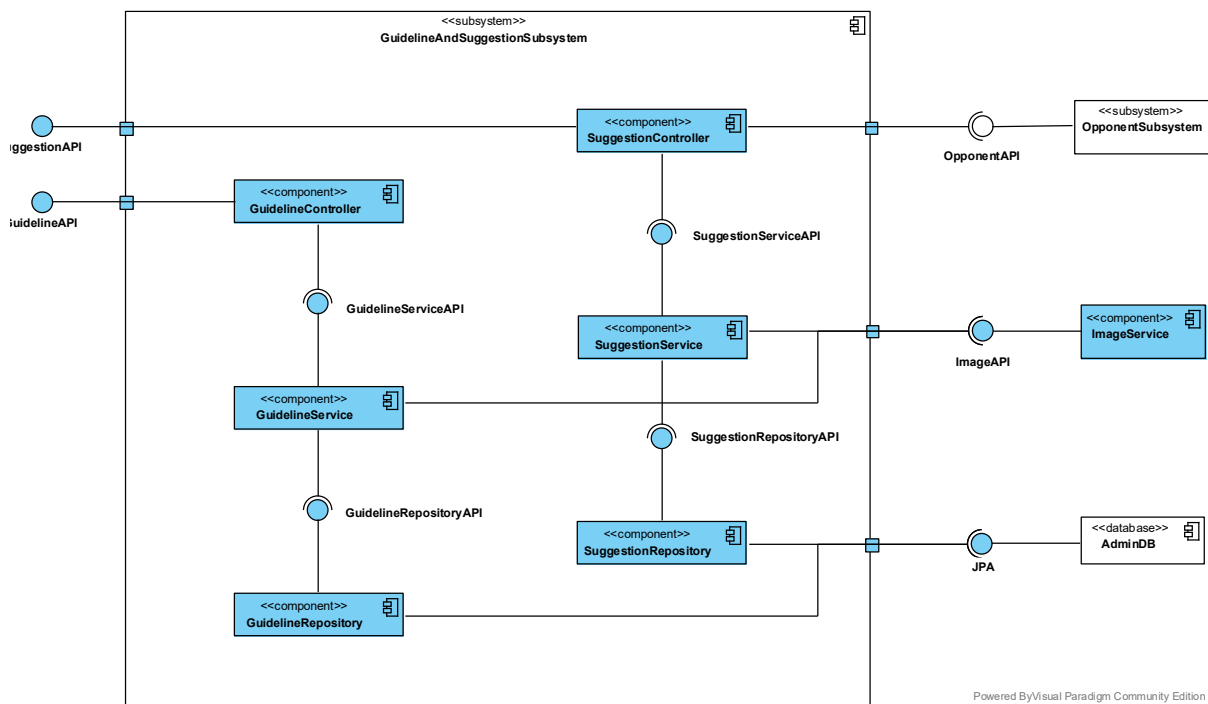


Figura 11 – Diagramma del nuovo sottosistema *GuidelineAndSuggestionSubsystem*

Il seguente diagramma mostra il sottosistema GuidelineAndSuggestionSubsystem all'interno del microservizio complessivo AdminService. Sono evidenziate le interfacce esposte dal microservizio (GuidelineAPI, SuggestionAPI e OpponentAPI) e richieste (JPA per accedere al database MySQL del microservizio AdminDB e FilesystemAPI per avere accesso al file system dove caricare, scaricare e cancellare le immagini dei suggerimenti e linee guida e classiUT). In blu sono mostrati i nuovi componenti / sottosistemi / interfacce.

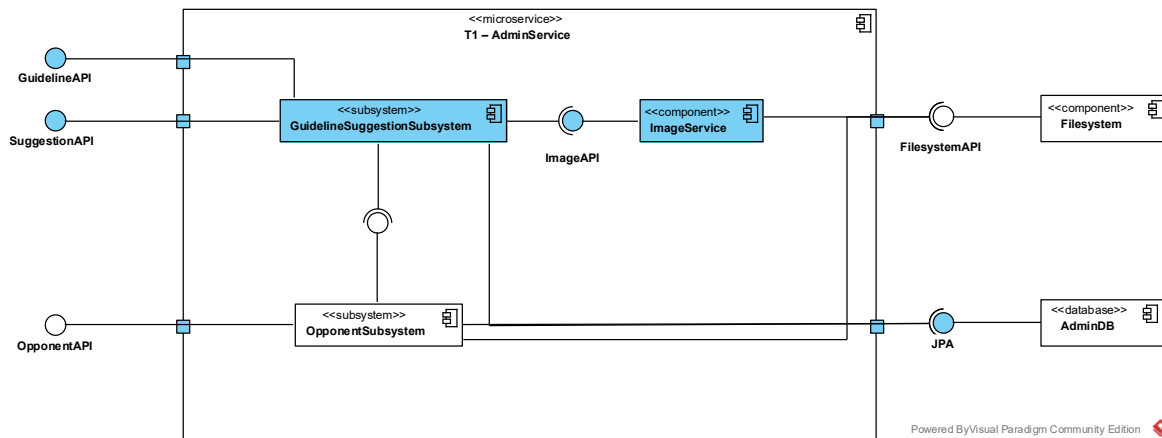


Figura 12 – Component Diagram che mostra l'interfacciamento del sottosistema GuidelineAndSuggestionSubsystem con gli altri componenti del microservizio T1

Le interfacce **GuidelineAPI** e **SuggestionAPI** sono definite con lo standard OpenAPI 3.0.1, mostrate nella sezione seguente tramite interfaccia Swagger.

3.6 REST API

Le nuove REST API sono state documentate con lo standard OpenAPI 3.0.1.

3.6.1 Criteri di accettazione

È stato scelto il formato JSON per il file caricato dall'Admin relativo all'operazione di caricamento di Suggerimenti e Linee Guida. In particolare:

- Il sistema permette il caricamento di Suggerimenti e Linee Guida solo a utenti con ruolo Admin.
- Il sistema accetta solo file JSON nella richiesta di caricamento di Suggerimenti e Linee Guida, con un formato definito in OpenAPI nella sezione successiva.
- Se il file JSON ha un formato o struttura errata, il sistema restituisce un errore 400 BAD REQUEST.
- I Suggerimenti / Linee Guida presenti nel file JSON devono seguire le regole di validazione:
 - o Il primo elemento deve avere come campo "order" il valore 1
 - o I successivi elementi devono avere il campo "order" progressivo

In caso di violazione, il sistema restituisce un errore 400 BAD REQUEST.

3.6.2 Schemi degli input

```
components:
  schemas:
    ClassUTSuggestionsDTO:
      type: object
      properties:
        className:
          type: string
          description: Il nome della classe a cui aggiungere i suggerimenti
        suggestions:
          type: array
          description: Lista dei suggerimenti da caricare
          items:
            $ref: '#/components/schemas/SuggestionDTO'
    SuggestionDTO:
      type: object
      properties:
        order:
          type: integer
          description: L'ordine progressivo del suggerimento tra quelli
della classe a cui si riferisce
        hint:
          type: string
          description: Contenuto informativo testuale del suggerimento
        level:
          type: string
          enum:
            - LOW
            - MEDIUM
            - HIGH
          description: Classificazione del suggerimento
    GuidelineDTO:
      type: object
      properties:
        order:
          type: integer
          description: L'ordine progressivo della linea guida tra tutte le
linee guida
        hint:
          type: string
          description: Contenuto informativo testuale della linea guida
```


Schemas



ClassUTSuggestionsDTO ▾ {



className

▾ **string**

Il nome della classe a cui aggiungere i suggerimenti

suggestions

▾ [

Lista dei suggerimenti da caricare

SuggestionDTO > {...}]

}

SuggestionDTO ▾ {



order

▾ **integer**

L'ordine progressivo del suggerimento tra quelli della classe a cui si riferisce

hint

▾ **string**

Contenuto informativo testuale del suggerimento

level

▾ **string**

Classificazione del suggerimento

Enum:

▾ [LOW, MEDIUM, HIGH]

}

GuidelineDTO ▾ {



order

▾ **integer**

L'ordine progressivo della linea guida tra tutte le linee guida

hint

▾ **string**

Contenuto informativo testuale della linea guida

}

3.6.3 Un esempio

Formato JSON Suggerimento

```
{
  "className": "Calcolatrice",
  "suggestions": [
    {
      "order": 1,
      "hint": "Testo Suggerimento 1",
      "level": "LOW"
    },
    {
      "order": 2,
      "hint": "Testo Suggerimento 2",
      "level": "LOW"
    },
    {
      "order": 3,
      "hint": "Testo Suggerimento 3",
      "level": "MEDIUM"
    },
    {
      "order": 4,
      "hint": "Testo Suggerimento 4",
      "level": "HARD"
    }
  ]
}
```

Formato JSON Linea Guida

```
[
  {
    "order": 1,
    "hint": "Testo Linea Guida 1"
  },
  {
    "order": 2,
    "hint": "Testo Linea Guida 2"
  },
  {
    "order": 3,
    "hint": "Testo Linea Guida 3"
  }
]
```

L'immagine a sinistra mostra un esempio di file JSON inserito nella richiesta di caricamento dei Suggerimenti per la classe Calcolatrice. I suggerimenti sono ordinati con il campo `order`, classificati con il campo `level` e presentano il campo `hint` come contenuto informativo testuale del suggerimento.

L'immagine a destra mostra un esempio di file JSON inserito nella richiesta di caricamento delle Linee Guida, valide per tutte le classi. Le linee guida sono ordinate con il campo `order` e classificate con il campo `level` e presentano il campo `hint` come contenuto informativo testuale della linea guida.

Di seguito è presente un riepilogo delle nuove REST API implementate, mostrate con l'interfaccia Swagger:

suggestion-controller



POST	/suggestions/upload	✓
POST	/suggestions/upload/{className}/{order}	✓
GET	/suggestions/{className}	✓
DELETE	/suggestions/{className}/{order}	✓
DELETE	/suggestions/image/{className}/{order}	✓

guideline-controller



POST	/guidelines/upload	✓
POST	/guidelines/upload/{order}	✓
GET	/guidelines	✓
DELETE	/guidelines/{order}	✓
DELETE	/guidelines/image/{order}	✓

3.6.4 Suggestion Controller

3.6.5 UploadSuggestions

```
/suggestions/upload:
  post:
    tags:
      - suggestion-controller
    operationId: uploadSuggestions
    description: API per effettuare l'UPSERT dei suggerimenti di una classe
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/ClassUTSuggestionsDTO"
    responses:
      "200":
        description: Suggerimenti caricati con successo.
      "404":
        description: Classe non trovata.
      "401":
        description: Unauthorized.
      "500":
        description: Si è verificato un errore interno.
```

POST

/suggestions/upload

API per effettuare l'UPSERT dei suggerimenti di una classe

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value

Schema

```
{
  "className": "Calcolatrice",
  "suggestions": [
    {
      "order": 1,
      "hint": "Testo Suggerimento 1",
      "level": "LOW"
    },
    {
      "order": 2,
      "hint": "Testo Suggerimento 2",
      "level": "MEDIUM"
    }
  ]
}
```

Responses

Code	Description	Links
200	Suggerimenti caricati con successo.	No links
401	Unauthorized.	No links
404	Classe non trovata.	No links
500	Si è verificato un errore interno.	No links

3.6.6 UploadSuggestionImage

```
/suggestions/upload/{className}/{order}:  
  post:  
    tags:  
      - suggestion-controller  
    operationId: uploadSuggestionImage  
    description: API per effettuare l'UPSERT dell'immagine di un  
suggerimento di una classe  
    parameters:  
      - name: className  
        in: path  
        required: true  
        description: Il nome della classe a cui aggiungere il suggerimento  
        schema:  
          type: string  
      - name: order  
        in: path  
        required: true  
        description: Il numero progressivo del suggerimento tra quelli della  
classe selezionata a cui aggiungere l'immagine  
        schema:  
          type: integer  
    requestBody:  
      required: true  
      content:  
        multipart/form-data:  
          schema:  
            type: object  
            properties:  
              image:  
                type: string  
                format: binary  
                description: Il file immagine da caricare (MultipartFile)  
    responses:  
      "200":  
        description: Immagine caricata con successo.  
      "404":  
        description: Classe o Suggerimento non trovati.  
      "401":  
        description: Unauthorized.  
      "500":  
        description: Si è verificato un errore interno.
```

POST

/suggestions/upload/{className}/{order}

API per effettuare l'UPSERT dell'immagine di un suggerimento di una classe

Parameters

Try it out

Name	Description
className * required string (path)	Il nome della classe a cui aggiungere il suggerimento className
order * required integer (path)	Il numero progressivo del suggerimento tra quelli della classe selezionata a cui aggiungere l'immagine order

Request body required

multipart/form-data

image

Il file immagine da caricare (MultipartFile)

string(\$binary)

Responses

Code	Description	Links
200	Immagine caricata con successo.	No links
401	Unauthorized.	No links
404	Classe o Suggerimento non trovati.	No links
500	Si è verificato un errore interno.	No links

3.6.7 GetSuggestions

```
/suggestions/{className}:  
  get:  
    tags:  
      - suggestion-controller  
    operationId: viewSuggestions  
    description: API per effettuare la READ dei suggerimenti di una classe  
    parameters:  
      - name: className  
        in: path  
        required: true  
        description: Il nome della classe a cui aggiungere il suggerimento  
        schema:  
          type: string  
    responses:  
      "200":  
        description: Suggerimenti letti con successo.  
        content:  
          application/json:  
            schema:  
              $ref: "#/components/schemas/ClassUTSuggestionsDTO"  
      "404":  
        description: Classe non trovata.  
      "401":  
        description: Unauthorized.  
      "500":  
        description: Si è verificato un errore interno.
```


GET

/suggestions/{className}

^

API per effettuare la READ dei suggerimenti di una classe

Parameters

Try it out

Name	Description
className * required string (path)	Il nome della classe a cui aggiungere il suggerimento className

Responses

Code	Description	Links
200	Suggerimenti letti con successo. Media type application/json Controls Accept header. Example Value Schema <pre>{ "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "LOW" }, { "order": 2, "hint": "Testo Suggerimento 2", "level": "MEDIUM" }]}</pre>	No links
401	Unauthorized.	No links
404	Classe non trovata.	No links
500	Si è verificato un errore interno.	No links

3.6.8 DeleteSuggestion

```
/suggestions/{className}/{order}:
  delete:
    tags:
      - suggestion-controller
    operationId: deleteSuggestion
    description: API per effettuare la DELETE di un suggerimento di una
classe
    parameters:
      - name: className
        in: path
        required: true
        description: Il nome della classe a cui cancellare il suggerimento
        schema:
          type: string
      - name: order
        in: path
        required: true
        description: Il numero progressivo del suggerimento tra quelli della
classe selezionata da cancellare
        schema:
          type: integer
    responses:
      "200":
        description: Suggerimento eliminato con successo.
      "404":
        description: Classe o Suggerimento non trovato.
      "401":
        description: Unauthorized.
      "500":
        description: Si è verificato un errore interno.
```

DELETE`/suggestions/{className}/{order}`

API per effettuare la DELETE di un suggerimento di una classe

Parameters[Try it out](#)

Name	Description
className * required string (path)	Il nome della classe a cui cancellare il suggerimento className
order * required integer (path)	Il numero progressivo del suggerimento tra quelli della classe selezionata da cancellare order

Responses

Code	Description	Links
200	Suggerimento eliminato con successo.	<i>No links</i>
401	Unauthorized.	<i>No links</i>
404	Classe o Suggerimento non trovato.	<i>No links</i>
500	Si è verificato un errore interno.	<i>No links</i>

3.6.9 DeleteSuggestionImage

```
/suggestions/image/{className}/{order}:
  delete:
    tags:
      - suggestion-controller
    operationId: deleteSuggestionImage
    description: API per effettuare la DELETE di un'immagine di un
suggerimento di una classe
    parameters:
      - name: className
        in: path
        required: true
        description: Il nome della classe a cui si riferisce il suggerimento
        schema:
          type: string
      - name: order
        in: path
        required: true
        description: Il numero progressivo del suggerimento tra quelli della
classe selezionata
        schema:
          type: integer
    responses:
      "200":
        description: Immagine del suggerimento eliminata con successo.
      "404":
        description: Classe o Suggerimento non trovato.
      "401":
        description: Unauthorized.
      "500":
        description: Si è verificato un errore interno.
```

DELETE**/suggestions/image/{className}/{order}**

API per effettuare la DELETE di un'immagine di un suggerimento di una classe

Parameters

[Try it out](#)**Name****Description****className** * required

Il nome della classe a cui si riferisce il suggerimento

string
(path)**order** * required

Il numero progressivo del suggerimento tra quelli della classe selezionata

integer
(path)

Responses

Code**Description****Links**

200

Immagine del suggerimento eliminata con successo.

No links

401

Unauthorized.

No links

404

Classe o Suggerimento non trovato.

No links

500

Si è verificato un errore interno.

No links

3.6.10 Guideline Controller

3.6.11 UploadGuidelines

```
/guidelines/upload:
  post:
    tags:
      - guideline-controller
    operationId: uploadGuidelines
    description: API per effettuare l'UPSERT delle linee guida
    requestBody:
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: "#/components/schemas/GuidelineDTO"
      required: true
    responses:
      "200":
        description: Suggerimenti caricati con successo.
      "401":
        description: Unauthorized.
      "500":
        description: Si è verificato un errore interno.
```

POST

/guidelines/upload

API per effettuare l'UPSERT delle linee guida

Parameters

Try it out

No parameters

Request body required

application/json

Example Value | Schema

```
[
  {
    "order": 1,
    "hint": "Testo Linea Guida 1"
  },
  {
    "order": 2,
    "hint": "Testo Linea Guida 2"
  }
]
```

Responses

Code	Description	Links
200	Suggerimenti caricati con successo.	No links
401	Unauthorized.	No links
500	Si è verificato un errore interno.	No links

3.6.12 UploadGuidelineImage

```
/guidelines/upload/{order}:  
  post:  
    tags:  
      - guideline-controller  
    operationId: uploadGuidelineImage  
    description: API per effettuare l'UPSERT dell'immagine di una linea guida  
    parameters:  
      - name: order  
        in: path  
        required: true  
        description: Il numero progressivo della linea guida a cui aggiungere l'immagine  
    schema:  
      type: string  
    requestBody:  
      required: true  
      content:  
        multipart/form-data:  
          schema:  
            type: object  
            properties:  
              image:  
                type: string  
                format: binary  
                description: Il file immagine da caricare (MultipartFile)  
    responses:  
      "200":  
        description: Immagine caricata con successo.  
      "404":  
        description: Linea guida non trovata.  
      "401":  
        description: Unauthorized.  
      "500":  
        description: Si è verificato un errore interno.
```


POST

/guidelines/upload/{order}

^

API per effettuare l'UPSERT dell'immagine di una linea guida

Parameters

Try it out

Name	Description
order * required string (path)	Il numero progressivo della linea guida a cui aggiungere l'immagine

Request body required

multipart/form-data

image	Il file immagine da caricare (MultipartFile)
string(\$binary)	

Responses

Code	Description	Links
200	Immagine caricata con successo.	No links
401	Unauthorized.	No links
404	Linea guida non trovata.	No links
500	Si è verificato un errore interno.	No links

3.6.13 GetGuidelines

```
/guidelines:
  get:
    tags:
      - guideline-controller
    operationId: viewGuidelines
    description: API per effettuare la READ delle linee guida
    responses:
      "200":
        description: Linee guida caricate con successo.
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: "#/components/schemas/GuidelineDTO"
      "401":
        description: Unauthorized.
      "500":
        description: Si è verificato un errore interno.
```

GET /guidelines ^

API per effettuare la READ delle linee guida

Parameters Try it out

No parameters

Responses

Code	Description	Links
200	Linee guida caricate con successo. Media type application/json Controls Accept header. Example Value Schema <pre>[{ "order": 0, "hint": "string" }]</pre>	No links
401	Unauthorized.	No links
500	Si è verificato un errore interno.	No links

3.6.14 DeleteGuidelines

```
/guidelines/{order}:
  delete:
    tags:
      - guideline-controller
    operationId: deleteGuideline
    description: API per effettuare la DELETE di una linea guida
    parameters:
      - name: order
        in: path
        required: true
        description: Il numero progressivo della linea guida da cancellare.
        schema:
          type: integer
    responses:
      "200":
        description: Linea guida eliminata con successo.
      "404":
        description: Linea guida non trovato.
      "401":
        description: Unauthorized.
      "500":
        description: Si è verificato un errore interno.
```

DELETE /guidelines/{order} ^

API per effettuare la DELETE di una linea guida

Parameters Try it out

Name	Description
order * required integer (path)	Il numero progressivo della linea guida da cancellare. order

Responses

Code	Description	Links
200	Linea guida eliminata con successo.	No links
401	Unauthorized.	No links
404	Linea guida non trovato.	No links
500	Si è verificato un errore interno.	No links

3.6.15 DeleteGuidelineImage

```
/guidelines/image/{order}:  
  delete:  
    tags:  
      - guideline-controller  
    operationId: deleteGuidelineImage  
    description: API per effettuare la DELETE dell'immagine associata ad una  
    linea guida  
    parameters:  
      - name: order  
        in: path  
        required: true  
        description: Il numero progressivo della linea guida a cui  
        cancellare l'immagine.  
    schema:  
      type: integer  
    responses:  
      "200":  
        description: Immagine della linea guida eliminata con successo.  
      "404":  
        description: Linea guida non trovato.  
      "401":  
        description: Unauthorized.  
      "500":  
        description: Si è verificato un errore interno.
```

DELETE**/guidelines/image/{order}**

API per effettuare la DELETE dell'immagine associata ad una linea guida

Parameters

[Try it out](#)**Name****Description****order** * required

Il numero progressivo della linea guida a cui cancellare l'immagine.

integer

(path)

Responses

Code**Description****Links**

200

Immagine della linea guida eliminata con successo.

No links

401

Unauthorized.

No links

404

Linea guida non trovato.

No links

500

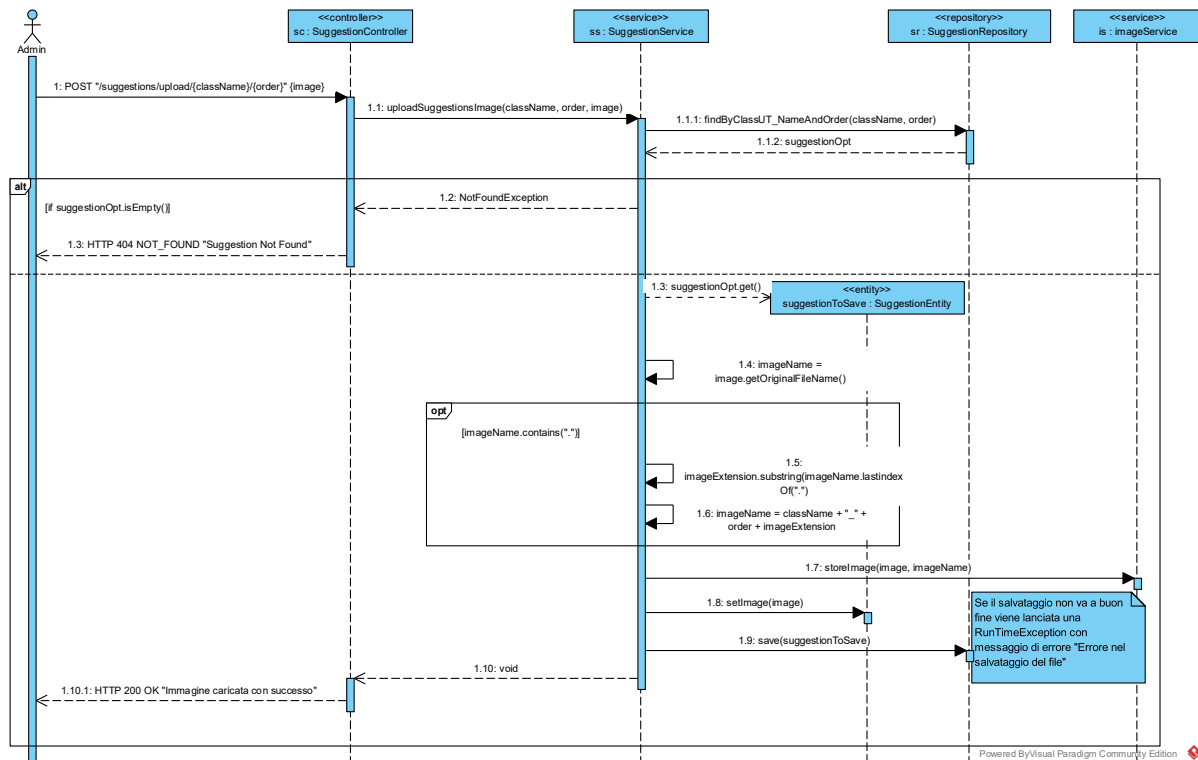
Si è verificato un errore interno.

No links

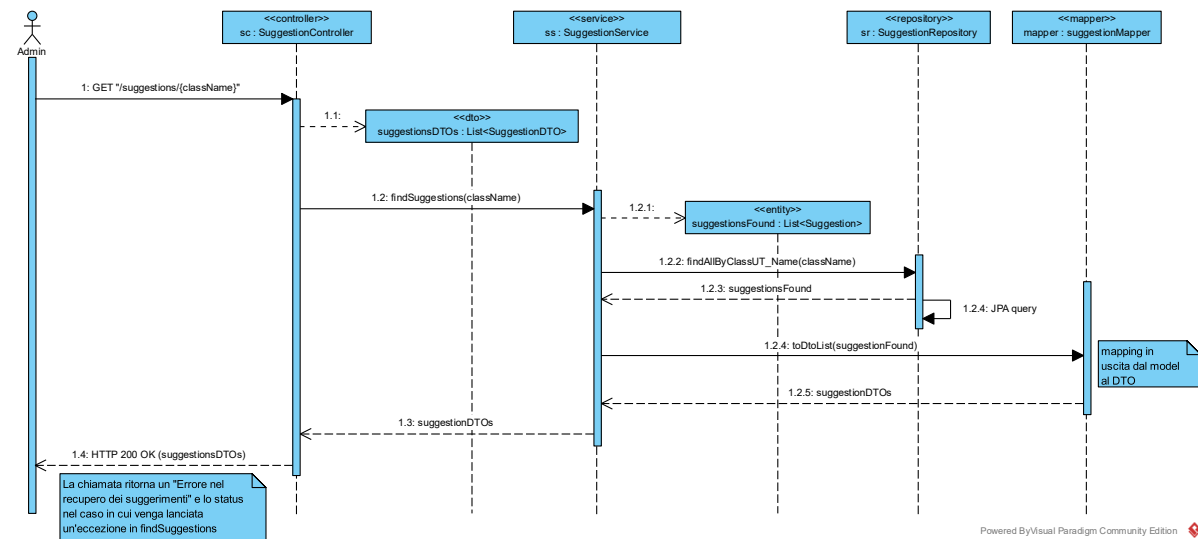
Sono presentati in seguito i diagrammi di sequenza rappresentanti le operazioni effettuate dal sistema in relazione ai Suggerimenti. Sono stati omessi i diagrammi di sequenza delle Linee Guida per la loro similitudine a quelli dei Suggerimenti.

[illegible]

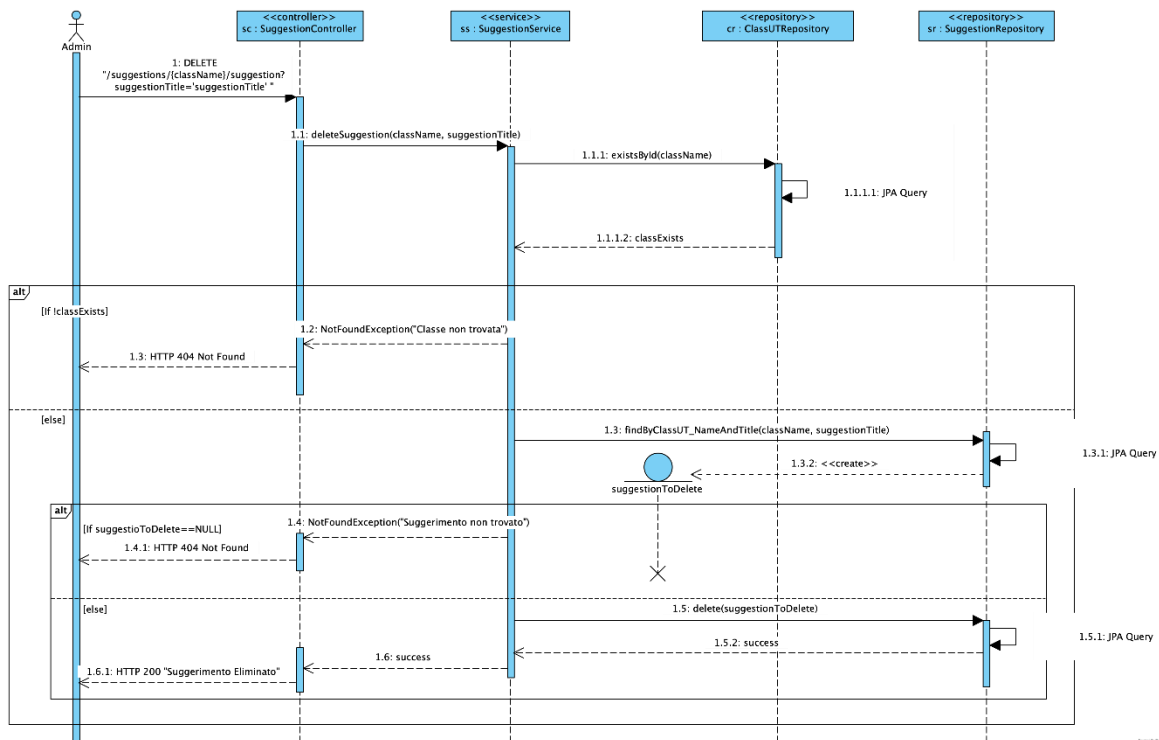
3.7.2 UploadSuggestionImage



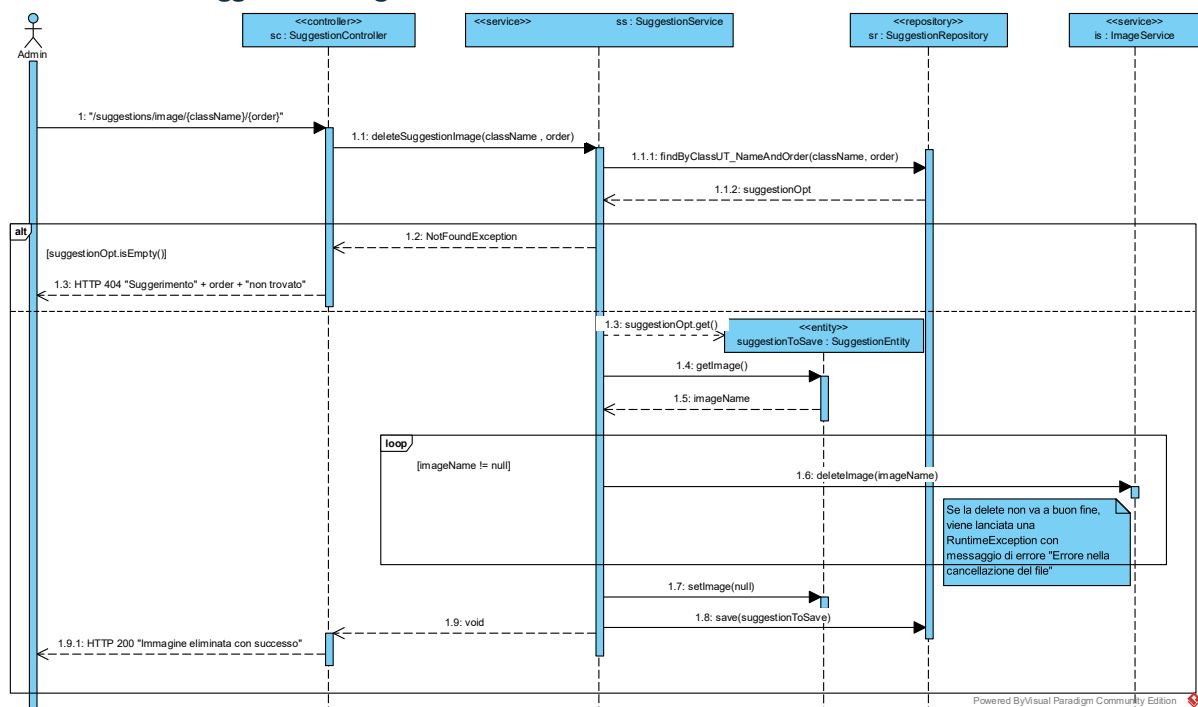
3.7.3 GetSuggestions



3.7.4 DeleteSuggestion



3.7.5 DeleteSuggestionImage



4. Implementazione e struttura del progetto modificato

4.1 Refactoring

Qui sono presentati alcuni problemi che sono stati rilevati nel sistema e risolti durante lo sviluppo.

4.1.1 Anti-pattern individuati

- Violazione del principio di Single Responsibility in MVC: La generazione della risposta HTTP era delegata ai Service in maniera non conforme al pattern MVC. La creazione delle risposte ora è stata spostata nei Controller.
- Hardcoded secrets
- Inadequate authentication / authorization

4.1.2 Gestione delle eccezioni centralizzata

Per migliorare la modularità e la chiarezza del codice, è stata introdotta la classe **GlobalExceptionHandler**, che ha il compito di gestire tutte le eccezioni generate all'interno del microservizio. Tale approccio segue il pattern *Centralized Exception Handling*, che risulta utile nel separare la gestione degli errori dalla logica applicativa. Tale classe cattura tutte le eccezioni lanciate dal sistema e non gestite, restituendo l'opportuna risposta HTTP.

4.1.3 Security

Ad ogni richiesta HTTP va verificato che l'utente sia autenticato. In precedenza, ogni metodo dei vari Controller controllava l'autenticazione effettuando **manualmente un controllo del token JWT**. Tuttavia, il package security **era già predisposto a effettuare questo controllo** per ogni richiesta entrante, prima che raggiungesse il Controller, rendendo di conseguenza ridondante il secondo controllo.

Tale controllo è effettuato dalla classe (filter Spring) **AuthTokenFilter**, che intercetta in automatico ogni richiesta HTTP e verifica l'autenticazione.

4.1.4 AuthTokenFilter

Nella vecchia versione il controller presentava l'input del jwt per effettuare un controllo di sicurezza ridondante. È mostrato un esempio di metodo di un controller prima del refactoring:

```
@PostMapping("/URI/RICHIESTA")
public ResponseEntity<?> aControllerMethod(
    aType anInputParameter,
    @CookieValue(name = "jwt", required = false) String jwt)
{
    // ...
    return aService.aServiceMethod(anInputParameter, jwt);
}
```

Il service, invece, gestiva il controllo della validità del token e generava la risposta HTTP. È mostrato un esempio di metodo di un Service prima del refactoring:

```
public ResponseEntity<?> aServiceMethod(
    aType anInputParameter,
    String jwt)
{
    if (!jwtService.isJwtValid(jwt)) {
        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Unauthorized");
    }
    // ... logica del service ...
    return ResponseEntity.ok("...");
}
```

Ogni metodo, dunque, effettuava un controllo ridondante del JWT, presentando l'input: @CookieValue(name = "jwt", required = false) String jwt. Tale controllo era delegato al Service, il quale dovrebbe occuparsi della sola logica di business. I metodi del service restituivano risposte ResponseEntity<?>, mentre dovrebbe essere compito del Controller generare la risposta HTTP in base all'occorrenza.

Ora il controller prende in input solo gli argomenti della richiesta, per poi passali al Service, e determina quale risposta HTTP ResponseEntity<?> restituire, con l'aiuto del GlobalExceptionHandler per le risposte HTTP in caso di eccezioni (tra cui UnauthorizedException).

```
@PostMapping("/URI/RICHIESTA")
public ResponseEntity<?> aControllerMethod(aType anInputParameter) {
    // ...
    aService.aServiceMethod(anInputParameter);
    // ...
    return ResponseEntity.ok("...");
}
```

Il service effettua solo la logica di business del sistema.

```
public void aServiceMethod(aType anInputParameter) {
    // ... logica del service ...
}
```

Ora il GlobalExceptionHandler cattura a monte, automaticamente, l'eccezione UnauthorizedException sollevata da AuthTokenFilter:

```
@ExceptionHandler(UnauthorizedException.class)
public ResponseEntity<String> handleUnauthorized(UnauthorizedException ex)
{
    return ResponseEntity
        .status(HttpStatus.UNAUTHORIZED)
        .body(ex.getMessage());
}
```

}

Il flusso di validazione della validazione effettuata da **AuthTokenFilter** è mostrato nel seguente diagramma di sequenza.

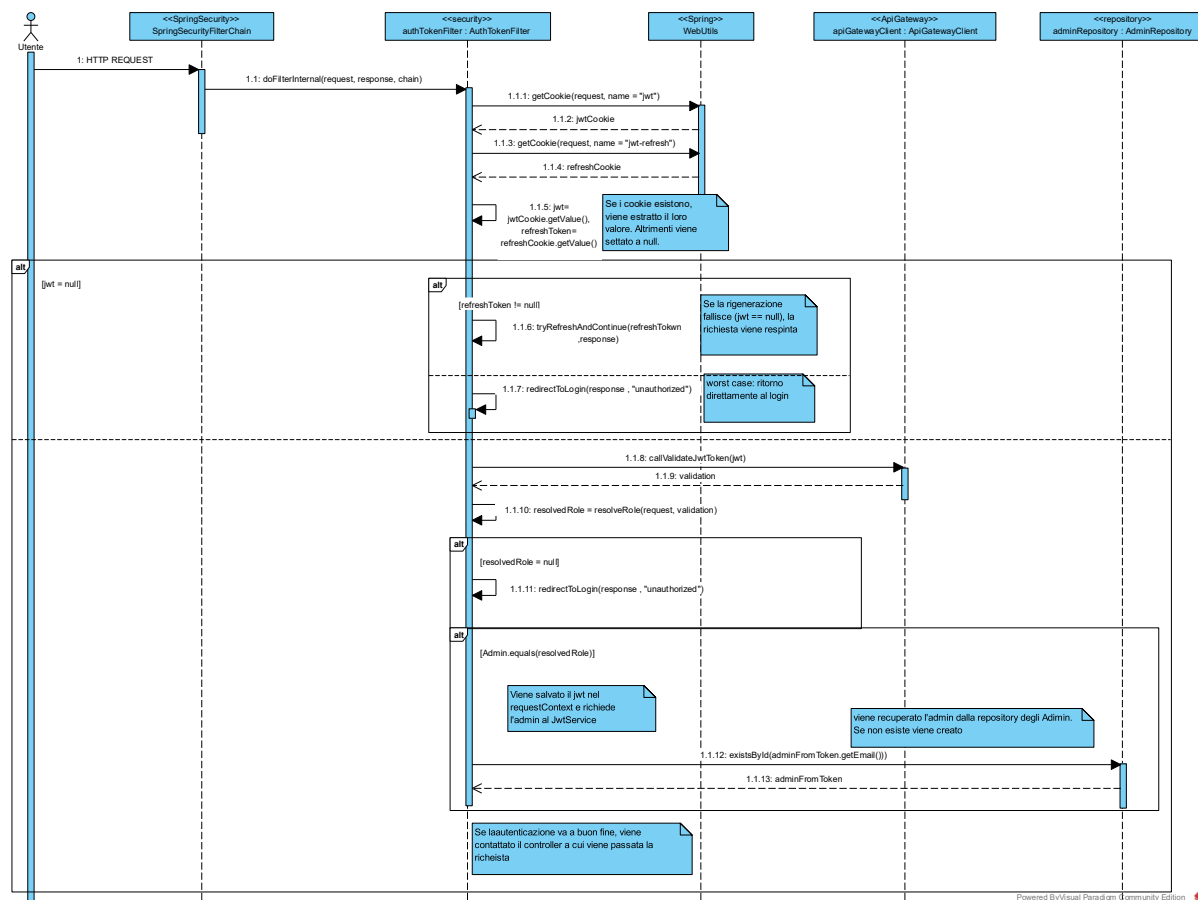


Figura 13 – Flusso di validazione di AuthTokenFilter

4.2 Descrizione delle modifiche al microservizio T1

Di seguito sono elencate tutte le modifiche, file per file, effettuate nel microservizio T1 per completare la migrazione e aggiungere le funzionalità richieste. Le modifiche coinvolgono sia file di codice del backend (.java), sia del frontend (.js, .html), sia file di configurazione (pom, env, etc.)

4.2.1 File di configurazione

File	Descrizione	Creato o Modificato
.env	Sostituiti username e password MONGODB_ROOT con equivalenti per MYSQL (MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD)	Modificato
docker-compose.yml	Sono state sostituite le variabili di ambiente per la connessione MongoDB (MONGODB_ROOT_PASSWORD e MONGODB_ROOT_USERNAME) con quelle per Spring Data e MySQL (SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD). Modificato il campo <i>depends_on</i> da "mongo_db" a "mysql_db".	Modificato

	Sostituito il servizio “mongo_db” con “mysql_db”, con la configurazione necessaria (variabili d’ambiente per password e user, porta esposta 3306, volume “/var/lib/mysql”)	
pom.xml	- aggiornamento di Spring Boot da 2.7.0 a 3.2.0 per poter effettuare la migrazione Jakarta e conseguente aggiornamento delle versioni di altre dipendenze spring - sostituzione di <i>spring-boot-starter-data-mongodb</i> in <i>spring-boot-starter-data-jpa</i> per la migrazione a jakarta - aggiunto <i>mysql-connetor-java</i> per la connessione a mysql - aggiunto spring-boot-starter-validation per abilitare le operazioni di validazione dei vari campi - aggiunta la dipendenza a MapStruct per generare i Mapper tra DTO e Model in maniera automatizzata - aggiunta la dipendenza a Lombok per generare getter, setter, costruttori e altro in maniera automatizzata - aggiunto h2 come database in-memory per i test - aggiunto spring-boot-starter-test e mockito-core per i test - aggiunto Apache Tika per validare i file ricevuti come immagini	Modificato
application.properties	Aggiunto l’import del file .env. Sostituite le configurazioni di mongodb con quelle di mysql, jpa e hibernate. Creata la variabile images.storage-path per il path delle immagini nel volume T0	Modificato
- message_it.properties - message_en.properties - message.properties	Aggiunte nuove coppie chiave-valore per consentire internazionalizzazione dell’applicazione riguardo alle nuove funzionalità (suggerimenti e linee guida) e ai messaggi di errore	Modificato

4.2.2 Frontend

File	Descrizione	Creato o Modificato
opponents_main.js	Aggiunta la sezione per la visualizzazione delle linee guida nella <i>navigation bar</i>. Small fix per mostrare l’errore internazionale. Modificati i nomi difficoltà degli opponents da Beginner/Intermediate/Advanced a Easy/Medium/Hard per essere coerente con i nomi presenti nelle classi.	Modificato
api.js	Aggiunte le funzioni che si appoggiano al backend per i servizi legati ai suggerimenti e alle linee guida: - callDeleteSuggestion - callDeleteSuggestionImage - callUploadSuggestions - callUploadSuggestionImage - callGetSuggestions - callDeleteGuideline - callDeleteGuidelineImage - callUploadGuidelines - callUploadGuidelineImage - callGetGuidelines - callDownloadSuggestions - callDownloadGuidelines	Modificato
endpoints.js	Aggiunti gli endpoints che rappresentano i path del backend relative alle nuove funzionalità aggiunte in api.js: - UPLOAD_GUIDELINES - UPLOAD_GUIDELINE_IMAGE - DOWNLOAD_GUIDELINE - GET_GUIDELINES - DELETE_GUIDELINE - DELETE_GUIDELINE_IMAGE - UPLOAD_SUGGESTIONS - UPLOAD_SUGGESTION_IMAGE	Modificato

	- DOWNLOAD_SUGGESTION - GET_SUGGESTIONS - DELETE_SUGGESTION - DELETE_SUGGESTION_IMAGE Aggiunta costante IMAGE_BASE_URL corrispondente al path utilizzato per accedere alle immagini nel VolumeT0 (opportunamente mappata in ResourceConfig.java nel backend) Modificati i nomi difficoltà degli opponents da Beginner/Intermediate/Advanced a Easy/Medium/Hard per essere coerente con i nomi presenti nelle classi.	
opponents_edit.html	Modificata senza particolari aggiunte	Modificato
opponents_main.html	Aggiunta la sezione sotto ogni classe per mostrare la lista dei suggerimenti, insieme a: - Bottone per scaricare il file json contenente i suggerimenti di una classe - Bottone per effettuare l'upload dei suggerimenti ad una classe - Bottone per effettuare la cancellazione di un singolo suggerimento - Bottone per effettuare l'upload dell'immagine di un singolo suggerimento - Riquadro per mostrare l'eventuale immagine associata ad un suggerimento, con annesso bottone per cancellare - Aggiunto il "badge" per la visualizzazione del livello di aiuto di un suggerimento (LOW, MEDIUM, HIGH) Refactoring di codice già presente	Modificato
opponents_upload.html	Risolto un bug che non permetteva di mostrare correttamente i nomi delle difficoltà degli opponents in lingua inglese, con la sostituzione dei nomi dell'enum da Beginner/Intermediate/Advanced a EASY/MEDIUM/HARD.	Modificato
guidelines_main.html	Nuova vista per mostrare la lista delle linee guida, insieme a: - Bottone per scaricare il file json contenente le linee guida - Bottone per effettuare l'upload delle linee guida - Bottone per effettuare la cancellazione di una singola linea guida - Bottone per effettuare l'upload dell'immagine di una singola linea guida - Riquadro per mostrare l'eventuale immagine associata ad una linea guida, con annesso bottone per cancellare	Creato
guidelines.js	Aggiunte le funzioni: - fetchAndDisplayGuidelines per ottenere e visualizzare la lista delle linee guida - deleteGuideline per effettuare la cancellazione di una linea guida - handleGuidelinesUpload per effettuare l'upload delle linee guida - uploadGuidelineImage per effettuare l'upload dell'immagine di una linea guida - downloadGuidelines per effettuare il download del file json contenente le linee guida - deleteGuidelineImage per effettuare la cancellazione delle immagini di una linea guida	Creato
suggestions.js	Aggiunte le funzioni: - fetchAndDisplaySuggestions per ottenere e visualizzare la lista dei suggerimenti - deleteSuggestion per effettuare la cancellazione di un suggerimento - handleSuggestionUpload per effettuare l'upload dei suggerimenti - uploadSuggestionImage per effettuare l'upload dell'immagine di un suggerimento	Creato

	- downloadSuggestions per effettuare il download del file json contenente i suggerimenti di una classe - deleteSuggestionImage per effettuare la cancellazione delle immagini di un suggerimento	
--	---	--

4.2.3 Backend

Le modifiche sono state raggruppate per package.

4.2.4 Package api

File	Descrizione	Creato o Modificato
ApiGatewayClient.java	Modificati gli import relativi ai dto e javax aggiornato a jakarta per adeguarsi alla migrazione a Spring Boot 3 / Jakarta EE. Al fine di non propagare il jwt token dal Controller al Service fino all'ApiGatewayClient (per le richieste da mandare all'esterno, per esempio verso T23), il token jwt viene ora recuperato da JwtRequestContext laddove necessario, centralizzando in questo modo la gestione del token. Quindi è stato reso necessario un refactoring riguardo la gestione del jwt, il quale non viene più passato come parametro esplicito ai metodi di ApiGatewayClient, ma viene: - recuperato automaticamente dal metodo <code>JwtRequestContext.getJwtToken()</code> - inserito nell'header HTTP come COOKIE prima di ogni chiamata In caso di token mancante viene sollevata un'eccezione. Small fix di nomi di alcune variabili per rendere il codice più leggibile e meno ambiguo (esempio: <code>classUT</code> rinominato in <code>className</code>, perché non è un'istanza di <code>ClassUT</code> ma solo il nome). Sostituita l'autenticazione Bearer con quella tramite cookie JWT, come richiesto da T23.	Modificato

4.2.5 Package config

File	Descrizione	Creato o Modificato
ResourceConfig.java	È stata aggiunta questa classe per configurare Spring MVC in modo da esporre le immagini relative a suggerimenti e linee guida, presenti sul file system, come risorse http accessibili dal frontend. Per farlo: - utilizza annotazione <code>@Configuration</code>, che indica a Spring che la classe è un elemento di configurazione del contesto applicativo - implementa <code>WebMvcConfigurer</code> per permettere di personalizzare il comportamento di default di SpringMVC, di cui effettua l'override del metodo <code>addResourceHandlers</code> - nel metodo <code>@Override addResourceHandlers</code> per registrare un "resource handler", ovvero una regola che comunica a Spring di mappare tutte le richieste intercettate nel path di tipo <code>"/t1/images/"</code> al path location = <code>"file:/VolumeT0/FolderTree/Images/"</code>, corrispondente al percorso delle immagini nel file system presente nel volume T0. Ad esempio la richiesta <code>"GET /t1/images/Calcolatrice_1.png"</code> viene mappata in <code>"/VolumeT0/FolderTree/Images/Calcolatrice_1.png"</code>. In particolare, il percorso del file system <code>"/VolumeT0/FolderTree/Images/"</code> viene prelevato dal file <code>application.properties</code> con l'annotazione di spring <code>@Value</code>.	Creato

4.2.6 Package validation

Questo package è stato aggiunto per gestire la logica di validazione personalizzata implementata, non direttamente fornita dalle annotazioni di jakarta.validation. In particolare attualmente contiene la classe di validazione per gestire la sequenzialità degli ordini in una lista di SuggestionDTO e GuidelineDTO, insieme alla definizione dell'annotazione apposita.

File	Descrizione	Creato o Modificato
ValidOrder.java	Definisce l'annotazione <code>@ValidOrder</code> che può essere utilizzata su un qualunque parametro di una funzione o campo di una classe (indicata con <code>@Target({ ElementType.PARAMETER, ElementType.FIELD })</code>) per indicare a Spring di effettuare la validazione a runtime (<code>@Retention(RetentionPolicy.RUNTIME)</code>) specificata dalla classe <code>OrderValidator</code> (impostata con <code>@Constraint(validatedBy = OrderValidator.class)</code>).	Creato
OrderValidator.java	Implementa <code>ConstraintValidator<ValidOrder, List<? extends GuidelineDTO>></code> per specificare la logica di validazione di un <code>List<T></code> con <code>T</code> che può essere <code>GuidelineDTO</code> o una qualunque classe che la estende (in questo caso solo <code>SuggestionDTO</code>), in modo da presentare il campo "order". In particolare, effettua l'override del metodo "isValid", il quale restituisce true se la lista ricevuta supera la validazione, false altrimenti. Nel caso in esame è richiesto che, in corrispondenza del campo "order", il primo elemento della lista presenti valore 1 e che gli elementi successivi presentino valori interi progressivi 2, 3, 4, Il presente validatore interpreta una lista null e una lista vuota come valide dal punto di vista dell'ordinamento.	Creato

4.2.7 Package util

Dalla classe `Util.java` sono stati estratti i metodi relativi alla classe `Interaction`, i quali sono stati spostati nell'apposito servizio `InteractionService`, al fine di migliorare la modularità.

4.2.8 Package security

File	Descrizione	Creato o Modificato
AuthTokenFilter.java	Aggiunta una strategia di sincronizzazione dell'admin nel database locale a partire dal token jwt validato dal microservizio T23, in quanto l'admin è necessario per il database mysql perché associato a numerose classi tramite chiave esterna.	Modificato
FilterConfig.java	Ora il metodo <code>authTokenFilterRegistration</code> prende in input direttamente l'oggetto <code>AuthTokenFilter</code> anziché <code>l'ApiGatewayClient</code> .	Modificato

4.2.9 Package model

Questo package definisce le classi model che astraggono le tabelle del database. Hanno quindi subito una radicale modifica per poter effettuare la migrazione da MongoDB a jakarta JPA.

La maggior parte delle modifiche è comune a tutti i model. Per ogni model:

- sono stati sostituiti gli import relativi a `mongodb` con `jakarta`, in particolare sostituendo l'annotazione di `mongodb @Document` con quelle di `jakarta @Entity` e `@Table` (quest'ultima per specificare il nome della tabella corrispondente nel database)
- è stata utilizzata l'annotazione `@Id` per specificare il campo legato alla chiave primaria, eventualmente insieme `@GeneratedValue` per specificare la generazione automatica dell'id nei casi in cui la chiave primaria non sia una stringa.
- Sono state utilizzate le annotazioni necessarie per indicare le relazioni SQL nelle classi model:

- *@ManyToOne* per indicare che un campo è legato ad una relazione molti ad uno (ovvero è un riferimento ad un oggetto della classe Model con cui è presente la relazione). Utilizzata in congiunzione con *@JoinColumn(name = "nomeFK", referencedColumnName = "nomePKriferita")* per specificare che il campo riferito è mappato nella tabella come chiave esterna ad un'altra tabella, permettendo di indicare il nome della colonna che costituisce la chiave esterna (*nomeFK*) e il nome della chiave primaria nella tabella riferita (*nomePKriferita*).
- *@OneToMany(mappedBy = "nomeCampo")* per indicare che un campo è legato ad una relazione uno a molti (ovvero è una lista di riferimenti a oggetti della classe Model con cui è presente la relazione), indicata con una associazione corrispondente *@ManyToOne* sul campo "*nomeCampo*" della classe Model associata. Eventualmente definisce il cascade type come REMOVE per indicare il trigger ON DELETE CASCADE sul campo del database (permettendo la rimozione automatica dal database di tutte le tuple delle tabelle che presentano una chiave esterna verso il campo in questione).
- *@ManyToMany* per indicare che un campo è legato ad una relazione molti a molti (ovvero è una lista di riferimenti a oggetti della classe Model con cui è presente la relazione). Utilizzata in congiunzione con *@JoinTable()* per indicare la tabella associativa creata nel database per rappresentare la relazione molti a molti.
- Utilizzo delle annotazioni di lombok per generare automaticamente metodi getter, setter, costruttori senza parametri e con parametri, rimuovendo di conseguenza tutto il codice relativo ai metodi generati automaticamente da tali annotazioni.
- Utilizzo di *@Column* per indicare eventualmente il nome della colonna corrispondente al campo nella tabella (qualora sia diverso dal nome del campo) e anche per indicare ulteriori constraint sul campo come UNIQUE e NOT NULL.

File	Descrizione	Creato o Modificato
Achievement.java	Classe model associata alla tabella ACHIEVEMENTS	Modificato
Admin.java	Classe model associata alla tabella ADMINS	Modificato
Assignment.java	Classe model associata alla tabella ASSIGNMENTS	Modificato
Category.java	Classe model associata alla tabella CATEGORIES	Creato
ClassUT.java	Classe model associata alla tabella CLASSES_UT	Modificato
ClassUTScalata.java	Classe model associata alla tabella CLASSES_SCALATE. È stata creata come classe model che astrae la tabella associativa tra CLASSES_UT e SCALATE, in modo da poter specificare anche i campi dell'associazione (<i>level</i> e <i>timeLimit</i>). Quindi presenta due campi "ClassUT classUT" e "Scalata scalata" con annotazione <i>@ManyToOne</i> e <i>@JoinColumn</i> per configurare una relazione molti ad uno con le classi Model associate alle tabelle riferite. Utilizza l'annotazione <i>@EmbeddedId</i> per specificare che la chiave primaria è la classe <i>@Embeddable ClassUtScalataId</i> (necessaria per indicare una chiave primaria composta dalle due chiavi esterne) e quindi l'annotazione <i>@MapsId</i> sui rispettivi campi delle chiavi esterne classUT e scalata per associarli al campo dell'oggetto ClassUTScalataId corrispondente (rispettivamente "className" e "scalataName")	Creato
ClassUTScalataId.java	È una classe <i>@Embeddable</i> che implementa <i>Serializable</i> , utilizzata da ClassUTScalata come chiave primaria composta.	Creato
Guideline.java	Classe model associata alla tabella GUIDELINES	Creato
Interaction.java	Classe model associata alla tabella INTERACTIONS	Modificato
InteractionType.java	Classe enum per definire le possibili tipologie di interazione (REPORT e LIKE), frutto di un refactoring che migliora la manutenibilità e modificabilità del sistema (in quanto prima era presenta un semplice intero 0 per report e 1 per like; quindi, richiedeva una conoscenza di questi "numeri magici")	Creato
Operation.java	Classe model associata alla tabella OPERATIONS	Modificato
OperationType.java	Classe enum per definire le possibili tipologie di operazioni (UPLOAD, DELETE, UPDATE), frutto di un refactoring che migliora la manutenibilità e modificabilità del sistema (in quanto prima era presenta un semplice intero 0 per upload, 1 per delete, 2 per update; quindi, richiedeva una conoscenza di questi "numeri magici")	Creato
Opponent.java	Classe model associata alla tabella OPPONENTS	Modificato

Scalata.java	Classe model associata alla tabella SCALATA	Modificato
Suggestion.java	Classe model associata alla tabella SUGGESTIONS	Creato
SuggestionLevel.java	Classe enum per definire le possibili tipologie di livelli di aiuto di un suggerimento	Creato
Team.java	Classe model associata alla tabella TEAMS. Data la non esistenza della classe model Student (in quanto il microservizio T1 si è utilizzato solo dagli admin, mentre la tabella degli Student è memorizzata nel database presente in T23), sono state utilizzate alcune annotazioni per specificare il comportamento del campo "List<String> studentIds" della classe Team, che dovrebbe in linea teorica rappresentare una lista di chiavi esterne alla tabella degli studenti. Non avendo a disposizione la classe Model Student, utilizza le seguenti annotazioni per specificare un comportamento di relazione uno a molti con i vari studenti senza avere a disposizione la tabella students: - @ElementCollection - @CollectionTable	Modificato
TeamAdmin.java	Classe associativa tra Team e Admin non necessaria perché generata attraverso le associazioni di jakarta.	Eliminato

A seguito di un refactoring, sono stati spostati i package repository e dto in com.groom.manvsclass, mentre prima erano contenuti nel package com.groom.manvsclass.model.

4.2.10 Package repository

Questo package definisce le interfacce che estendono *JpaRepository<TModel, TId>* al posto di *MongoRepository*, utilizzate dal layer Service per effettuare le operazioni CRUD sul database e anche per definire query personalizzate in linguaggio JPQL, utilizzando l'annotazione *@Query*. Questo package è stato estratto da com.groom.manvsclass.model e spostato in com.groom.manvsclass, in modo da rispecchiare il diagramma dei package.

File	Descrizione	Creato o Modificato
AchievementRepository.java	Interfaccia Repository associata alla classe Achievement.	Creato
AdminRepository.java	Interfaccia Repository associata alla classe Admin.	Modificato
AssignmentRepository.java	Interfaccia Repository associata alla classe Assignment.	Modificato
ClassUTRepository.java	Interfaccia Repository associata alla classe ClassUT. Rinominato da "ClassRepository.java" per chiarezza	Modificato
GuidelineRepository.java	Interfaccia Repository associata alla classe Guideline.	Creato
InteractionRepository.java	Interfaccia Repository associata alla classe Interaction.	Modificato
MongoRepository.java	Classe vuota non utilizzata.	Eliminato
OperationRepository.java	Interfaccia Repository associata alla classe Operation.	Modificato
OpponentRepository.java	Interfaccia Repository associata alla classe Opponent.	Modificato
OpponentRepositoryImpl.java	Classe eliminata, che definiva query personalizzate effettuate in MongoDB per gli Opponent.	Eliminato
ScalataRepository.java	Interfaccia Repository associata alla classe Scalata.	Creato
SearchRepository.java	Interfaccia eliminata, che veniva utilizzata per dichiarare query personalizzate in MongoDB, implementata in SearchRepositoryImpl.	Eliminato
SearchRepositoryImpl.java	Classe eliminata, che definiva query personalizzate effettuate in MongoDB di utilizzo generale, che è stata eliminata in quanto le query corrispondenti sono state migrate nelle apposite interfacce Repository, implementate in linguaggio JPQL.	Eliminato
TeamAdminRepository.java	Interfaccia eliminata, che veniva utilizzata per dichiarare query in MongoDB relative alla tabella associativa TeamAdmin.	Eliminato
TeamRepository.java	Interfaccia Repository associata alla classe Team.	Creato

4.2.11 Package service

Il package service definisce i servizi invocati dai Controller per realizzare la logica di business, interagendo con il Repository per effettuare operazioni con il database. Per quanto riguarda le classi Service già presenti, le modifiche riguardano i seguenti aspetti:

- Refactoring legato alla separazione delle responsabilità, in quanto alcuni metodi restituivano direttamente risposte HTTP di tipo `ResponseEntity<?>`
- Refactoring legato all'autenticazione con il token jwt, che è stata rimossa in vista della presenza del filtro `AuthTokenFilter`
- Refactoring necessario ad uniformarsi ai nuovi repository JPA in seguito alla migrazione del database.

File	Descrizione	Creato o Modificato
AdminService.java	Refactoring	Modificato
AssignmentService.java	Refactoring	Modificato
ClassUTService.java	Refactoring	Modificato
EmailService.java	Refactoring	Modificato
GuidelineService.java	Definisce la business logic relativi alle operazioni sulle linee guida: - <code>uploadGuidelines</code> per caricare le linee guida nel database, effettuando una update se già presenti - <code>uploadGuidelineImage</code> per caricare un'immagine associata ad una linea guida, tramite <code>ImageService</code>. Qualora sia presente già un'immagine, questa viene prima cancellata dal file system. - <code>findGuidelines</code> restituisce una lista contenente tutte le linee guida presenti nel database - <code>deleteGuideline</code> per cancellare una linea guida con un opportuno valore del campo "order". Qualora sia associata ad un'immagine, questa viene prima cancellata dal file system. - <code>deleteGuidelineImage</code> per cancellare l'immagine associata ad una linea guida, tramite <code>ImageService</code>. Tutti i metodi utilizzano all'occorrenza <code>GuidelineMapper</code> per convertire gli oggetti in input DTO negli oggetti corrispondenti Model / gli oggetti in output Model negli oggetti corrispondenti DTO	Creato
ImageService.java	Definisce i metodi per l'accesso al filesystem relativo alle operazioni per gestire le immagini di suggerimenti e linee guida: - <code>storeImage</code> per caricare un'immagine nel filesystem. Utilizza Tika per verificare che il file ricevuto (<code>MultipartFile</code>) sia sicuro, in quanto viene confrontato il formato reale del file con quelli previsti per le immagini. - <code>deleteImage</code> per cancellare un'immagine dal filesystem	Creato
InteractionService.java	Refactoring	Modificato
JwtService.java	Modificato per rispecchiare il fatto che la chiave primaria dell'Admin è email e non username	Modificato
NotificationService.java	Refactoring	Modificato
OpponentService.java	Refactoring	Modificato
ScalataService.java	Refactoring	Modificato
SecurityService.java	Wrapper di <code>JwtRequestContext</code> per poter facilitare il testing, permettendone la configurazione attraverso <code>@MockBean</code> , in quanto <code>JwtRequestContext</code> presenta solo metodi statici e non è mockabile.	Creato
StudentService.java	Refactoring	Modificato
SuggestionService.java	Definisce la business logic relativi alle operazioni sui suggerimenti: - <code>uploadSuggestions</code> per caricare i suggerimenti associati ad una specifica classe nel database, effettuando una update se già presenti	Creato

	- uploadSuggestionImage per caricare un'immagine associata ad un suggerimento tramite ImageService. Qualora sia presente già un'immagine, questa viene prima cancellata dal file system. - findSuggestions restituisce una lista contenente tutti i suggerimenti associati ad una specifica classe presente nel database - deleteSuggestion per cancellare un suggerimento associato ad una specifica classe con un opportuno valore del campo "order". Qualora sia associata ad un'immagine, questa viene prima cancellata dal file system. - deleteSuggestionImage per cancellare l'immagine associata ad un suggerimento, tramite ImageService. Tutti i metodi utilizzano all'occorrenza SuggestionMapper per convertire gli oggetti in input DTO negli oggetti corrispondenti Model / gli oggetti in output Model negli oggetti corrispondenti DTO	
TeamService.java	Refactoring	Modificato
UploadOpponentService.java	Refactoring	Modificato

TeamModificationRequest.java è stata spostata nel package DTO data la sua natura.

4.2.12 Package controller

Il package controller mappa gli endpoint REST API agli opportuni metodi che gestiscono le funzionalità esportate del sistema, usufruendo dei servizi del Service. Per quanto riguarda le classi Controller già presenti, le modifiche riguardano i seguenti aspetti:

- Refactoring legato alla separazione delle responsabilità, in quanto alcuni metodi si aspettavano come valore di ritorno dal Service delle risposte HTTP di tipo ResponseEntity<?>. Ora le risposte HTTP sono create dal Controller
- Refactoring legato all'autenticazione con il token jwt, che è stata rimossa in vista della presenza del filtro AuthTokenFilter

File	Descrizione	Creato o Modificato
AdminController.java	Refactoring	Modificato
AssignmentController.java	Refactoring	Modificato
GuidelineController.java	Mappa gli endpoint relativi alle operazioni sulle linee guida: - uploadGuidelines POST per caricare le linee guida nel database - uploadGuidelineImage POST per caricare un'immagine associata ad una linea guida. - findGuidelines GET restituisce una lista contenente tutte le linee guida presenti nel database. - deleteGuideline DELETE per cancellare una linea guida con un opportuno valore del campo "order". - deleteGuidelineImage DELETE per cancellare l'immagine associata ad una linea guida.	Creato
HomeController.java	Refactoring	Modificato
InteractionController.java	Refactoring	Modificato
OpponentController.java	Refactoring	Modificato
ScalataController.java	Refactoring	Modificato
StudentController.java	Refactoring	Modificato
SuggestionController.java	Mappa gli endpoint relativi alle operazioni sui suggerimenti: - uploadSuggestions POST per caricare i suggerimenti associati ad una classe nel database - uploadSuggestionImage POST per caricare un'immagine associata ad un suggerimento. - findSuggestions GET restituisce una lista contenente tutti i suggerimenti presenti nel database. - deleteSuggestion DELETE per cancellare un suggerimento con un opportuno valore del campo "order".	Creato

	deleteSuggestionImage DELETE per cancellare l'immagine associata ad un suggerimento.	
TeamController.java	Refactoring	Modificato

4.2.13 Package controller.view

è stata aggiunta la sola classe GuidelineViewController per mappare la nuova vista relativa alle linee guida al metodo "showGuidelinesPage"

4.2.14 Package dto

Le classi DTO sono utilizzate per mappare i dati contenuti nei JSON forniti in ingresso al sistema nelle chiamate REST.

Questo package è stato estratto da com.groom.manvsc.class.model e spostato in com.groom.manvsc.class, in modo da rispecchiare il diagramma dei package.

I dto già esistenti erano limitati e quindi sono stati aggiunti i dto necessari a ricevere le informazioni nei vari controller. Sono state utilizzate molte annotazioni di jakarta.validation per abilitare la validazione dei campi dei JSON forniti in ingresso, quali:

- @NotNull
- @NotBlank
- @Positive
- @Size
- @Min
- @Max
- @ValidOrder: annotazione custom definita nel package com.groom.manvsc.class.validation

File	Descrizione	Creato o Modificato
AssignmentDTO.java	-	Creato
ClassUTDTO.java	-	Creato
ClassUTScalataDTO.java	-	Creato
ClassUTSuggestionDTO.java	-	Creato
GuidelineDTO.java	-	Creato
InteractionDTO.java	-	Creato
ScalataDTO.java	-	Creato
SuggestionDTO.java	-	Creato
TeamDTO.java	-	Creato

4.2.15 Package mapper

Le classi Mapper sono utilizzate dai Service per ottenere gli oggetti Model a partire dai DTO ricevuti in input / gli oggetti DTO da restituire in output a partire dai Model ricevuti dai Repository. Sono stati implementati tramite MapStruct per una generazione automatica dei metodi di conversione tra DTO e Model e viceversa. L'aggiunta dei Mapper consente ai Service di non avere la conoscenza sulla esatta dei DTO, ma solo dei Model, e analogamente ai Controller di non avere la conoscenza esatta sui Model, il che comporta una migliore separazione delle responsabilità.

File	Descrizione	Creato o Modificato
AssignmentMapper.java	Mapper tra DTO e Model relativi a Assignment	Creato
GuidelineMapper.java	Mapper tra DTO e Model relativi a Guideline	Creato
InteractionMapper.java	Mapper tra DTO e Model relativi a Interaction	Creato
ScalataMapper.java	Mapper tra DTO e Model relativi a Scalata	Creato
SuggestionMapper.java	Mapper tra DTO e Model relativi a Suggestion	Creato
TeamMapper.java	Mapper tra DTO e Model relativi a Team	Creato

4.2.16 Package exception

Questo package è stato estratto da com.groom.manvsc.class.service e spostato in com.groom.manvsc.class.

È stata definita la classe `GlobalExceptionHandler` per centralizzare la gestione delle eccezioni e alleggerire il lavoro ai Controller, i quali non devono più occuparsi dei casi limite di errore. La generazione delle risposte http in caso di errore è quindi delegata completamente a questa classe.

Sono inoltre state definite le seguenti eccezioni che estendono `RuntimeException`, lanciate dai Service:

File	Descrizione	Creato o Modificato
<code>DuplicatedEntryException.java</code>	-	Creato
<code>ForbiddenException.java</code>	-	Creato
<code>InvalidImageException.java</code>	-	Creato
<code>NotFoundException.java</code>	-	Creato
<code>UnauthorizedException.java</code>	-	Creato
<code>GlobalExceptionHandler.java</code>	Gestore globale delle eccezioni	Creato

5. Deployment diagram

È presentato il deployment diagram relativo **alle sole parti interessate** del sistema.

Nel microservizio T1 `AdminService` è stato aggiunto il sottosistema **`GuidelineAndSuggestionsSubsystem`** descritto nel component diagram in Figura 11, il quale si occupa di gestire le richieste relative ai suggerimenti e alle linee guida. Esso interagisce con il component **`ImageService`**, il quale accede al Volume T0 per caricare, scaricare o eliminare le immagini. Il sottosistema espone l'interfaccia **`GuidelineAndSuggestionsAPI`** richiesta da T56 `GameEngine` per accedere ai suggerimenti durante la partita. Tutto il container `AdminService` richiede l'interfaccia `AuthenticationAPI`, esposta da T23 `StudentRepository`, per verificare l'autenticazione dell'utente a valle della ricezione di una richiesta.

L'interfacciamento tra i vari container utilizza l'API Gateway come intermediario, che è stato omesso per mettere il focus sulle modifiche apportate al sistema e sulle dipendenze tra le interfacce dei vari container.

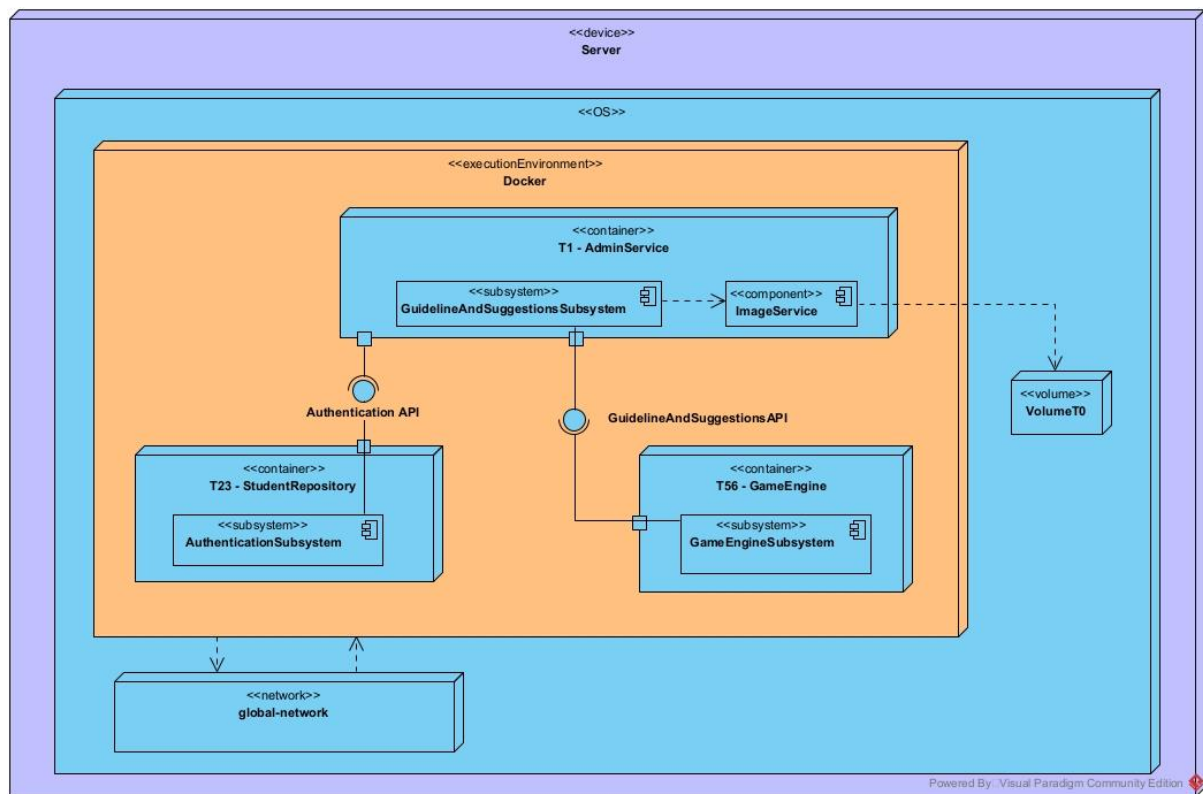


Figura 14 – Diagramma di deploy delle sole parti modificate

6. Descrizione dei Test effettuati

6.1 Unit testing

Sono stati effettuati i test di unità relativi moduli aggiunti e modificati. È questo il caso delle classi Controller, Service e Repository relative alle operazioni sui Suggerimenti e sulle Linee Guida (Guideline), insieme al nuovo modulo ImageService. Infine sono state testate tutte le classi Repository, in quanto hanno subito modifiche durante la migrazione a JpaRepository.

6.1.1 Controller

I test dei controller sono stati implementati utilizzando l'annotazione `@WebMvcTest`, un'annotazione di Spring che isola il layer MVC per eseguire test unitari dei controller, caricando solo il contesto spring web necessario e quindi escludendo i Service, i Repository e altri bean applicativi. Inoltre con `@AutoConfigureMockMvc(addFilters = false)` sono stati disabilitati i filtri di sicurezza per i test, in modo da consentire il testing della logica dei controller senza dover gestire l'autenticazione. In particolare, alcuni campi sono indicati con l'annotazione `@MockBean` di Spring, ovvero sono creati come oggetti simulati, in modo da permettere la configurazione del loro comportamento nei vari casi di test.

Inoltre per tutti i controller sono stati utilizzati:

- **MockMvc** per simulare richieste HTTP al controller
- **ObjectMapper** per serializzare e deserializzare i JSON per i test
- **ArgumentCaptor<T>** per verificare che i metodi dei service siano stati invocati con gli argomenti corretti
- Un attributo `@MockBean` del service corrispondente al controller sotto test (**SuggestionService** per i test di SuggestionController e **GuidelineService** per i test di GuidelineController), il cui comportamento è configurato in base alle precondizioni richieste dal caso di test (ad esempio per impostare i valori di ritorno della chiamata "findGuidelines")

SuggestionController

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/suggestions/upload" con Json Body : className : "className" "suggestionDTOs": [{ "order": 1, "hint": "Testo_Suggerimento_1" }, { "order": 2, "hint": "Testo_Suggerimento_2"}]	http status 200 e "Suggerimenti caricati con successo"	Suggerimenti caricati nel database correttamente	Pass	Verifica l'upload di due suggerimenti con formato valido

2	Il Sistema è predisposto ad accettare richieste di inserimento, ma non esiste className : "ClassInesistente"	Richiesta Post HTTPx "/opponents/suggestions/upload" con Json Body : className : "classInesistente" "suggestionDTOs": [{ "order": 1, "hint": "Testo_Suggerimento_1" }, { "order": 2, "hint": "Testo_Suggerimento_2"}]	NotFoundException, http status 404 con "Classe non trovata"	Suggerimenti non caricati nel database	Pass	Verifica l'upload di due suggerimenti per una classe inesistente
3	Il Sistema è predisposto ad accettare richieste di inserimento, ma con classe non valida	Richiesta Post HTTP "/opponents/suggestions/upload" con Json Body : className : null/ " /empty" "suggestionDTOs": [{ "order": 1, "hint": "Testo_Suggerimento_1" }, { "order": 2, "hint": "Testo_Suggerimento_2"}]	http BadRequest 400 e "Errore di validazione"	Suggerimenti non caricati nel database	Pass	Verifica l'upload di due suggerimenti per className non validi tramite test parametrizzato
4	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/suggestions/upload" con Json Body : className : "className" "suggestionDTOs": [{ "order": 2, "hint": "Testo_Suggerimento_2" }, { "order": 1, "hint": "Testo_Suggerimento_1"}]	http BadRequest 400 e "Errore di valutazione"	Suggerimenti non caricati nel database	Pass	Verifica l'upload di due suggerimenti con ordine progressivo erroneamente invertito
5	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/suggestions/upload" con Json Body : "suggestionDTOs": [{ "order": 2, "hint": "Testo_Suggerimento_2" }, { "order": 1, "hint": "Testo_Suggerimento_1"}]	http BadRequest 400 e "Errore di valutazione"	Suggerimenti non caricati nel database	Pass	Verifica l'upload di due suggerimenti con classe mancante
6	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/suggestions/upload" con Json Body : className : "className"	http BadRequest 400 e "Errore di valutazione"	Suggerimenti non caricati nel database	Pass	Verifica l'upload di una richiesta con campo suggerimenti mancante
7	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/suggestions/upload" con Json Body : "\"className\" = \"Calcolatrice\""	http BadRequest 400 e "Formato di richiesta non valido"	Suggerimenti non caricati nel database	Pass	Verifica l'upload di una richiesta con formato invalido del body
8	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/suggestions/upload"	http BadRequest 400 e "Formato di richiesta non valido"	Suggerimenti non caricati nel database	Pass	Verifica l'upload con Json body mancante
9	Il Sistema è predisposto ad accettare richieste di get	Richiesta GET http "/opponents/suggestions/Calcolatrice"	http 200 con className = "Calcolatrice" e "suggestionDTOs": [{ "order": 1, "hint": "...", "level":	Suggerimenti correttamente restituiti all'utente	Pass	Verifica la get di suggerimenti appartenenti ad una classe valida

			"...", "image": "..."}, { "order": 2, "hint": "...", "level": "...", "image": "..."}]			
10	Il Sistema è predisposto ad accettare richieste di get	Richiesta GET http "/opponents/suggestions/ClassInesistente"	NotFoundException http status notFound 404 e "ClassInesistente"	Suggerimenti non resituiti all'utente	Pass	Verifica get di suggerimenti appartenenti ad una classe inesistente
11	Il sistema è predisposto ad accettare richieste di get	Richiesta get "/opponents/suggestions/empty(" ")	http 400 badRequest	Suggerimenti non restituiti all'utente	Pass	Verifica get di suggerimenti con className non valido
12	Il Sistema è predisposto ad accettare richieste di delete ed è presente il suggerimento con ordine 1 nella classe Calcolatrice	http delete "/opponents/suggestions/Calcolatrice/1"	http 200 e "Suggerimento eliminato con successo"	Suggerimento selezionato eliminato con successo	Pass	Verifica delete suggerimento esistente della classe calcolatrice
13	Il Sistema è predisposto ad accettare richieste di delete e non è presente la classe "Calcolatrice"	http delete "/opponents/suggestions/Calcolatrice/1"	NotFoundException http not found 404 e "Suggerimento non trovato"	Suggerimento selezionato non eliminato	Pass	Verifica delete di un suggerimento per una classe inesistente
14	Il Sistema è predisposto ad accettare richieste di delete	http delete "/opponents/suggestions/ /1"	http 400 badRequest e "Errore di validazione"	Suggerimento non eliminato	Pass	Verifica delete di un suggerimento con className non valido
15	Il Sistema è predisposto ad accettare richieste di delete	http delete "/opponents/suggestions/Calcolatrice/-1"	http 400 badRequest e "Errore di validazione"	Suggerimento non eliminato	Pass	Verifica delete di un suggerimento con order non valido
16	Il sistema è predisposto ad accettare richieste di updateImage e esiste classe Calcolatrice e suggerimento con ordine 1	http post /opponents/suggestions/upload/Calcolatrice/1 e multipartFile image	http 200 e "Immagine caricata con successo"	Immagine caricata correttamente	Pass	Verifica upload di un immagine per un suggerimento esistente

17	Il sistema è predisposto ad accettare richieste di updateImage e esiste classe Calcolatrice inesistente	http post /opponents/suggestions/upload/Calcolatrice/1 e multipartFile image	http not found 404	Immagine non caricata	Pass	Verifica upload di un'immagine per un suggerimento di una classe inesistente
18	Il sistema è predisposto ad accettare richieste di updateImage	http get "/opponents/suggestions/upload/ /1"	http 400 badRequest e "Errore di validazione"	Immagine non caricata	Pass	Verifica update di un'immagine con formato classe non valido
19	Il sistema è predisposto ad accettare richieste di updateImage	http post /opponents/suggestions/upload/Calcolatrice/-1 e multipartFile image	http 400 badRequest e "Errore di validazione"	Immagine non caricata	Pass	Verifica update di un'immagine con ordine non valido
20	Il sistema è predisposto ad accettare richieste di deleteImage e esiste classe "className"	http delete "/opponents/suggestions/image/className/1"	http 200 e "Immagine del suggerimento eliminata con successo."	Immagine eliminata con successo	Pass	Verifica delete immagine esistente
21	Il sistema è predisposto ad accettare richieste di deleteImage e non esiste la classe "ClassInesistente"	"/opponents/suggestions/image/ClassInesistente /1"	http 404 notFound e "ClassInesistente"	Immagine non cancellata	Pass	Verifica deleteImage di classe non esistente
22	Il sistema è predisposto ad accettare richieste di deleteImage e non esiste la classe " "	"/opponents/suggestions/image/ /1"	http 400 badRequest e "Errore di validazione"	Immagine non cancellata	Pass	Verifica deleteImage di classe con formato invalido
23	Il sistema è predisposto ad accettare richieste di deleteImage e esiste classe "Calcolatrice"	http delete "/opponents/suggestions/image/Calcolatrice/-1"	http 400 badRequest e "Errore di validazione"	Immagine non cancellata	Pass	Verifica deleteImage con ordine progressivo sbagliato

GuidelineController

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione

1	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/guidelines/upload" con Json Body : "guidelineDTOs": [{ "order": 1, "hint": "Testo_Linea_Guida_1"}, { "order": 2, "hint": "Testo_Linea_Guida_2"}]	http status 200 e "Linee guida caricate con successo"	Guidelines caricate nel database correttamente	Pass	Verifica l'upload di due guidelines con formato valido
2	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/guidelines/upload" con Json Body : "guidelineDTOs": [{ "order": 2, "hint": "Testo_Linea_guida_2"}, { "order": 1, "hint": "Testo_Linea_Guida_1"}]	http 400 badRequest e "Formato richiesta non valido"	Guideline non caricate nel database	Pass	Verifica l'upload di due guidelines con ordine non corretto
3	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/guidelines/upload" con Json body : "GuidelinesDtos":	http 400 badRequest e "Formato richiesta non valido"	Guideline non caricate nel database	Pass	Verifica l'upload di due guidelines con corpo Json non valido
4	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/guidelines/upload" Con Json body "guidelineDTOs": "\order\" = \"1\""	http 400 badRequest e "Formato richiesta non valido"	Guideline non caricate nel database	Pass	Verifica l'upload di due guidelines con guideline list mancante
5	Il Sistema è predisposto ad accettare richieste di inserimento	Richiesta Post HTTP "/opponents/guidelines/upload"	http 400 badRequest e "Formato richiesta non valido"	Guideline non caricate nel database	Pass	Verifica l'upload di due guidelines con corpo Json mancante
6	Il Sistema è predisposto ad accettare richieste di get, ed esistono le linee guida 1 e 2	Richiesta Get http "/opponents/guidelines"	http 200 e body contenente guidelines	Guidelines restituite all'utente	Pass	Verifica get di guidelines
7	Il sistema è predisposto alla cancellazione di linee guida ed è presente la linea guida 1	Richiesta Delete http "/opponents/guidelines/1"	http 200 e "Linea guida eliminata con successo"	Guideline cancellata	Pass	Verifica delete guideline esistente
8	Il sistema è predisposto alla cancellazione di linee guida	Richiesta Delete http "/opponents/guidelines/-1"	http 400 badRequest e "Errore di validazione"	Guideline non cancellata	Pass	Verifica delete di linea guida con ordine non corretto
10	Il sistema è predisposto ad aggiungere un immagine ad una guideline esistente	Richiesta Post "/opponents/guidelines/upload/1" Con Multipart image	http 200 e "Immagine caricata con successo"	Immagine per una guideline inserita	Pass	Verifica della post di un'immagine per una guideline

11	Il sistema è predisposto ad aggiungere un immagine	Richiesta Post "/opponents/guidelines/upload/-1" Con Multipart image	http 400 badRequest e "Errore di validazione"	Immagine non inserita	Pass	Verifica della post di un'immagine per una guideline con ordine errato
12	Il sistema è predisposto per la delete di un'immagine per un suggerimento esistente	Richiesta Delete http "/opponents/guidelines/image/1"	http 200 e "Immagine della linea guida eliminata con successo"	Immagine eliminata	Pass	Verifica delete di un'immagine per un suggerimento esistente
13	Il sistema è predisposto per la delete di un'immagine per un suggerimento	Richiesta Delete http "/opponents/guidelines/image/-1"	http 400 badRequest e "Errore di validazione"	Immagine non cancellata	Pass	Verifica delete di un'immagine per un suggerimento con ordine sbagliato

6.1.2 Service

Le classi Service sono state testate con Mockito, che ha permesso di configurare il comportamento mockato dei repository in ogni caso di test, in modo da simulare le precondizioni del database (ad esempio per simulare la presenza di un Suggerimento nel database).

SuggestionService

UploadSuggestions

Piano di test del metodo:

suggestionService.uploadSuggestions(String className, List<SuggestionDTO> suggestionDTOs).

I D	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste la ClassUT "Calcolatrice" nel database.	className = "Calcolatrice" suggestionDTOs = [SuggestionDTO(order=1, hint="Testo_Suggerimento", level=LOW, image=null), SuggestionDTO(order=2, hint="Testo_Suggerimento", level=LOW, image=null)]	#suggestionMapper.toEntityList calls = 1 con suggestionDTOs; #classUTRepository.findById calls = 1 con className; #suggestionRepository.findAllByClassUT_Name calls = 1 con className; #suggestionRepository.saveAll calls = 1 con la lista corretta.	Suggerimenti salvati nel database.	Pass	Upload di suggerimenti con dati in input nel formato valido e classe esistente nel database.
2	Non esiste la ClassUT "Classe_Non_Esistente" nel database.	className = "Classe_Non_Esistente" suggestionDTOs = [SuggestionDTO(order=1,	Il metodo lancia <i>NotFoundException</i> ; #suggestionMapper.toEntityList calls = 1 con suggestionDTOs; #classUTRepository.findById calls = 1 con className;	Suggerimento non caricato nel database, stato di eccezione.	Pass	Upload di suggerimenti con dati in input nel formato valido e classe

		hint="Testo_Suggerimento", level=LOW, image=null]	#suggestionRepository.findAllByClass UT_Name calls = 0; #suggestionRepository.saveAll calls = 0;			inesistente nel database.
3	Esiste la ClassUT "Calcolatrice" nel database.	className = "Calcolatrice" suggestionDTOs = []	#suggestionMapper.toEntityList calls = 0; #classUTRepository.findById calls = 0 con className; #suggestionRepository.findAllByClass UT_Name calls = 0; #suggestionRepository.saveAll calls = 0;	Database invariato.	Pass	Upload di una lista vuota di suggerimenti e classe esistente nel database.
4	Esiste la ClassUT "Calcolatrice" nel database	className = "Calcolatrice" suggestionDTOs = [SuggestionDTO(order=1, hint="Testo_Suggerimento", level=LOW, image=null), SuggestionDTO(order=-1, hint="Testo_Suggerimento", level=LOW, image=null)]	Il metodo lancia <i>DataIntegrityViolationException</i> ; #suggestionMapper.toEntityList calls = 1 con suggestionDTOs; #classUTRepository.findById calls = 1 con className; #suggestionRepository.findAllByClass UT_Name calls = 1 con className; #suggestionRepository.saveAll calls = 0;	Nessun suggerimento caricato nel database, stato di eccezione.	Pass	Upload di un suggerimento o con input validi e un altro no. Verifica che venga lanciata un'eccezione unchecked, in modo da scatenare il rollback della transazione (vista l'annotazione @Transactional del service) e dunque non salvare nessuno dei due suggerimenti.
5	Esiste la ClassUT "Calcolatrice" nel database	className = "Calcolatrice" suggestionDTOs = [SuggestionDTO(order=1, hint="Testo_Suggerimento", level=LOW, image=null), SuggestionDTO(order=1, hint="Testo_Suggerimento", level=LOW, image=null)]	Il metodo lancia <i>DataIntegrityViolationException</i> ; #suggestionMapper.toEntityList calls = 1 con suggestionDTOs; #classUTRepository.findById calls = 1 con className; #suggestionRepository.findAllByClass UT_Name calls = 1 con className; #suggestionRepository.saveAll calls = 1 con la lista corretta;	Nessun suggerimento caricato nel database, stato di eccezione.	Pass	Upload di due suggerimenti duplicati. Verifica che venga lanciata un'eccezione unchecked, in modo da scatenare il rollback della transazione (vista l'annotazione @Transactional del

						service) e dunque non salvare nessuno dei due suggerimenti.
--	--	--	--	--	--	---

FindSuggestions

Piano di test del metodo:

suggestionService.findSuggestions(String className).

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste la ClassUT "Calcolatrice" nel database e ci sono suggerimenti associati.	className = "Calcolatrice"	#classUTRepository.existsById calls = 1 con className; #suggestionRepository.findAllByClassUT_Name calls = 1 con className; #suggestionMapper.toDtoList calls = 1 con la lista dei model Suggestion atteso dal repository;	Suggerimenti restituiti correttamente.	Pass	Ricerca suggerimenti di una classe esistente nel database e suggerimenti associati presenti.
2	Esiste la ClassUT "Calcolatrice" nel database e non ci sono suggerimenti associati.	className = "Calcolatrice"	#classUTRepository.existsById calls = 1 con className; #suggestionRepository.findAllByClassUT_Name calls = 1 con className; #suggestionMapper.toDtoList calls = 1 con lista vuota;	Lista vuota restituita correttamente.	Pass	Ricerca suggerimenti di una classe esistente nel database ma con nessun suggerimento associato.
3	Non esiste la ClassUT "Classe_Non_Esistente" nel database.	className = "Classe_Non_Esistente"	Il metodo lancia <i>NotFoundException</i> ; #classUTRepository.existsById calls = 1 con className; #suggestionRepository.findAllByClassUT_Name calls = 1 con className; #suggestionMapper.toDtoList calls = 0;	Stato di eccezione.	Pass	Ricerca suggerimenti di una classe non esistente nel database.

DeleteSuggestion

Piano di test del metodo:

suggestionService.deleteSuggestion(String className, int suggestionOrder).

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1.	className = "Calcolatrice" suggestionOrder = 1	#suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e suggestionOrder; #suggestionRepository.delete calls = 1 con il model Suggestion atteso dal repository;	Suggerimento cancellato correttamente.	Pass	Cancellazione di un suggerimento esistente associato ad una classe esistente nel database.
2	Non esiste la ClassUT "Classe_Non_Esistente" nel database.	className = "Classe_Non_Esistente" suggestionOrder = 1	Il metodo lancia <i>NotFoundException</i> ; #suggestionRepository.findAllByClassUT_Name calls = 1 con className e suggestionOrder; #suggestionRepository.delete calls = 0;	Nessun suggerimento cancellato, stato di eccezione.	Pass	Cancellazione di un suggerimento indicando una classe non esistente nel database.
3	Esiste la ClassUT "Calcolatrice" nel database ma non c'è nessun suggerimento associato con order 2.	className = "Calcolatrice" suggestionOrder = 2	Il metodo lancia <i>NotFoundException</i> ; #suggestionRepository.findAllByClassUT_Name calls = 1 con className e suggestionOrder; #suggestionRepository.delete calls = 0;	Nessun suggerimento cancellato, stato di eccezione.	Pass	Cancellazione di un suggerimento indicando una classe esistente e un order di un suggerimento non presente.

UploadSuggestionImage

Piano di test del metodo:

suggestionService.uploadSuggestionImage(String className, int order, MultipartFile image).

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1, il quale non presenta nessun immagine.	className = "Calcolatrice" order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	#suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 0; #imageService.storeImage calls = 1 con multipartFile e "Calcolatrice_1.png" #suggestionRepository.save calls = 1 con il model Suggestion atteso (quello iniziale ma con image pari a "Calcolatrice_1.png");	Immagine aggiunta correttamente con imageService e campo image del suggerimento aggiornato nel database con il nome costruito "Calcolatrice_1.png".	Pass	Caricamento di un'immagine ad un suggerimento presente nel database che inizialmente non presenta immagine.
2	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1, il quale presenta già un'altra immagine con nome "Calcolatrice_1.jpg"	className = "Calcolatrice" order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	#suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 1 con "Calcolatrice_1.png"; #imageService.storeImage calls = 1 con multipartFile e "Calcolatrice_1.png" #suggestionRepository.save calls = 1 con il model Suggestion atteso (quello iniziale ma con image pari a "Calcolatrice_1.png");	Immagine aggiunta correttamente con imageService e campo image del suggerimento aggiornato nel database con il nome costruito "Calcolatrice_1.png".	Pass	Caricamento di un'immagine ad un suggerimento presente nel database che inizialmente presenta già un'altra immagine.
3	Esiste la ClassUT "Calcolatrice" nel database che non presenta alcun suggerimento con order 99.	className = "Calcolatrice" order = 99 multipartFile mockato, configurato con il nome fileName = "immagine.png"	Il metodo lancia <i>NotFoundException</i> . #suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 0; #imageService.storeImage calls = 0; #suggestionRepository.save calls = 0;	Nessuna immagine aggiunta al database, stato di eccezione.	Pass	Caricamento di un'immagine ad un suggerimento assente nel database.
4	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1, il quale presenta già un'altra immagine con nome "Calcolatrice_1.png". ImageService presenta un'errore nella cancellazione dell'immagine precedente.	className = "Calcolatrice" order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	Il metodo lancia <i>RuntimeException</i> . #suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 1 con "Calcolatrice_1.png"; #imageService.storeImage calls = 0; #suggestionRepository.save calls = 0;	Nessuna immagine aggiunta al database, stato di eccezione.	Pass	Caricamento di un'immagine ad un suggerimento presente nel database che inizialmente presenta già un'altra immagine, ma con un errore di ImageService nella cancellazione dell'immagine precedente.
5	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1, il quale non presenta un'immagine. ImageService presenta un'errore nel salvataggio dell'immagine.	className = "Calcolatrice" order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	Il metodo lancia <i>RuntimeException</i> . #suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 0; #imageService.storeImage calls = 0; #suggestionRepository.save calls = 0;	Nessuna immagine aggiunta al database, stato di eccezione.	Pass	Caricamento di un'immagine ad un suggerimento presente nel database che inizialmente non presenta un'immagine, ma con un errore di ImageService nel salvataggio dell'immagine.

DeleteSuggestionImage

Piano di test del metodo:

suggestionService.deleteSuggestionImage(String className, int order).

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1, il quale presenta un'immagine con nome "Calcolatrice_1.png".	className = "Calcolatrice" order = 1	#suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 1 con "Calcolatrice_1.png" #suggestionRepository.save calls = 1 con il model Suggestion atteso (quello iniziale ma con image pari a null);	Immagine cancellata correttamente con imageService e campo image del suggerimento aggiornato nel database con il valore null.	Pass	Cancellazione dell'immagine di un suggerimento presente nel database che inizialmente presenta immagine.

2	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1, il quale inizialmente non presenta alcuna immagine.	className = "Calcolatrice" order = 1	#suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 0; #suggestionRepository.save calls = 0;	Stato del sistema invariato.	Pass	Cancellazione dell'immagine di un suggerimento presente nel database che inizialmente non presenta immagine.
3	Esiste la ClassUT "Calcolatrice" nel database ma non esiste alcun suggerimento associato con order 99.	className = "Calcolatrice" order = 99	Il metodo lancia <i>NotFoundException</i> . #suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 0; #suggestionRepository.save calls = 0;	Nessuna immagine cancellata, stato di eccezione.	Pass	Cancellazione dell'immagine di un suggerimento assente nel database.
4	Esiste la ClassUT "Calcolatrice" nel database e c'è un suggerimento associato con order 1, il quale presenta un'immagine con nome "Calcolatrice_1.png". ImageService presenta un problema nella cancellazione dell'immagine.	className = "Calcolatrice" order = 1	Il metodo lancia <i>RuntimeException</i> . #suggestionRepository.findAllClassUT_NameAndOrder calls = 1 con className e order; #imageService.deleteImage calls = 0; #suggestionRepository.save calls = 0;	Nessuna immagine cancellata, stato di eccezione.	Pass	Cancellazione dell'immagine di un suggerimento presente nel database che inizialmente presenta un'altra immagine, ma si presenta un errore nella cancellazione dell'immagine.

GuidelineService

UploadGuidelines

Piano di test del metodo:

guidelineService.uploadGuidelines(List<GuidelineDTO> guidelineDTOs).

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Sistema attivo.	guidelineDTOs = [guidelineDTO(order=1, hint="Testo_Linea Guida", image=null), guidelineDTO(order=2, hint="Testo_Linea Guida", image=null)]	#guidelineMapper.toEntityList calls = 1 con guidelineDTOs; #guidelineRepository.findAllGuidelines calls = 1; #guidelineRepository.saveAll calls = 1 con la lista corretta.	Linee Guida salvate nel database.	Pass	Upload di linee guida con dati in input nel formato valido.
2	Sistema attivo.	guidelineDTOs = []	#guidelineMapper.toEntityList calls = 0; #guidelineRepository.findAllGuidelines calls = 0; #guidelineRepository.saveAll calls = 0;	Database invariato.	Pass	Upload di una lista vuota di linee guida nel database.
3	Sistema attivo.	guidelineDTOs = [guidelineDTO(order=1, hint="Testo_Linea Guida", level=LOW, image=null), guidelineDTO(order=-1, hint="Testo_Linea Guida", level=LOW, image=null)]	Il metodo lancia <i>DataIntegrityViolationException</i> ; #guidelineMapper.toEntityList calls = 1 con guidelineDTOs; #guidelineRepository.findAllGuidelines calls = 1; #guidelineRepository.saveAll calls = 0;	Nessuna Linea Guida caricata nel database, stato di eccezione.	Pass	Upload di una Linea Guida con input validi e un'altra no. Verifica che venga lanciata un'eccezione unchecked, in modo da scatenare il rollback della transazione (vista l'annotazione @Transactional del service) e dunque non salvare nessuna delle due linee guida.
4	Sistema attivo.	guidelineDTOs = [guidelineDTO(order=1, hint="Testo_Linea Guida", level=LOW, image=null), guidelineDTO(order=1, hint="Testo_Linea Guida", level=LOW, image=null)]	Il metodo lancia <i>DataIntegrityViolationException</i> ; #guidelineMapper.toEntityList calls = 1 con guidelineDTOs; #guidelineRepository.findAllGuidelines calls = 1; #guidelineRepository.saveAll calls = 0;	Nessuna Linea Guida caricata nel database, stato di eccezione.	Pass	Upload di due linee guida duplicate. Verifica che venga lanciata un'eccezione unchecked, in modo da scatenare il rollback della transazione (vista l'annotazione @Transactional del service) e dunque non salvare nessuno delle due linee guida.

FindGuidelines

Piano di test del metodo *guidelineService.findGuidelines()*.

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Sistema attivo con linee guida presenti nel database.	-	#guidelineRepository.findAllGuidelines calls = 1; #guidelineMapper.toDtoList calls = 1 con la lista dei model Guideline atteso dal repository;	Linee guida restituite correttamente.	Pass	Ricerca linee guida presenti nel database.
2	Sistema attivo con linee guida assenti nel database.	-	#guidelineRepository.findAllGuidelines calls = 1; #guidelineMapper.toDtoList calls = 1 con la lista vuota.	Lista vuota restituita correttamente.	Pass	Ricerca linee guida assenti nel database.

DeleteGuideline

Piano di test del metodo *guidelineService.deleteGuideline(int order)*.

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste la linea guida con order 1.	order = 1	#guidelineRepository.findByOrder calls = 1 con order=1; #guidelineRepository.delete calls = 1 con il model Guideline atteso dal repository;	Linea Guida cancellata correttamente.	Pass	Cancellazione di una Linea Guida esistente nel database.
2	Non esiste la linea guida con order 2.	order = 2	Il metodo lancia <i>NotFoundException</i> ; #guidelineRepository.findByOrder calls = 1 con order=1; #guidelineRepository.delete calls = 0;	Nessuna Linea Guida cancellata, stato di eccezione.	Pass	Cancellazione di una Linea Guida indicando order inesistente.

UploadGuidelineImage

Piano di test del metodo *guidelineService.uploadGuidelineImage(int order, MultipartFile image)*.

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste una Linea Guida con order 1, la quale non presenta nessun immagine.	order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	#guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 0; #imageService.storeImage calls = 1 con multipartFile e "1.png" #guidelineRepository.save calls = 1 con il model Guideline atteso (quello iniziale ma con image pari a "1.png");	Immagine aggiunta correttamente con imageService e campo image della Linea Guida aggiornato nel database con il nome costruito "1.png".	Pass	Caricamento di un'immagine ad una Linea Guida presente nel database che inizialmente non presenta immagine.
2	Esiste una Linea Guida con order 1, la quale presenta già un'altra immagine con nome "1.jpg"	order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	#guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 1 con "1.png"; #imageService.storeImage calls = 1 con multipartFile e "1.png" #guidelineRepository.save calls = 1 con il model Guideline atteso (quello iniziale ma con image pari a "1.png");	Immagine aggiunta correttamente con imageService e campo image della Linea Guida aggiornato nel database con il nome costruito "Calcolatrice_1.png".	Pass	Caricamento di un'immagine ad una Linea Guida presente nel database che inizialmente presenta già un'altra immagine.
3	Non esiste alcuna Linea Guida con order 99.	order = 99 multipartFile mockato, configurato con il nome fileName = "immagine.png"	Il metodo lancia <i>NotFoundException</i> . #guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 0; #imageService.storeImage calls = 0; #guidelineRepository.save calls = 0;	Nessuna immagine aggiunta al database, stato di eccezione.	Pass	Caricamento di un'immagine ad una Linea Guida assente nel database.
4	Esiste una Linea Guida con order 1, la quale presenta già un'altra immagine con nome "1.png" ImageService presenta	order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	Il metodo lancia <i>RuntimeException</i> . #guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 1 con "1.png"; #imageService.storeImage calls = 0; #guidelineRepository.save calls = 0;	Nessuna immagine aggiunta al database, stato di eccezione.	Pass	Caricamento di un'immagine ad una Linea Guida presente nel database che inizialmente presenta già un'altra immagine, ma con un errore di ImageService nella cancellazione

	un'errore nella cancellazione dell'immagine precedente.					dell'immagine precedente.
5	Esiste una Linea Guida con order 1, la quale non presenta un'immagine. ImageService presenta un'errore nel salvataggio dell'immagine.	order = 1 multipartFile mockato, configurato con il nome fileName = "immagine.png"	Il metodo lancia <i>RuntimeException</i> . #guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 0; #imageService.storeImage calls = 0; #guidelineRepository.save calls = 0;	Nessuna immagine aggiunta al database, stato di eccezione.	Pass	Caricamento di un'immagine ad una Linea Guida presente nel database che inizialmente non presenta un'immagine, ma con un errore di ImageService nel salvataggio dell'immagine.

DeleteGuidelineImage

Piano di test del metodo *guidelineService.deleteGuidelineImage(int order)*.

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Esiste una Linea Guida con order 1, la quale presenta un'immagine con nome "1.png".	order = 1	#guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 1 con "Calcolatrice_1.png" #guidelineRepository.save calls = 1 con il model Guideline atteso (quello iniziale ma con image pari a null);	Immagine cancellata correttamente con imageService e campo image della Linea Guida aggiornato nel database con il valore null.	Pass	Cancellazione dell'immagine di una Linea Guida presente nel database che inizialmente presenta immagine.
2	Esiste una Linea Guida con order 1, la quale inizialmente non presenta alcuna immagine.	order = 1	#guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 0; #guidelineRepository.save calls = 0;	Stato del sistema invariato.	Pass	Cancellazione dell'immagine di una Linea Guida presente nel database che inizialmente non presenta immagine.
3	Non esiste alcuna Linea Guida con order 99.	order = 99	Il metodo lancia <i>NotFoundException</i> . #guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 0; #guidelineRepository.save calls = 0;	Nessuna immagine cancellata, stato di eccezione.	Pass	Cancellazione dell'immagine di una Linea Guida assente nel database.
4	Esiste una Linea Guida con order 1, la quale presenta un'immagine con nome "1.png". ImageService presenta un problema nella cancellazione dell'immagine.	order = 1	Il metodo lancia <i>RuntimeException</i> . #guidelineRepository.findByOrder calls = 1 con order; #imageService.deleteImage calls = 0; #suggestionRepository.save calls = 0;	Nessuna immagine cancellata, stato di eccezione.	Pass	Cancellazione dell'immagine di una Linea Guida presente nel database che inizialmente presenta un'altra immagine, ma si presenta un errore nella cancellazione dell'immagine.

6.1.3 ImageService

StoreImage

Piano di test del metodo:

imageService.storeImage(MultipartFile image, String imageName).

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Directory di storage esiste e è vuota.	image mockato, configurato con un contenuto pari ad un immagine png valida (array di byte dove i primi 8 byte corrispondono alla PNG signature, ovvero ai seguenti byte in esadecimale: 0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A) imageName = "Calcolatrice_1.png"	Il file è salvato nel filesystem Il contenuto del file corrisponde al contenuto di input	Il file "Calcolatrice_1.png" esiste nella directory di storage con il contenuto corretto	Pass	Salvataggio di un'immagine con dati validi.

2	Directory di storage esiste e è vuota, ma la copia presenta un errore.	image mockato, configurato con un errore IOException. imageName = "Calcolatrice_1.png"	Il metodo lancia <i>RuntimeException</i> . Nessun file viene salvato	Directory di storage non contiene il file Lo stato del filesystem non è cambiato	Pass	Salvataggio di un'immagine con errore durante la copia.
3	Directory di storage contiene il file "Calcolatrice_1.png" con contenuto "vecchio"	image mockato, configurato con un contenuto pari ad un array di byte valido "nuovo contenuto". imageName = "Calcolatrice_1.png"	Il file è sovrascritto nel filesystem Il contenuto del file corrisponde al contenuto di input	Il file "Calcolatrice_1.png" contiene il nuovo contenuto	Pass	Salvataggio di un'immagine con lo stesso nome di una esistente

DeleteImage

Piano di test del metodo:

imageService.deleteImage(String imageName).

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Directory di storage contiene il file "Calcolatrice_1.png"	imageName = "Calcolatrice_1.png"	Il file è cancellato dal filesystem	Il file "Calcolatrice_1.png" non esiste più nella directory di storage	Pass	Cancellazione di un'immagine esistente
2	Directory di storage esiste e il file "Calcolatrice_123.png" non esiste	imageName = "Calcolatrice_123.png"	Nessun file viene cancellato.	Lo stato del filesystem non è cambiato	Pass	Tentativo di cancellare un'immagine che non esiste (deleteIfExists)

6.1.4 Repository

Le classi Repository sono state testate con l'annotazione `@DataJpaTest`, utilizzando il database H2 come database in-memory per evitare di inizializzare il database reale.

SuggestionRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo con ClassUT "Calcolatrice".	Oggetto Suggestion valido associato alla ClassUT.	Suggestion salvata correttamente e recuperabile tramite ID.	Suggestion persistita nel database.	Pass	Verifica il salvataggio di un Suggestion e il corretto recupero dal database.
2	Database attivo.	Lista di 2 Suggestion valide associate alla stessa ClassUT.	Entrambe le Suggestion salvate e recuperabili.	Tutte le Suggestion persistite correttamente.	Pass	Verifica il salvataggio multiplo di Suggestion.
3	Suggestion esistente nel DB.	Modifica oggetto: order=2.	Suggestion aggiornata correttamente con il nuovo order.	Record aggiornato nel database.	Pass	Verifica l'aggiornamento di una Suggestion esistente.
4	Suggestion esistente nel DB.	Long ID della Suggestion da eliminare.	Ricerca per ID restituisce Optional.empty().	Record Suggestion eliminato dal database.	Pass	Verifica la cancellazione di una Suggestion.
5	DB con: 2 Suggestion per "Calcolatrice", 1 per "AltraClasse".	Input query: findAllByClassUT_Name("Calcolatrice")	Lista contenente esattamente le 2 Suggestion della classe corretta.	Nessuna modifica al database.	Pass	Verifica il recupero delle Suggestion per nome della ClassUT.
6	Nessuna Suggestion associata a "Classe_Inesistente".	Input query: findAllByClassUT_Name("Classe_Inesistente").	Lista vuota.	Database invariato.	Pass	Verifica il comportamento della ricerca quando la ClassUT non esiste.
7	Suggestion salvata (Class="Calcolatrice", Order=1).	Input query: name="Calcolatrice", order=1.	Optional contenente la Suggestion trovata.	Nessuna modifica al database.	Pass	Verifica il recupero di una Suggestion per nome ClassUT e order.
8	Suggestion salvata (Order=1).	Input query: name="Calcolatrice", order=2.	Optional vuoto (Nessuna Suggestion trovata).	Database invariato.	Pass	Verifica il comportamento quando l'order non esiste.
9	Suggestion salvata (Class="Calcolatrice").	Input query: name="Classe_Non_Esistente", order=1.	Optional vuoto (Nessuna Suggestion trovata).	Database invariato.	Pass	Verifica il comportamento della ricerca con ClassUT errata.

10	Database attivo.	Suggestion con order=-1.	Eccezione ConstraintViolationException.	Suggestion non salvata.	Pass	Verifica la validazione del campo order (non negativo).
11	Database attivo.	Suggestion con hint nullo, vuoto o solo spazi.	Eccezione ConstraintViolationException.	Suggestion non salvata.	Pass	Verifica la validazione del campo hint (Not Blank).
12	Database attivo.	Suggestion con hint di 300 caratteri.	Eccezione DataIntegrityViolationException.	Suggestion non salvata.	Pass	Verifica il vincolo di lunghezza massima del campo hint.
13	Database attivo.	Suggestion con valori validi di SuggestionLevel(enum).	Suggestion salvata e recuperata con il livello corretto.	Suggestion persistita correttamente.	Pass	Verifica il salvataggio con tutti i valori validi di level.
14	Database attivo.	Suggestion con level=null.	Eccezione ConstraintViolationException.	Suggestion non salvata.	Pass	Verifica la validazione del campo level (Not Null).
15	Database attivo.	Suggestion con date=null.	Eccezione DataIntegrityViolationException.	Suggestion non salvata.	Pass	Verifica il vincolo di non null sul campo date.
16	Database attivo.	Suggestion con image=null.	Suggestion salvata correttamente.	Suggestion persistita con image nulla.	Pass	Verifica il salvataggio di una Suggestion senza immagine.
17	Database attivo.	Suggestion con classUT=null.	Eccezione ConstraintViolationException.	Suggestion non salvata.	Pass	Verifica il vincolo di foreign key su ClassUT (Not Null).
18	Suggestion salvata associata a una ClassUT.	Azione: Eliminazione della ClassUT padre.	Ricerca della Suggestion per ID restituisce vuoto.	Cascade delete applicata correttamente (Suggestion rimossa).	Pass	Verifica il comportamento di cancellazione e a cascata.

GuidelineRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo	title="Base", order=1, hint="Suggerimento", date=2024-12-13, image="image.png"	Guideline salvata correttamente e recuperabile tramite ID	Guideline persistita nel database	Pass	Verifica il salvataggio di una Guideline
2	Database attivo	[{title="G1", order=1, hint="Hint1", date=2024-12-13}, {title="G2", order=2,	Tutte le Guideline salvate correttamente	Entrambe persistite nel database	Pass	Verifica il salvataggio di più Guideline

		hint="Hint2", date=2024-12-13}]				
3	Guideline salvata	order=2	Guideline aggiornata correttamente	Database aggiornato	Pass	Verifica l'aggiornamento dell'order
4	Guideline salvata	ID della Guideline	Guideline non più presente	Record Guideline eliminato	Pass	Verifica la cancellazione di una Guideline
5	Database attivo	---	Lista con tutte le guideline	Database invariato	Pass	Verifica il recupero di tutte le Guideline
6	Nessuna Guideline presente	---	Lista vuota	Database invariato	Pass	Verifica la ricerca di Guideline inesistente
7	Guideline salvata	order=1	Guideline trovata correttamente	Database invariato	Pass	Verifica la ricerca tramite order
8	Guideline salvata	order=999	Nessun risultato	Database invariato	Pass	Verifica la ricerca tramite order inesistente
9	Database attivo	order=-1	Eccezione ConstraintViolationException	Nessuna Guideline salvata	Pass	Verifica vincolo di validità sull'order
10	Database attivo	hint=null / "" / " "	Eccezione ConstraintViolationException	Nessuna Guideline salvata	Pass	Verifica vincolo di validità sull'hint
11	Database attivo	hint="aaaa...aaaa" (300 caratteri)	Eccezione DataIntegrityViolationException	Nessuna Guideline salvata	Pass	Verifica vincolo di lunghezza sull'hint
12	Database attivo	date=null	Eccezione DataIntegrityViolationException	Nessuna Guideline salvata	Pass	Verifica vincolo NOT NULL sulla data
13	Database attivo	image=null	Guideline salvata correttamente	Guideline persistita con image null	Pass	Verifica che l'immagine nulla sia consentita

AdminRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo	Nuovo Admin con email="admin@test.com" e username="adminUser"	Admin salvato correttamente e recuperabile tramite ID	Admin persistito nel database	Pass	Verifica il salvataggio di un Admin
2	Admin salvato nel database	Aggiornamento: username="updatedUser", resetToken="updated-Token", invitationToken="invite-updated"	Admin aggiornato correttamente	Database aggiornato con i nuovi valori	Pass	Verifica l'aggiornamento dei campi di un Admin
3	Admin salvato nel database	Eliminazione tramite ID	Admin non più presente	Record Admin eliminato	Pass	Verifica la cancellazione di un Admin
4	Admin salvato nel database	Ricerca tramite username="adminUser"	Admin trovato correttamente	Database invariato	Pass	Verifica la ricerca di un Admin tramite username
5	Admin salvato nel database	Ricerca tramite username="usernameInesistente"	Nessun Admin trovato	Database invariato	Pass	Verifica la ricerca per username inesistente
6	Admin salvato nel database	Ricerca tramite resetToken="reset-token-base"	Admin trovato correttamente	Database invariato	Pass	Verifica la ricerca tramite resetToken

7	Admin salvato nel database	Ricerca tramite resetToken="inesistente-token"	Nessun Admin trovato	Database invariato	Pass	Verifica la ricerca per resetToken inesistente
8	Admin salvato nel database	Ricerca tramite invitationToken="invitation-token-base"	Admin trovato correttamente	Database invariato	Pass	Verifica la ricerca tramite invitationToken
9	Admin salvato nel database	Ricerca tramite invitationToken="inesistente-token"	Nessun Admin trovato	Database invariato	Pass	Verifica la ricerca per invitationToken inesistente

AssignmentRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Nessun Assignment presente	title="Titolo1", description="Descrizione1", team="Team" con studenti ["A","B","C"]	Assignment salvato correttamente e recuperabile tramite ID	Assignment salvato correttamente nel database	Pass	Verifica il salvataggio di un Assignment
2	Assignment salvato nel database	title="Titolo_Aggiornato", description="Descrizione aggiunta"	Assignment aggiornato correttamente	Database aggiornato con i nuovi valori	Pass	Verifica l'aggiornamento dei campi di un Assignment
3	Assignment salvato nel database	ID dell'Assignment da eliminare	Assignment non più presente	Record Assignment eliminato	Pass	Verifica la cancellazione di un Assignment
4	Assignment salvato nel database	teamId = ID di "Team"	Lista contenente gli Assignment associati a quel team	Database invariato	Pass	Verifica la ricerca di Assignment tramite team ID
5	Nessun Assignment con team ID	teamId = 999L	Lista vuota	Database invariato	Pass	Verifica il comportamento per team ID non esistente
6	Assignment salvato su più team	teamIds = [ID "Team", ID "AltroTeam"]	Lista contenente gli Assignment dei due team	Database invariato	Pass	Verifica la ricerca di Assignment tramite lista di team ID
7	Nessun Assignment con team ID	teamIds = [111L, 222L]	Lista vuota	Database invariato	Pass	Verifica il comportamento per lista di team ID non esistente
8	Assignment salvato nel database	teamName = "Team"	Lista contenente gli Assignment associati a quel team	Database invariato	Pass	Verifica la ricerca di Assignment tramite team name
9	Nessun Assignment con team name	teamName = "None"	Lista vuota	Database invariato	Pass	Verifica il comportamento per team name inesistente
10	Assignment salvato nel database	title = "TitoloSpeciale"	Assignment trovato correttamente	Database invariato	Pass	Verifica la ricerca di Assignment tramite title
11	Nessun Assignment con title	title = "TitoloCheNonEsiste"	Nessun Assignment trovato	Database invariato	Pass	Verifica il comportamento per title inesistente

ScalataRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo con Admin persistito (" admin@test.com ").	Scalata(Name="Scalata1", Levels=5, Admin=admin).	La Scalata viene salvata correttamente e tutti i campi (nome, livelli, admin) corrispondono all'inserimento.	Scalata persistita nel database.	Pass	Verifica il salvataggio di una Scalata.
2	Scalata ("Scalata1") salvata nel DB.	Modifica oggetto: Description="Descrizione Aggiornata", NumLevels=10.	La Scalata recuperata dal database mostra la descrizione e il numero di livelli aggiornati.	Il record nel DB ha descrizione e livelli aggiornati.	Pass	Verifica l'aggiornamento dei campi di una Scalata.
3	Scalata ("Scalata1") salvata nel DB.	ID della Scalata ("Scalata1").	La ricerca della Scalata dopo l'operazione di cancellazione non produce alcun risultato.	Record Scalata rimosso dal database.	Pass	Verifica la cancellazione di una Scalata.
4	Scalata salvata e associata all'Admin "adminUser".	Input query: username="adminUser".	Viene restituita una lista contenente la Scalata collegata specificamente all'utente "adminUser".	Database invariato.	Pass	Verifica la ricerca di Scalata tramite Username dell'Admin.
5	Scalata salvata e associata all'Admin " admin@test.com ".	Input query: email="admin@test.com" .	Viene restituita una lista contenente la Scalata collegata specificamente all'email " admin@test.com ".	Database invariato.	Pass	Verifica la ricerca di Scalata tramite Email dell'Admin.
6	Database attivo.	1. Salvataggio: Scalata1 (Level=5), Scalata2 (Level=10). 2. Input query: findByNumLevels(5) e findByNumLevels(10).	La prima ricerca restituisce solo la Scalata con 5 livelli; la seconda restituisce solo quella con 10 livelli.	Database invariato.	Pass	Verifica il filtraggio delle Scalate per numero di livelli.

TeamRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo con Admin persistito (" admin@test.com ").	Team (Name="Team1", Students=["A", "B", "C"]) associato all'Admin.	Team salvato correttamente	Team persistito nel database.	Pass	Verifica il salvataggio di un Team con lista di studenti.
2	Team ("Team1") salvato nel DB.	Modifica oggetto: Name="TeamUpdated", NumStudents=5.	Team aggiornato correttamente	Il record nel DB ha nome e numero studenti aggiornati.	Pass	Verifica l'aggiornamento dei campi di un Team.
3	Team salvato nel DB.	Long id del Team da eliminare.	Team eliminato correttamente	Record Team rimosso dal database.	Pass	Verifica la cancellazione di un Team.
4	Team ("Team1") salvato nel DB.	1. Input: existsByName("Team1") 2. Input: existsByName("UnknownTeam")	1. Restituisce true 2. Restituisce false	Database invariato.	Pass	Verifica il controllo di esistenza per nome (Positivo e Negativo).
5	Team salvato e associato all'Admin " admin@test.com ".	Input query: findByAdmin_Email("admin@test.com") .	Restituisce una lista contenente il Team dell'admin specificato.	Database invariato.	Pass	Verifica la ricerca di Team tramite Email dell'Admin.

6	Team ("Team1") salvato nel DB.	Input query: <code>findByName("Team1")</code> .	Restituisce il Team "Team1".	Database invariato.	Pass	Verifica la ricerca di un Team tramite il suo nome.
7	Team ("Team1") associato all'Admin " admin@test.com ".	Input query: <code>email="admin@test.com", name="Team1"</code> .	Restituisce il Team corretto.	Database invariato.	Pass	Verifica la ricerca combinata per Email dell'Admin e Nome del Team.
8	Team salvato con lista studenti ["A", "B", "C"].	1. Query: <code>findById("B")</code> 2. Query: <code>findById("Z")</code>	1. Restituisce Team presente. 2. Restituisce Optional vuoto.	Database invariato.	Pass	Verifica la ricerca di un Team contenente uno specifico ID studente nella lista.

ClassUTRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo	<code>name="BaseClass", description="Base description", uri=null, difficulty=null, date=oggi</code>	ClassUT salvata correttamente e recuperabile tramite ID	ClassUT persistita nel database	Pass	Verifica il salvataggio di una ClassUT
2	Database Attivo	<code>name="ClassWithCategory", description="Desc", category="CategoriaBase"</code>	ClassUT salvata correttamente con la categoria associata	ClassUT persistita nel database	Pass	Verifica il salvataggio di una ClassUT con categoria
3	ClassUT salvata nel database	<code>description="Descrizione aggiornata", uri="http://updated-url.com", difficulty=HARD, date=oggi+2</code>	ClassUT aggiornata correttamente	Database aggiornato con i nuovi valori	Pass	Verifica l'aggiornamento dei campi di una ClassUT
4	ClassUT salvata nel database	name della ClassUT da eliminare	ClassUT non più presente	Record ClassUT eliminato	Pass	Verifica la cancellazione di una ClassUT
5	ClassUT salvata nel database	<code>nameContains="BASE", descriptionContains="TEST"</code>	Lista contenente classUT1 e classUT2	Database invariato	Pass	Verifica la ricerca tramite name o description ignorando case
6	ClassUT salvata nel database	<code>difficulty=EASY</code>	Lista contenente solo ClassUT con difficoltà EASY	Database invariato	Pass	Verifica il filtraggio per difficoltà
7	ClassUT salvata nel database	ordina per date asc	Lista ordinata dalla data più vecchia alla più recente	Database invariato	Pass	Verifica l'ordinamento per data
8	ClassUT salvata nel database	ordina per name asc	Lista ordinata alfabeticamente per nome	Database invariato	Pass	Verifica l'ordinamento per nome
9	ClassUT salvata nel database	<code>nameContains="PROG", difficulty=EASY</code>	Lista contenente solo ClassUT1	Database invariato	Pass	Verifica la ricerca per name contenente testo e difficoltà
10	ClassUT salvata nel database	<code>categoryName="CategoriaA"</code>	Lista contenente solo ClassUT appartenente a quella categoria	Database invariato	Pass	Verifica la ricerca tramite nome categoria
11	ClassUT salvata nel database	<code>text="Prog", categoryName="CategoriaA"</code>	Lista contenente solo ClassUT1	Database invariato	Pass	Verifica la ricerca e filtro combinato per testo e categoria
12	ClassUT salvata nel database	ordina per date (metodo <code>orderByDate</code>)	Lista ordinata dalla data più vecchia alla più recente	Database invariato	Pass	Verifica l'ordinamento tramite metodo <code>orderByDate</code>
13	ClassUT salvata nel database	ordina per name (metodo <code>orderByName</code>)	Lista ordinata alfabeticamente per nome	Database invariato	Pass	Verifica l'ordinamento tramite metodo <code>orderByName</code>

InteractionRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo	admin="admin@test.com", class="TestClass", Interaction(LIKE)	Oggetto salvato e recuperato identico	Interaction persistita	Pass	Verifica il salvataggio di un'Interaction
2	Interaction salvata	Modifica campi: type=REPORT, desc="Descrizione aggiornata"	Oggetto recuperato con dati aggiornati	Database aggiornato	Pass	Verifica l'aggiornamento dei campi
3	Interaction salvata	ID dell'interaction da eliminare	Interaction eliminata	Record eliminato	Pass	Verifica la cancellazione di un'Interaction
4	Database attivo	Setup: 1 LIKE, 1 REPORT. Query: findByType(LIKE)	Lista contenente solo interazioni di tipo LIKE	Database invariato	Pass	Verifica il filtraggio per tipo
5	Database attivo	Setup: 2 LIKE, 1 REPORT su "TestClass". Query: count(LIKE) e count(REPORT)	count(LIKE) = 2 count(REPORT) = 1	Database invariato	Pass	Verifica i conteggi multipli per classe e tipo
6	Database attivo	classUTName="TestClass", type=REPORT (nessuna interazione presente)	Restituisce 0	Database invariato	Pass	Verifica il conteggio su dati inesistenti

OperationRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo	adminEmail="admin@test.com", classUTName="JavaBasics", type=UPLOAD, date=oggi	Operation salvata correttamente e recuperabile tramite ID	Operation persistita nel database	Pass	Verifica il salvataggio di un'Operation
2	Operation salvata nel DB	Modifica oggetto: type=DELETE	Operation aggiornata correttamente	Il record nel DB ha type=DELETE	Pass	Verifica l'aggiornamento del tipo di un'Operation
3	Operation salvata	Modifica oggetto: date=Oggi - 5 giorni	Data dell'Operation aggiornata correttamente	Il record nel DB ha la data aggiornata	Pass	Verifica l'aggiornamento della data
4	Operation salvata nel DB	ID dell'operation	Operation eliminata	Record Operation eliminato	Pass	Verifica la cancellazione di un'Operation
5	Operation salvata nel DB	ID dell'operation	Operation eliminata, Admin e ClassUT ancora presenti	Operation rimossa, ma Admin e ClassUT persistono	Pass	Verifica che la cancellazione non elimini Admin o ClassUT
6	Operation salvata nel DB	ID=operation salvata	Operation trovata correttamente	Database invariato	Pass	Verifica la ricerca di Operation per ID
7	Database attivo e nessuna Operation con ID 999	ID=999	Nessuna Operation trovata	Database invariato	Pass	Verifica la ricerca di Operation non esistente
8	Database attivo	Due operation: op1 type=UPLOAD, op2 type=UPDATE	Lista di tutte le operation con size=2	Database invariato	Pass	Verifica la ricerca di tutte le Operation

OpponentRepository

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Database attivo con ClassUT ("TestClass").	Opponent(Type="SIMPLE", Jacoco, Evosuite) associato alla ClassUT.	Opponent salvato correttamente, ID non nullo	Opponent persistito nel database.	Pass	Verifica il salvataggio di un Opponent con i relativi punteggi.
2	Opponent (Type="SIMPLE") salvato nel DB.	Modifica oggetto: Type="ADVANCED".	Opponent aggiornato correttamente	Il record nel DB ha il tipo aggiornato.	Pass	Verifica l'aggiornamento del tipo di un Opponent.
3	Opponent salvato nel DB.	Long id dell'Opponent da eliminare.	Opponent eliminato	Record Opponent rimosso dal database.	Pass	Verifica la cancellazione di un Opponent.
4	Opponent salvato nel DB.	Long id dell'Opponent salvato.	Opponent trovato tramite findById	Database invariato.	Pass	Verifica la ricerca di Opponent tramite ID.
5	Database attivo.	Salvataggio di 2 Opponents distinti. Input metodo: findAllOpponents().	findAllOpponents restituisce lista di size=2	Database invariato.	Pass	Verifica il recupero di tutti gli Opponents salvati.
6	Opponent salvato (Class="TestClass", Type="SIMPLE", Diff=EASY).	Input query: class="TestClass", type="SIMPLE", diff=EASY.	Opponent trovato	Database invariato.	Pass	Verifica la ricerca personalizzata per Classe, Tipo e Difficoltà.
7	Opponent salvato con coverage="coverage_data".	Input query: class="TestClass", type="SIMPLE", diff=EASY.	Coverage restituita = "coverage_data"	Database invariato.	Pass	Verifica il recupero della stringa di coverage.
8	Opponent salvato con JacocoScore (Instruction=80).	Input query: class="TestClass", type="SIMPLE", diff=EASY.	JacocoScore restituita, instructionCoverage=80	Database invariato.	Pass	Verifica il recupero specifico del JacocoScore.
9	Opponent salvato con EvosuiteScore (Line=90).	Input query: class="TestClass", type="SIMPLE", diff=EASY.	EvosuiteScore restituita, lineCoverage=90	Database invariato.	Pass	Verifica il recupero specifico dell'EvosuiteScore.

6.2 Integration testing

6.2.1 Postman

UploadSuggestions

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	La classe "Calcolatrice" è già stata caricata nel database.	<pre>{ "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggestimento 1", "level": "LOW" }, { "order": 2, "hint": "Testo Suggestimento 2", "level": "MEDIUM" }] }</pre>	200 OK	I suggerimenti sono stati aggiunti nel database.	Pass	Upload di un file json contenente due suggerimenti senza errori.

		<pre>] } </pre>				
2	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "hint": "Testo Suggestimento 1", "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.		Upload di un file json contenente un suggerimento privo del campo "order"
3	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggestimento 1", "level": "LOW" }, { "order": 3, "hint": "Testo Suggestimento 2", "level": "MEDIUM" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di due suggerimenti con il campo "order" non progressivo
4	{ La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 2, "hint": "Testo Suggestimento 1", "level": "LOW" }, { "order": 3, "hint": "Testo Suggestimento 2", "level": "MEDIUM" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di due suggerimenti con il campo "order" che non parte da 1
5	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento senza campo "hint"

6	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "", "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con campo "hint" vuoto
7	{ La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento senza campo "level"
8	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con campo "level" mancante
9	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "EXTREME" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con campo "level" con valore non ammissibile
10	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "LOW", "newField": "newValue" }] } </pre>	200 OK	Il suggerimento viene salvato ignorando il campo aggiuntivo	Pass	Upload di un suggerimento con campo aggiuntivo non previsto

11	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con formato json errato (parentesi mancante)
12	-	<pre> { "className": "ClasseInesistente", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "LOW" }] } </pre>	404 Not Found	Nessun suggerimento salvato.	Pass	Upload di un suggerimento riferito a una classe inesistente
13	-	<pre> { } </pre>	400 Bad Request		Pass	Upload di un file json vuoto
14	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [] } </pre>	200 OK	Non accade nulla	Pass	Upload di un file json con lista di suggerimenti vuota
15	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "AAA...A, "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con campo "hint" che supera i 255 caratteri
15	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "AAA...A, "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con campo "hint" che supera i 255 caratteri

16	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": ["lista", "non", "valida"], "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con un campo che presenta un valore non ammissibile (lista nel campo "hint" al posto di una stringa)
17	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "AAA...A, "level": "LOW" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con campo "hint" che supera i 255 caratteri
18	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "low" }] } </pre>	400 Bad Request	Nessun suggerimento salvato.	Pass	Upload di un suggerimento con campo "level" che presenta un valore ammissibile ma minuscolo
19	La classe "Calcolatrice" è già stata caricata nel database.	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "LOW" }, { "order": 2, "hint": "Testo Suggerimento 2", "level": "low" }] } </pre>	400 Bad Request	Nessuno dei due suggerimenti viene inserito	Pass	Upload di un file contenente un primo suggerimento valido e un secondo suggerimento non valido al fine di verificare l'atomicità della transazione
20	La classe "Calcolatrice" è già stata	<pre> { "className": "Calcolatrice", "suggestions": [</pre>	200 OK	Il suggerimento	Pass	Upload di un suggerimento contenente

caricata nel database.	<pre> { "order": 1, "hint": "Testo Suggerimento 1", "level": "LOW" }]</pre>		viene inserito senza spazi		oltre al testo degli spazi nel campo "hint"
------------------------	--	--	----------------------------	--	---

GetSuggestions

GET /opponents/suggestions/{className}

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	La classe "Calcolatrice" è già stata caricata nel database senza suggerimenti.		200 OK	<pre> { "className": "Calcolatrice", "suggestions": [] }</pre>	Pass	Get dei suggerimenti di una classe senza suggerimenti nel database.
2	La classe "Calcolatrice" è già stata caricata nel database con due suggerimenti: - Il primo con order = 1, hint = "Testo Suggerimento 1", level = "LOW" e nessuna immagine associata. - Il secondo con order = 2, hint = "Testo Suggerimento 2", level = "MEDIUM", nessuna immagine associata		200 OK	<pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "date": "2025- 12-14", "image": null, "level": "LOW" }, { "order": 2, "hint": "Testo Suggerimento 2", "date": "2025- 12-14", "image": null,</pre>	Pass	Get dei suggerimenti di una classe con due suggerimenti.

				<pre> "level": "MEDIUM" }] } </pre>		
--	--	--	--	--	--	--

DeleteSuggestion

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	È stato caricato il seguente file json: <pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggestimento 1", "level": "LOW" }] } </pre>	DELETE {{baseUrl}}/opponents/suggestions/Calcolatrice/1	200 OK	Il suggerimento viene cancellato	Pass	Delete di un suggerimento esistente di una classe esistente
2	È stato caricato il seguente file json: <pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggestimento 1", "level": "LOW" }] } </pre>	DELETE {{baseUrl}}/opponents/suggestions/ClassInesistente/1	404 Not Found	Nessun suggerimento viene cancellato	Pass	Delete di un suggerimento esistente di una classe non esistente

3	È stato caricato il seguente file json: <pre> { "className": "Calcolatrice", "suggestions": [{ "order": 1, "hint": "Testo Suggerimento 1", "level": "LOW" }] } </pre>	DELETE {{baseUrl}}/opponents/suggestions/Calcolatrice/2	404 Not Found	Nessun suggerimento viene cancellato	Pass	Delete di un suggerimento non esistente di una classe esistente
---	---	--	---------------	--------------------------------------	------	---

UploadGuidelines

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	-	<pre> [{ "order": 1, "hint": "Testo Suggerimento 1", }, { "order": 2, "hint": "Testo Suggerimento 2" }] </pre>	200 OK	Le linee guida sono state aggiunte nel database.	Pass	Upload di un file json contenente due linee guida senza errori.
2	-	<pre> [{ "hint": "Testo Suggerimento 1" }] </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un file json contenente una linea guida priva del campo "order"
3	-	<pre> [{ "order": 1, "hint": "Testo Suggerimento 1" }, { "order": 3, </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un file json contenente due linee guida con il campo

		<pre> "hint": "Testo Suggerimento 2" }] </pre>				"order" non progressivo
4	-	<pre> [{ "order": 2, "hint": "Testo Suggerimento 1" }, { "order": 3, "hint": "Testo Suggerimento 2" }] </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un file json contenente due suggerimenti con il campo "order" che non parte da 1
5	-	<pre> [{ "order": 1 }] </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un file json contenente una linea guida senza campo "hint"
6	{ -	<pre> [{ "order": 1, "hint": "" }] </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un file json contenente una linea guida con campo "hint" vuoto
7	-	<pre> [{ "order": 1, "hint": "Testo Suggerimento 1", "newField": "newValue" }] </pre>	200 OK	La linea guida viene salvata ignorando il campo aggiuntivo	Pass	Upload di un file json contenente una linea guida con un campo aggiuntivo non previsto
8	-	<pre> [{ "order": 1, "hint": "Testo Suggerimento 1" }] </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un file json con formato errato (parentesi mancante)
9	-	<pre> [] </pre>	200 OK	Non succede nulla	Pass	Upload di un file json contenente una lista vuota
10	-	<pre> [{ "order": 1, "hint": "AAA...A" }] </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un file json contenente una linea

		]				guida con campo "hint" che supera i 255 caratteri
11	-	[<pre> { "order": 1, "hint": ["lista", "non", "valida"] }] </pre>	400 Bad Request	Nessuna linea guida aggiunta.	Pass	Upload di un suggerimento con un campo che presenta un valore non ammissibile (lista nel campo "hint" al posto di una stringa)
12	-	[<pre> { "order": 1, "hint": "Testo Suggerimento 1" }, { "order": 2, "hint": ["Testo", "Suggerimento", "2"] }] </pre>	400 Bad Request	Nessuna delle due linee guida viene inserita	Pass	Upload di un file json contenente una prima linea guida valida e una seconda non valida al fine di verificare l'atomicità della transazione
13	-	[<pre> { "order": 1, "hint": " Testo Suggerimento 1 " }] </pre>	200 OK	Il suggerimento viene inserito senza spazi	Pass	Upload di un suggerimento contenente oltre al testo degli spazi nel campo "hint"

GetGuidelines

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Nessuna linea guida è stata caricata.		200 OK	[]	Pass	Get delle linee guida, senza aver caricato nessuna linea guida nel sistema.

2	Sono state caricate le seguenti linee guida nel sistema: - Il primo con order = 1, hint = “Testo Linea Guida 1” e nessuna immagine associata. - Il secondo con order = 2, hint = “Testo Linea Guida 2” e nessuna immagine associata		200 OK	<pre>[{ "order": 1, "hint": "Testo Suggestimento 1", "date": "2025-12-14", "image": null }, { "order": 2, "hint": "Testo Suggestimento 2", "date": "2025-12-14", "image": null }]</pre>	Pass	Get delle linee guida dopo aver caricato due linee guida nel sistema.
---	---	--	--------	---	------	---

DeleteGuideline

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	È stato caricato il seguente file json: <pre>{ [{ "order": 1, "hint": "Testo Suggestimento 1" }] }</pre>	DELETE {{baseUrl}}/opponents/guidelines/1	200 OK	La linea guida viene cancellata	Pass	Delete di una linea guida esistente
2	È stato caricato il seguente file json: <pre>{ [{ "order": 1, "hint": "Testo Suggestimento 1" }] }</pre>	DELETE {{baseUrl}}/opponents/guidelines/2	404 Not Found	Nessuna linea guida viene cancellata	Pass	Delete di una linea guida non esistente

UploadSuggestionImage

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Sono stati caricati la classe Calcolatrice e il suggerimento 1. La dimensione dell'immagine è inferiore ai 5 Mb.	POST{{baseUrl}}/opponents/suggestions/Calcolatrice/1	200 OK	L'immagine viene caricata correttamente	Pass	Upload di un'immagine per un suggerimento esistente di una classe esistente di dimensioni inferiori ai 5 Mb
2	Sono stati caricati la classe Calcolatrice e il suggerimento 1. La dimensione dell'immagine è inferiore ai 5 Mb.	POST{{baseUrl}}/opponents/suggestions/Calcolatrice/2	404 Not Found	Nessuna immagine caricata	Pass	Upload di un'immagine per un suggerimento non esistente di una classe esistente di dimensioni inferiori ai 5 Mb
3	Sono stati caricati la classe Calcolatrice e il suggerimento 1. La dimensione dell'immagine è inferiore ai 5 Mb.	POST{{baseUrl}}/opponents/suggestions/ClassInesistente/1	404 Not Found	Nessuna immagine caricata	Pass	Upload di un'immagine per un suggerimento esistente di una classe non esistente di dimensioni inferiori ai 5 Mb
4	Sono stati caricati la classe Calcolatrice e il suggerimento 1. La dimensione dell'immagine è superiore ai 5 Mb.	POST{{baseUrl}}/opponents/suggestions/Calcolatrice/1	413 Payload Too Large	Nessuna immagine caricata	Pass	Upload di un'immagine per un suggerimento esistente di una classe esistente di dimensioni superiori ai 5 Mb

UploadGuidelineImage

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	È stata caricata la linea guida 1. La dimensione dell'immagine è inferiore ai 5 Mb.	POST{{baseUrl}}/opponents/guidelines/1	200 OK	L'immagine viene caricata correttamente	Pass	Upload di un'immagine per una linea guida esistente di dimensioni inferiori ai 5 Mb
2	È stata caricata la linea guida 1. La dimensione dell'immagine è inferiore ai 5 Mb.	POST{{baseUrl}}/opponents/guidelines/2	404 Not Found	Nessuna immagine caricata	Pass	Upload di un'immagine per una linea guida non esistente di dimensioni inferiori ai 5 Mb
3	È stata caricata la linea guida 1. La dimensione dell'immagine è superiore ai 5 Mb.	POST{{baseUrl}}/opponents/guidelines/1	413 Payload Too Large	Nessuna immagine caricata	Pass	Upload di un'immagine per una linea guida esistente di dimensioni superiori ai 5 Mb

DeleteSuggestionImage

ID	Preconditi on	Input	Expect ed Output	Postconditi on	Resu lt	Descrizio ne
1	Sono stati caricati la classe Calcolatrice, il suggerimento 1 e la relativa immagine	DELETE {{baseUrl}}/opponents/suggestions/Calcolatrice/1	200 OK	L'immagine viene cancellata correttamente	Pass	Delete di un'immagine per un suggerimento esistente di una classe esistente

2	Sono stati caricati la classe Calcolatrice, il suggerimento 1 e la relativa immagine	DELETE{{baseUrl}}/opponents/suggestions/Calcolatrice/2	404 Not Found	Nessuna immagine cancellata	Pass	Delete di un'immagine per un suggerimento non esistente di una classe esistente
3	Sono stati caricati la classe Calcolatrice, il suggerimento 1 e la relativa immagine	DELETE{{baseUrl}}/opponents/suggestions/Classelnesistente/1	404 Not Found	Nessuna immagine cancellata	Pass	Delete di un'immagine per un suggerimento esistente di una classe non esistente

DeleteGuidelineImage

ID	Precondition	Input	Expected Output	Postcondition	Result	Descrizione
1	Sono state caricate la linea guida 1 e la relativa immagine.	DELETE{{baseUrl}}/opponents/guidelines/1	200 OK	L'immagine viene cancellata correttamente	Pass	Delete di un'immagine per una linea guida esistente
2	Sono state caricate la linea guida 1 e la relativa immagine.	DELETE{{baseUrl}}/opponents/guidelines/2	404 Not Found	Nessuna immagine cancellata	Pass	Delete di un'immagine per una linea guida non esistente

7. Possibili integrazioni

Una possibilità futura è quella di bloccare le operazioni di modifica dei suggerimenti relativi alle classi utilizzate durante una partita in corso, in modo tale da preservare l'integrità dei dati. Ovviamente per tale implementazione, in altri microservizi dovrà essere aggiunto un componente che possa tener traccia proprio delle partite in corso.